

ゲームデザインの基礎

Fundamentals of Game Design 講義テキスト

Ernest Adams 著（講義用日本語要約版）

第1回：ゲームとビデオゲーム — ゲームデザインの基礎

授業時間: 90分 **対応原著:** Ernest Adams 『Fundamentals of Game Design』 第2版 — Introduction / Chapter 1 **学習目標:** ゲームの本質的要素を理解し、ビデオゲームが従来のゲームとどう異なるかを説明できるようになる

1. はじめに — この授業で学ぶこと

ゲームデザインを学ぶにあたって、まず「ゲームとは何か」を明確にする必要がある。私たちは日常的に「ゲーム」という言葉を使うが、その意味するところは驚くほど広い。本講では、ゲームの構成要素を分解し、形式的な定義を確立する。そのうえで、コンピュータがゲームに何をもたらしたのか、ビデオゲームがどのように人を楽しませるのかを整理する。

本授業の原著 Fundamentals of Game Design は、ゲームデザインに特化した実践的な教科書である。プログラミング、アート、音楽、プロジェクト管理は扱わない。デザイナーとしての思考法と設計原則を身につけることが目的である。

2. ゲームとは何か

2.1 遊び (Play) の出発点

ゲームは人間の「遊びたい」という欲求と「ふりをする (pretend)」能力から生まれる。

- ・ **遊び (Play)** : 必須ではないが社会的に意味のある活動全般を指す、広い概念
- ・ **ふり (Pretending)** : 現実とは異なる「想像上の現実」を心の中につくる能力。
つくり手が自由に創造・放棄・変更できる

2.2 おもちゃ・パズル・ゲームの違い

遊びの対象は、**ルール**と**目標 (ゴール)**の有無によって3つに分類できる。

種類	ルール	目標	例
おもちゃ (Toy)	なし	なし	ボール、人形
パズル (Puzzle)	目標を定義する1つだけ	あり	ジグソーパズル
ゲーム (Game)	複数のルール	あり	チェス、サッカー

おもちゃには「正しい遊び方」がなく、子供は意図された用途を覆して遊ぶことすら楽しむ。パズルにはゴールがあるが、ゴールへの到達方法を制限するルールはほとんどない。ゲームはルールとゴールの両方を持ち、最も構造化された遊びである。

2.3 ゲームの定義

ゲーム (Game) : 想像上の現実 (pretended reality) の文脈で行われる遊び活動の一種であり、参加者がルールに従って行動しながら、少なくとも1つの恣意的かつ非自明な目標の達成を目指すもの。

この定義は厳密な数学的定義ではなく、大多数のケースをカバーする実用的な記述である。Salen & Zimmerman の Rules of Play、Raph Koster の A Theory of Fun など多くの先行研究が存在するが、いずれもルール・目標・遊び・ふりの要素に言及している点は共通している。

キーポイント： ゲームの本質的要素は **ルール、目標、遊び、ふり（pretending）** の4つである。

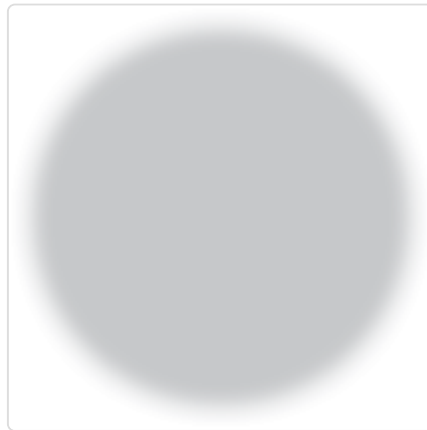
3. ゲームの本質的要素

3.1 遊び — 参加型の娯楽

遊びは**参加型（participatory）**の娯楽である。本や映画が**提示型（presentational）**であるのと対照的に、ゲームではプレイヤー自身が選択を行い、展開に影響を与える。毎回異なる選択ができ、異なる体験が得られる。ただし選択の自由はルールによって制約され、その制約の中で巧みに振る舞うことが求められる。

3.2 ふりと「マジックサークル」

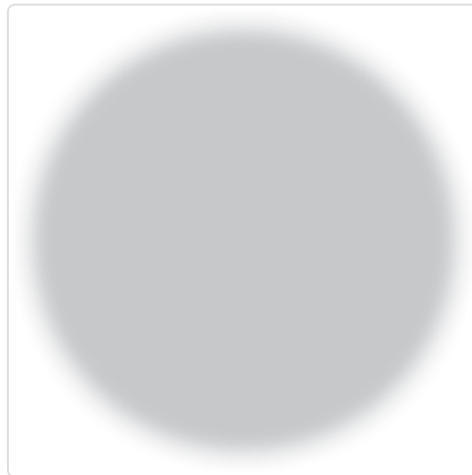
マジックサークル（Magic Circle）とは、ゲーム内で意味を持つ事柄と現実世界で意味を持つ事柄を隔てる境界のことである。オランダの歴史家ヨハン・ホイジンガ（Johan Huizinga）が Homo Ludens（1938年）で提唱した概念を基にしている。



マジックサークルの概念図 — ゲームの概念・状況・出来事と現実世界を分離する境界

プレイヤーはゲームに参加することでマジックサークルに入る。マルチプレイヤーゲームでは、全員が同じルールを受け入れる「合意」によってマジックサークルが成立する。

サッカーを考えてみよう。現実世界では「ボールをネットに蹴り入れる」ことに本質的な意味はない。しかしマジックサークルの中では、それは「得点」という重要な意味を持つ。ルールを受け入れ、意味を付与するという行為自体が「ふり」の一種である。



現実世界の行為がマジックサークル内で特別な意味を持つ例

マジックサークルが曖昧になる場合もある。古代メソアメリカの球技では敗者が生贄にされたとされ、ゲームの結果が現実世界の生死に直結していた。ギャンブルも、実際の金銭を賭けることでマジックサークルの境界を曖昧にする。

3.3 目標（ゴール）

ゲームには目標が必要である。目標のない遊びは「ゲームプレイ」ではない。ただし、Salen & Zimmermanが主張するような「定量化可能な結果」は必須ではない。SimCityのように明確な終了条件がなく、破産しない限り無限に続くゲームも存在する。初期のアーケードゲーム（Space Invaders など）では、そもそも達成不可能な目標が設定されていた。

- ・ **勝利条件（Victory Condition）**：勝者が決まる明確な状況（例：チェスのチェックメイト）
- ・ **敗北条件（Loss Condition）**：敗者が決まる状況。RollerCoaster Tycoon は負けることはあるが勝つことはできない
- ・ **終了条件（Termination Condition）**：ゲームが終わるルール

目標は非自明（nontrivial）でなければならない。何らかのチャレンジ要素が含まれていなければ、それはゲームとは呼べない。

3.4 ルール

ルールとは、プレイヤーがゲームの間に受け入れる定義と指示のことである。ルールが規定する主な内容は以下の通り。

- ・ **記号論 (Semiotics)** : ゲーム内のシンボルの意味と関係 (例: 野球の「イニング」「アウト」)
- ・ **ゲームプレイ**: プレイヤーに提示されるチャレンジとアクション
- ・ **プレイの順序 (Sequence of Play)** : ゲームを構成する活動の進行
- ・ **ゲームの目標**: ルールによって定義される
- ・ **終了条件**: ゲームを終了させる条件
- ・ **メタルール**: ルールについてのルール (ルール変更の手続きなど)

ルールは恣意的で構わないが、**曖昧であってはならず**、相互に矛盾してはいけない。

キーポイント: ルールはマジックサークル内でのみ意味を持つ定義と指示であり、デザイナーは必要に応じて自由に概念を発明できる。ただし、プレイヤーに理解可能でなければならない。

4. ゲームプレイの定義

4.1 チャレンジとアクション

- ・ **チャレンジ (Challenge)** : プレイヤーに課される非自明な課題。精神的または身体的な努力を要する
 - 固有のチャレンジ (1回限り)、繰り返しのチャレンジ、継続的チャレンジに分類できる
 - 例: アクションゲームで雑魚敵を倒す (繰り返し) → ボス敵を倒す (固有)
- ・ **アクション (Action)** : ルールがプレイヤーに許可する行為。ビデオゲームでは、ソフトウェアに組み込まれたアクションのみが可能

4.2 ゲームプレイの定義

ゲームプレイ (Gameplay) : 1. ゲームの目標に到達するためにプレイヤーが直面するチャレンジ 2. それらのチャレンジに対処するためにプレイヤーに許可されるアクション この2つの組み合わせである。

Sid Meierの「一連の興味深い選択」、Dino Diniの「楽しませるインタラクション」という有名な定義もあるが、本書ではチャレンジとアクションの関係性としてゲームプレイを捉える。デザイナーはこの2つを常にセットで考えなければならない。

キーポイント: ゲームプレイはチャレンジとアクションの組み合わせであり、ゲームデザインの核心である。

5. 公平性・対称性・競争と協力

5.1 公平性 (Fairness)

プレイヤーはルールがゲームの公平性を保証することを期待する。公平性はマジックサークルの外にある社会的メタルールであり、文化やプレイヤーによって基準が異なる。ビデオゲームではプレイヤーがルールを変更できないため、公平性の設計は特に重要である。

5.2 対称と非対称

- ・ **対称ゲーム (Symmetric)** : 全プレイヤーが同じルール・同じ勝利条件で遊ぶ
(例: バスケットボール、チェス)
- ・ **非対称ゲーム (Asymmetric)** : プレイヤーごとにルールや勝利条件が異なる
(例: 歴史的ウォーゲーム、Fox and Geese)

非対称ゲームでは公平性の確保が難しく、プレイヤーがルールを調整することも多い。

5.3 競争モード

モード	説明	例
2人対戦	1対1	チェス、バックギャモン
マルチプレイヤー競争	全員が敵	モノポリー、ポーカー
マルチプレイヤー協力	全員で同じ目標	LEGO Star Wars (2人協力)
チーム対戦	チーム内協力 + チーム間競争	サッカー、ブリッジ
シングルプレイヤー	個人 vs 状況	マリオシリーズ
ハイブリッド	協力と競争の混在	ディプロマシー

キーポイント： ゲームには必ずしも競争が必要ではない。協力型や創造型のゲームも存在する。競争モードの選択はマーケットを広げるが、設計の負荷も大きくなる。

6. 従来のゲームとビデオゲームの違い

コンピュータがゲームにもたらす重要な変化は以下の3つである。

6.1 ルールの隠蔽

従来のゲームではプレイヤー自身がルールを読み、実行し、遵守する必要があった。ビデオゲームではコンピュータがルールを実装・執行するため、プレイヤーはルールを意識せずにプレイできる。これにより、ゲームを「ルールのある人工的環境」ではなく「自分が参加している別世界」として体験でき、**没入感（immersion）が飛躍的に高まる**。

ただし、ルールが隠されていると最適な選択がわからないという欠点もある。試行錯誤でしか学べないゲームはフラストレーションを招く。

設計原則: 試行錯誤を避けよ。プレイヤーが問題の解決策を推論できるだけの手がかりを提供せよ。

6.2 ペースの制御

従来のゲームではプレイヤーがペースを決めるが、ビデオゲームではコンピュータがペースを制御する。高速でスリリングなゲームも、じっくり考える戦略ゲームも設計可能であり、緩急をつけることもできる。

6.3 ゲーム世界の提示

ビデオゲームはスクリーンとスピーカーを通じて、映像・アニメーション・音楽・効果音を駆使した架空の世界をプレイヤーに直接提示できる。従来のゲームでは主にプレイヤーの想像力に頼っていた部分を、視覚的・聴覚的に具現化できる点がビデオゲームの大きな強みである。

6.4 人工知能 (AI)

1959年にIBMのArthur Samuelがチェッカーをプレイするプログラムを開発して以来、AIはゲームに不可欠な技術となった。AIの主な用途は以下の通り。

- ・ **戦略計算:** 最適行動の決定
- ・ **経路探索 (Pathfinding):** 障害物のある空間での最適ルート探索
- ・ **自然言語処理/生成:** コマンド入力やNPCの会話
- ・ **パターン認識:** 音声認識、プレイヤー行動の分析
- ・ **シミュレートされた人物・生物:** 知的に振る舞うNPCの実現

キーポイント: コンピュータの最大の貢献は、ルールの実装をプレイヤーから解放し、没入感を高めたことである。さらにペース制御、世界の視覚化、AIによって、従来のゲームにはない多様な娯楽体験が可能になった。

7. ビデオゲームはどのように人を楽しませるか

7.1 ゲームプレイ

チャレンジとアクションそのものが娯楽の根幹である。「できるか?」という問いがテンションとドラマを生む。飛行機を操縦する、城を建てる、歌って踊るなど、現実では不可能な行為を体験できることもビデオゲームの大きな魅力である。

設計原則: ゲームプレイが最優先。ゲームデザインで最初に考えるべきことである。

主なチャレンジの種類と代表例:

カテゴリ	例
身体的協応（速度・精度・リズム）	Tetris, Need for Speed, Dance Dance Revolution
論理的推理	Minesweeper
パターン認識	Brain Age, Sonic the Hedgehog
時間制限	Frogger, IndyCar Racing
記憶・知識	You Don't Know Jack, Brain Age
探索	Doom, Ultima, Zork
紛争（戦略・戦術・生存）	Warcraft, Pac-Man, Space Invaders
経済	Civilization, SimEarth, The Sims
概念的推論	Planescape: Torment, The Incredible Machine
創造・構築	SimCity, The Sims

7.2 美的体験 (Aesthetics)

ビデオゲームは芸術形式であり、美的感覚を伴って設計されるべきである。アニメーションの滑らかさ、サウンドの質、UIのデザイン、物理挙動の自然さはすべて美的要素である。

設計原則: 醜いゲーム、ぎこちないゲームは、設計がどんなに革新的でも悪いゲームである。

7.3 ハーモニー (Harmony)

ゲームデザイナーのBrian Moriartyが提唱した概念で、**ゲームのすべての要素が一つの統一された全体に属している感覚**を意味する。ハーモニーはデザイン委員会やフォーカスグループからは生まれない。明確な意図に基づくデザイン判断から生まれる。不要な暴力表現、場違いなユーモア、ゲーム世界の外部への言及は、没入感を破壊する。

設計原則: すべてのデザイン判断が全体ビジョンと調和しているかを問い続けよ。

7.4 ストーリーテリング

ビデオゲームはルール実装の負担からプレイヤーを解放することで、物語体験とゲーム体験をほぼシームレスに融合できる。プレイヤーは物語の中にいて、その展開に影響を与えていると感じられる。これはビデオゲームが単なる新しい種類のゲームではなく、**まったく新しいメディア**である理由の一つである。

アドベンチャーゲームというジャンルは、「ゲーム」の形式的概念から離れ、**インタラクティブ・ストーリー**という新たなハイブリッドな娯楽形態へと進化しつつある。

7.5 リスクとリワード

リスク（不確実性）とリワード（報酬）のメカニズムは、ゲームプレイの興奮を高める。不確実性は、偶然（サイコロの目）、隠された情報（敵の位置）、相手の行動の予測不可能性などから生まれる。

設計原則: リスクには必ずリワードを伴わせよ。報酬なしにリスクを冒す動機はない。

7.6 新規性 (Novelty)

新しいものを見る・聞く・行う体験は人を楽しませる。現代のビデオゲームはゲーム進行に応じてゲームプレイ自体を変化させることができる（Battlefield 1942 では歩兵・戦車・航空機・船舶と視点を切り替えられる）。ただし、新規性だけでは持続的な興味は維持できない。「テーマと変奏」のアプローチ——新要素を導入し、次の要素に移る前に十分に探索させる——が効果的である。

7.7 学習

Raph Kosterは A Theory of Fun で、学習こそがゲームの楽しさの中核であると主張した。プレイヤーは敵の行動パターンを学び、新しい戦術を習得し、ゲームの仕組みを理解していく。学習し尽くすとゲームは退屈になり、プレイヤーは新しいゲームに移る。学習が楽しいのは、(1) 楽しい文脈で行われるとき、(2) 役立つ習熟をもたらすとき、の少なくとも一方が満たされる場合である。

7.8 創造的・自己表現的プレイ

人は設計したり、作ったり、カスタマイズしたりすることを愛する。自己表現の欲求は非常に強く、ゲームプレイ上不利な選択肢でも好みのものを選ぶプレイヤーは多い。研究によれば、女性プレイヤーは競争に勝つことよりも自己表現に動機づけられる傾向がある。

7.9 没入 (Immersion)

没入とは、外の世界を忘れ、ゲームの世界にのめり込む感覚である。コールリッジの「不信の自発的停止」に由来する概念だが、ゲーム業界では「マジックサークルの境界を見失う」ことを意味する。没入には3種類ある。

種類	説明	促進要因
戦術的没入	高速アクションに集中し、他のことを考	短時間で処理する小さなチャレンジの連続

種類	説明	促進要因
	える余裕がない状態 (Tetrisトランス)	
戦略的没入	観察・計算・計画に 深く集中する状態 (チェスの名手のよ うな状態)	ルールの明確な理解、計画可能な環 境
物語的没入	物語の中にいる感 覚。良い物語作り (魅力的なキャラク ター、興奮するプ ロット) が必要	ストーリーと矛盾しないゲームプレ イ

7.10 ソーシャル (社交)

ゲームは古来より社交活動であった。ビデオゲームでは以下のような形態がある。

- ・ **ローカルマルチプレイヤー**: 同じ場所で同じ画面を見てプレイ
- ・ **ネットワークプレイ**: インターネットを通じた分散型対戦
- ・ **LANパーティ**: 同じ部屋でLAN接続してプレイ
- ・ **グループプレイ**: 1人がコントローラーを持ち、他の人が見て助言する (ホットシート方式)

キーポイント: ビデオゲームの娯楽手段は多岐にわたる — ゲームプレイ、美的体験、ハーモニー、ストーリーテリング、リスクとリワード、新規性、学習、創造的プレイ、没入、ソーシャル。万人を喜ばせるゲームは存在しないため、ターゲットに合わせて娯楽要素を選択・設計すべきである。

8. 本講のまとめ

1. **ゲームの本質的要素**は「遊び」「ふり (pretending)」「目標」「ルール」の4つである

2. **マジックサークル**はゲーム世界と現実世界を隔てる概念的境界であり、プレイヤーの合意によって成立する
3. **ゲームプレイ**はチャレンジとアクションの組み合わせとして定義され、ゲームデザインの核心をなす
4. **コンピュータ**はルールの隠蔽、ペース制御、世界の視覚化、AIの提供を通じて、ゲーム体験を根本的に変革した
5. **ビデオゲームの娯楽要素**は10種類に分類でき、デザイナーはターゲットプレイヤーに合わせてこれらを適切に組み合わせる必要がある
6. **設計原則**: ゲームプレイ最優先、試行錯誤の回避、美的品質の確保、ハーモニーの追求、リスクには必ずリワードを

9. 振り返り問題

1. おもちゃ・パズル・ゲームの違いを、ルールと目標の観点から説明せよ。
2. 「マジックサークル」とは何か。サッカーの具体例を使って説明せよ。
3. ビデオゲームにおいてコンピュータがルールの隠蔽することの利点と欠点をそれぞれ述べよ。
4. 戦術的没入・戦略的没入・物語的没入の違いを、具体的なゲームタイトルを挙げて説明せよ。
5. 自分がよく遊ぶゲームを1つ選び、本講で学んだ10種類の娯楽要素のうちどれが含まれているかを分析せよ。

次回予告: 第2回「デザインの構成要素とプロセス」では、ビデオゲームの3大構成要素（コアメカニクス、ユーザーインターフェース、ストーリーテリングエンジン）と、プレイヤー中心設計のアプローチを学ぶ。

第2回：デザインの構成要素とプロセス

Fundamentals of Game Design Chapter 2 に基づく

1. ゲームデザインとは何か

ゲームデザインとは、以下の活動の総称である。

- ・ ゲームを**構想**する
- ・ ゲームの**動作の仕組み**を定義する
- ・ ゲームを構成する**要素**（概念的・機能的・芸術的要素など）を記述する
- ・ ゲームに関する情報を、制作チームに**伝達**する

本章では、まずプレイヤー中心デザイン（Player-Centric Design）というアプローチを学び、次にビデオゲームの中核となる構成要素——**コアメカニクス**と**ユーザーインターフェース（UI）**——を理解する。さらに、デザインプロセスの段階、チーム内の役割、デザインドキュメントについて概観する。

2. アート・エンジニアリング・クラフト

ゲームデザインを「芸術（アート）」と捉える人もいれば、「工学（エンジニアリング）」と見なす人もいる。しかし、どちらの見方も不完全である。

- ・ **アートとしてのゲームデザイン**：ゲームは純粋な美的表現が主目的ではない。映画やテレビと同様に共同作業で成り立つ表現形態であり、一人の「芸術家」に帰属するものではない。
- ・ **エンジニアリングとしてのゲームデザイン**：厳密な規格や形式的手法に縛られているわけではない。

ゲームデザインは**クラフト（職人技）**である。ゲームには美的要素と機能的要素の両方が含まれ、美しいだけでなく、楽しくプレイできなければならない。この両方を見事に融合させたとき、**エレガンス**と呼ばれる最高品質の職人技が生まれる。

キーポイント - ゲームデザインは「アート」でも「エンジニアリング」でもなく、両方を統合する**クラフト**である - 美的要素と機能的要素の見事な統合が**エレガンス**を生む

3. プレイヤー中心デザイン (Player-Centric Design)

3.1 二つの義務

プレイヤー中心ゲームデザインとは、デザイナーが想定プレイヤー (representative player) を思い描き、そのプレイヤーに対して二つの義務を負うという設計哲学である。

1. **楽しませる義務 (Duty to Entertain)** : ゲームの第一の機能はプレイヤーを楽しませることであり、デザイナーの自己表現欲よりもプレイヤーの楽しさが優先される。
2. **共感する義務 (Duty to Empathize)** : デザイナーはプレイヤーの立場に立ち、その好みや欲求に基づいて設計しなければならない。あらゆるデザイン上の判断において「これはプレイヤーの望みに合っているか？」と自問する。

3.2 よくある二つの誤解

誤解1: 「自分がプレイヤーの代表だ」

長年、デザイナーは自分が好きなものを顧客も好むと想定してきた。しかし、市場が多様化した現在、この前提は危険である。プレイヤーが10歳の少女であれ高齢者であれ、対象プレイヤーの立場で考えなければならない。『Bratz: Rock Angelz』のチーム（全員が成人男性）は、10歳の少女向けのゲームを成功させた。対象ユーザーと対話し、彼女たちの好みを研究したからである。

デザインルール: あなたはプレイヤーではない 自分が典型的プレイヤーだと思い込んではいけない。プレイヤー中心デザインは、自分とは全く異なる人物の立場に立つことを要求する。

誤解2: 「プレイヤーは対戦相手だ」

アーケードゲームのビジネスモデル（コイン投入を促すため難易度を高くする）に影響を受け、デザイナーがプレイヤーを「敵」として扱うことがある。しかし、宮本茂がアーケードデザインを離れたように、「難しい＝楽しい」ではない。デザインの目的はプレイヤーを様々な方法で楽しませることであり、進行を妨害することではない。

デザインルール：プレイヤーは敵ではない プレイヤーを対戦相手と見なしてはならない。ゲームデザインはプレイヤーを楽しませるものであり、対抗するものではない。

3.3 デザインに影響する他の動機

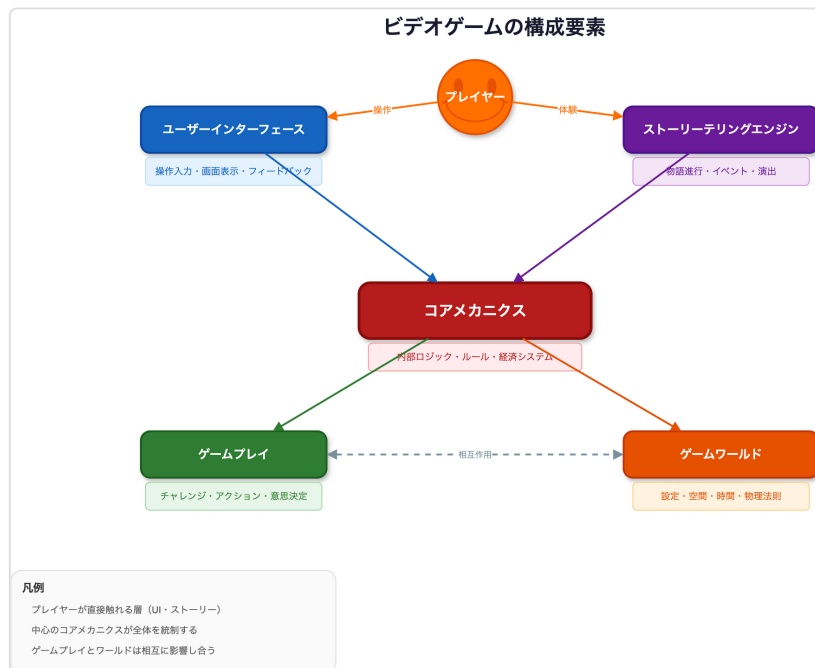
商業ゲームでは、エンターテインメント以外にも以下のような動機がデザインに影響を与える。

動機	説明	リスク
マーケット駆動	特定市場向けに要素を盛り込む	焦点がぼやけ、凡庸になりやすい
デザイナー駆動	デザイナーが全著作権を握る	プレイテストを無視し失敗しやすい（例：Daikatana）
ライセンス駆動	既存IP（映画・書籍等）を活用	創造的自由の制約、安易さへの慢心
テクノロジー駆動	技術力を誇示するため	技術偏重でゲーム体験が疎かに
アート駆動	視覚的革新を追求	ゲームプレイの軽視（成功例：Myst）

いずれか一つの動機に偏ると品質が低下する。優れたデザイナーは、全てを**プレイヤーの楽しさ**という上位目標のために統合する。

キーポイント - プレイヤー中心デザインの核は「楽しませる義務」と「共感する義務」 - 自分がプレイヤーだと思えない、プレイヤーを敵と見なさない - 市場・技術・ライセンス等の動機は、楽しさのために統合すべき

4. ビデオゲームの主要構成要素



ビデオゲームの構成要素 — プレイヤー・コアメカニクス・UI・ストーリーテリングエンジン・ゲームプレイ・ゲームワールドの関係

4.1 コアメカニクス (Core Mechanics)

コアメカニクスとは、ゲームの一般ルールを、アルゴリズムとして実装可能な記号的・数学的モデルに変換したものである。例えば、「イモムシはカタツムリより速い」というルールに対し、コアメカニクスは「イモムシは毎分5cm、カタツムリは毎分2cmで移動する」と具体的に定める。

コアメカニクスの役割： - ゲームが提供する**チャレンジ (課題)**を生成する - プレイヤーがとれる**アクション (行動)**を定義する - プレイヤーの行動がゲーム世界に及ぼす**効果**を決定する - 目標達成の**条件と結果**を規定する

従来のボードゲームではプレイヤーがルールを認識しているが、ビデオゲームではコアメカニクスは隠されており、プレイを通じてのみ体験される。

リアリズムの程度

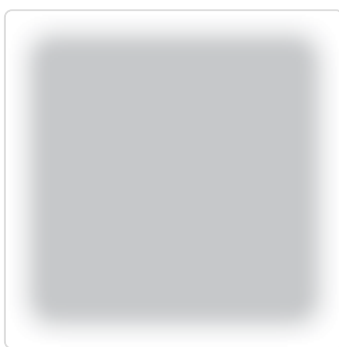
すべてのゲームは、**抽象的 (Abstract)** と **表象的 (Representational)** の連続体上に位置する。

- ・ Pac-Man：純粋に抽象的。現実の何かをシミュレートしていない
- ・ Grand Prix Legends：高度に表象的。実際のレーシングカーの危険性を精密に再現

ゲームのリアリズムの程度は、コンセプト決定時に選択し、コアメカニクスの複雑さを左右する。

4.2 ユーザーインターフェース (UI)

UIは、コアメカニクスとプレイヤーの間を**仲介**する。



コアメカニクス、UI、プレイヤーの関係

- ・ コアメカニクスが生成するチャレンジを、画面上のグラフィックやスピーカーからの音声として**出力**する
- ・ プレイヤーのボタン操作やコントローラー入力を、ゲーム内のアクションに**変換**する
- ・ ストーリーの提示やゲーム世界の感覚的表現（映像・音声・振動）を担う

UIは「プレゼンテーション層 (Presentation Layer)」とも呼ばれる。

インタラクションモデル

プレイヤーの入力をゲーム内アクションに変換する仕組みを**インタラクションモデル**と呼ぶ。代表的なモデルは以下の通り。

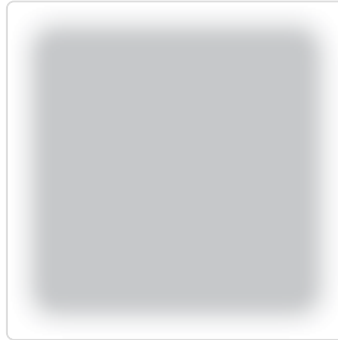
- ・ **マルチプレゼンス型**：プレイヤーが「外側」からゲーム世界のあらゆる場所に干渉できる

- ・ **アバターベース型**：プレイヤーがゲーム世界内のキャラクターを通じて行動する

カメラモデル

ゲーム世界を表示する仮想カメラの振る舞いを定義するシステムを**カメラモデル**と呼ぶ。

- ・ **静的カメラ**：固定視点（初期アーケードゲームや小規模ゲーム）
- ・ **動的カメラ**：プレイヤーの行動やイベントに応じてカメラが移動する



カメラモデルとインタラクションモデル

代表的なカメラモデル：一人称視点、三人称視点、トップダウン、サイドスクロール、アイソメトリックなど。

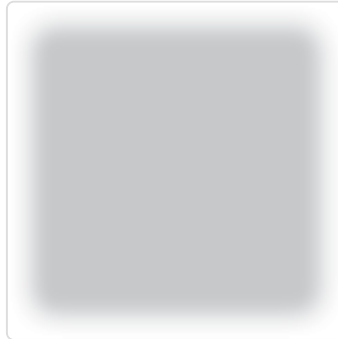
キーポイント - **コアメカニクス**はゲームプレイを生成する数学的・記号的モデル - **UI**はコアメカニクスとプレイヤーを仲介するプレゼンテーション層 - **インタラクションモデル**と**カメラモデル**はUIの重要な構成要素

5. ビデオゲームの構造

5.1 ゲームプレイモード (Gameplay Mode)

ゲームは通常、全てのチャレンジとアクションを同時に提示するわけではない。ある時点でプレイヤーが利用可能なゲームプレイのサブセットと、それを支えるUIの組み合わせを**ゲームプレイモード**と呼ぶ。

定義：ゲームプレイモード ゲームの全ゲームプレイのうち、ある時点で利用可能な特定のサブセットと、そのサブセットをプレイヤーに提示するUIを合わせたもの。



ゲームプレイモードの概念図

ゲームは同時に一つのゲームプレイモードしか取れない。ゲームプレイまたはUIが大きく変化したとき、別のモードに移行したと見なす。

例：Empire: Total War のモード切替



Empire: Total War キャンペーンモード

キャンペーンモード（ターンベースの戦略画面）では、都市建設や軍の編成など戦略的行動を行う。



Empire: Total War 海戦モード

海戦モード（リアルタイム戦闘画面）では、目の敵艦との戦術的戦闘のみに集中する。画面上の指標やメニューが完全に異なることに注目してほしい。

5.2 シェルメニューとシェルスクリーン

プレイヤーがゲーム世界に影響を与えない状態のモードを**シェルメニュー**と呼ぶ。ロード/セーブ、音量設定、画面解像度変更、入力デバイスの再設定などが該当する。タイトル画面やクレジット画面のような非対話的画面は**シェルスクリーン**と呼ばれる。

判断基準：プレイヤーの行動がコアメカニクスに影響する（たとえ遅延的であっても）なら**ゲームプレイモード**、影響しないなら**シェルメニュー/スクリーン**である。

5.3 ゲーム構造（Game Structure）

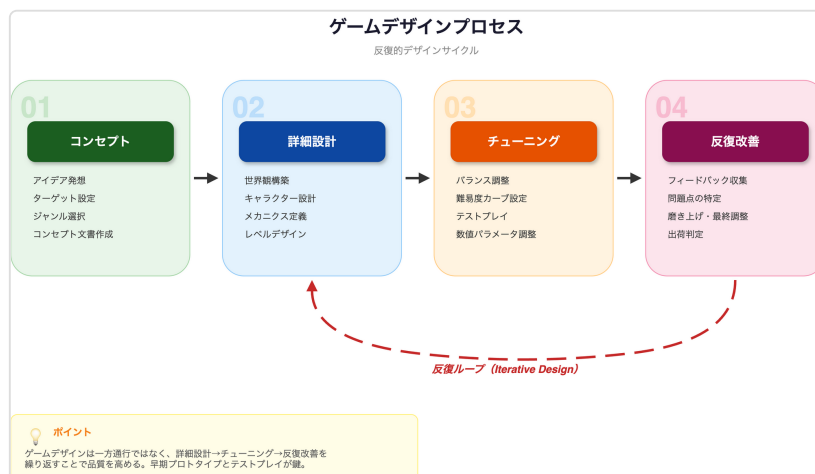
ゲームプレイモードとシェルメニューの集合、およびそれらの間の遷移関係が**ゲーム構造**を形成する。レースゲームであれば、ピットインで走行モードからピットストップモードへ遷移し、作業完了後に走行モードへ戻る、といった具合である。この構造は**フローボード**（フローチャートとストーリーボードの組み合わせ）で文書化すると有効である。

キーポイント - ゲームプレイモードは「利用可能なゲームプレイのサブセット+対応するUI」 - シェルメニューはゲーム世界に影響しない画面 - モードとメニューの遷移関係の総体がゲーム構造

6. デザインプロセスの三段階



デザインプロセスの三段階（原著図）



ゲームデザインプロセス — コンセプト→詳細設計→チューニング→反復改善の4段階フロー

6.1 コンセプト段階（Concept Stage）

プロジェクトの**土台**となる決定を行う段階。ここで決めた事項は、以降変更しない。

- ・ **ゲームコンセプトの策定**：どのようなゲームプレイで人を楽しませるか、その体験がなぜ魅力的かという根本的なアイデア
- ・ **オーディエンスの定義**：誰がこの体験を楽しむのか（ターゲット市場）
- ・ **プレイヤーの役割の決定**：アスリート、将軍、探検家、経営者など、プレイヤーが演じる役割

- ・ **夢の実現 (Fulfilling the Dream)** : プレイヤーが期待する体験の本質を特定する

デザインルール：コンセプト要素は恒久的 ゲームコンセプト、オーディエンス、プレイヤーの役割、実現する夢は、エラボレーション段階に入ったら変更してはならない。

6.2 エラボレーション段階 (Elaboration Stage)

コンセプトが固まった後、設計を一般的なものから具体的なものへと発展させる段階。プロトタイプを構築し、反復的に改良していく。

主なタスク： 1. **主要ゲームプレイモードの定義**：プレイヤーが最も多くの時間を過ごすモード 2. **主人公のデザイン**：プレイヤーが感情移入し、見て楽しいキャラクター 3. **ゲーム世界の定義**：物理的・時間的・環境的・感情的・倫理的な次元 4. **コアメカニクス**の設計：チャレンジとアクションの具体的な仕組み 5. **追加モードの作成**：必要最小限に留める 6. **レベルデザイン**：最初のプレイ可能レベル (first playable) の構築 7. **ストーリーの執筆**：レベルデザインと並行して進行

ビルド・テスト・イテレーション：著名デザイナーのマーク・サーニー（『スパイロ・ザ・ドラゴン』『ジャック×ダクスター』）は、プリプロダクション中に最低4つのプロトタイプを作っては捨てるべきだと主張している。プロトタイプ用の素材は最終製品に流用しない前提にすることで、迅速な試作が可能になる。

6.3 チューニング段階 (Tuning Stage)

全デザインが**フィーチャーロック**（新機能追加禁止）された後の段階。レベルやコアメカニクスに対する微調整のみを行い、ゲームを磨き上げる。

チューニングは**引き算のプロセス**である。新しい装飾を加えるのではなく、不完全な部分を取り除いて輝かせる作業であり、「良いゲーム」と「偉大なゲーム」の差を生む段階である。

キーポイント - **コンセプト段階**：変更不可の土台を決定（コンセプト・オーディエンス・役割・夢） - **エラボレーション段階**：プロトタイプと反復で具体化 - **チューニング段階**：フィーチャーロック後の磨き上げ（引き算のプロセス）

7. デザインチームの役割

大規模ゲームはチームで設計される。業界標準の肩書きはないが、一般的な役割は以下の通り。

役割	主な責任
リードデザイナー	デザイン全体の統括、ビジョンの守護者、チームマネジメント。プロジェクトに1名
ゲームデザイナー	ゲームプレイとコアメカニクスの定義・文書化、背景調査
レベルデザイナー/ワールドビルダー	UI・コアメカニクス・ゲームプレイを使って個々のレベルを構築。3Dモデリングやスクリプティングのスキルが必要
UIデザイナー	各モードの画面レイアウトと入力デバイスの機能を設計
ライター	イントロ、バックストーリー、台詞、カットシーンなどの執筆
アートディレクター	ビジュアルアセットの管理とビジュアルスタイルの策定
オーディオディレクター	音楽・効果音・環境音・ナレーションの管理



UIデザインの重要性：Trespasser: Jurassic Park の悪例

Trespasser: Jurassic Park は革新的なゲームだったが、不格好でバグの多いUIによって台無しになった例として知られる。優れたゲームもUIの失敗で崩壊しうることを示している。

デザインルール：委員会によるデザインを避けよ デザインを民主的プロセスとして扱ってはならない。一人が最終決定権を持ち、他のメンバーはその権限を認めなければならない。

キーポイント - リードデザイナーがビジョンの統括者であり最終決定者 - 各役割は専門スキルを持つが、全員がプレイヤー体験に貢献する - 優れたゲームもUI一つで台無しになり得る

8. デザインドキュメント

8.1 なぜドキュメントが必要か

アドホックなアプローチは現代の商業ゲーム開発では破綻する。ドキュメントの意義は二つある。

1. **口頭で合意した決定を記録**する（ペーパートレイル）

2. 曖昧なアイデアを明確な計画に変える。書かれていない要素は見落とされるか、チーム内で解釈がばらつく

「ゲームアイデア」と「デザイン決定」の違い - ゲームアイデア：「ドラゴンは卵を守るべきだ」 - デザイン決定：「卵がある間、メスのドラゴンは巣から視界外に出ない。敵が50m以内に近づくと他の活動を放棄して巣に戻り、卵を命がけで防衛する。敵が50m圏外に30秒以上いなくなるまで巣を離れない」

8.2 主なドキュメントの種類

ドキュメント	目的
ハイコンセプト	2～4ページの売り込み資料。数分で読めるようにコアアイデアを簡潔にまとめる
ゲームトリートメント	関心を持った相手への詳細説明資料。プレゼンテーション後に渡す
キャラクターデザイン	キャラクターの外見、ムーブセット（動作一覧）、背景情報を記録
ワールドデザイン	ゲーム世界の地図、雰囲気、美的スタイル、環境音の情報
フローボード	フローチャート＋ストーリーボード。ゲーム構造（モードとメニューの遷移）を視覚化
ストーリー/レベル進行	

ドキュメント	目的
	ストーリー展開とレベル間の遷移を大局的に記録
ゲームスクリプト（バイブル）	ルールとコアメカニクスの詳細仕様。理論上、これだけでゲームをプレイできるレベルの記述が理想

キーポイント - ドキュメントは「曖昧なアイデアを明確な決定に変える」ための手段 - ゲームアイデアとデザイン決定は全く異なる精度を持つ - フローボードはゲーム構造の可視化に最適

9. ゲームデザイナーに求められるスキル

ゲームデザインはタレント（天性の才能）とスキル（学習で獲得可能な能力）の両方を必要とするクラフトである。理想的なデザイナーに求められるスキルを以下にまとめる。

- ・ **想像力**：視覚的・聴覚的想像力、ドラマ的想像力、概念的想像力、ラテラルシンキング（水平思考）、演繹的推論
- ・ **技術的理解**：プログラムの動作原理、対象プラットフォームの能力と限界の把握
- ・ **分析力**：設計の長所・短所を見抜き、支配的戦略（例：Red Alert の「タンクラッシュ」）を設計段階で排除する能力
- ・ **数学力**：三角関数、確率の基礎、スプレッドシートを使ったゲーム経済のバランス調整
- ・ **美的感覚**：視覚スタイルの方向性を設定し、ステレオタイプに頼らない独自性を生み出す力
- ・ **幅広い教養と調査力**：歴史、文学、科学、政治など多分野の知識と、信頼できる情報源を使った調査能力

- ・ **文章力**：テクニカルライティング（仕様記述）、フィクションライティング（ストーリー）、ダイアログライティング（台詞）
- ・ **画力**：基本的なスケッチ能力（コンセプトアーティストがいれば必須ではない）
- ・ **統合力（シンセシス）**：異なる意見やアイデアを一つの一貫したビジョンに統合する力。チームにもビジョンの所有感を与えつつ、妥協と協調の中で設計を進める能力

キーポイント - デザイナーに求められるスキルは多岐にわたるが、全てを完璧に備える必要はない - 想像力・分析力・共感力・コミュニケーション力が特に重要 - プロフェッショナリズム（情熱がなくとも高品質な仕事を遂行する姿勢）はデザイナーの必須条件

本講のまとめ

1. **ゲームデザインはクラフト**であり、美的要素と機能的要素を統合する職人技である
2. **プレイヤー中心デザイン**は「楽しませる義務」と「共感する義務」を基盤とする
3. ビデオゲームの主要構成要素は**コアメカニクス**（ゲームプレイの生成エンジン）と**UI**（プレイヤーとの仲介層）である
4. **ゲームプレイモード**は利用可能なゲームプレイとUIの組み合わせであり、モードとシェルメニューの遷移関係が**ゲーム構造**を形成する
5. デザインプロセスは**コンセプト段階**（不変の土台）、**エラボレーション段階**（反復的具體化）、**チューニング段階**（磨き上げ）の三段階からなる
6. **デザインチーム**にはリードデザイナーを筆頭に、専門化された複数の役割がある
7. **デザインドキュメント**はアイデアを決定に変え、チーム全体で共有するための不可欠なツールである

第3回：ゲームコンセプトとゲームワールド

はじめに

ゲームデザインは一つのアイデアから始まる。しかし、アイデアだけではゲームにはならない。本講では、アイデアを**ゲームコンセプト**へと発展させるプロセスと、プレイヤーを没入させる**ゲームワールド**の設計手法を学ぶ。前半ではコンセプト段階で決定すべき要素（プレイヤーの役割、ジャンル、ターゲットオーディエンス、プログレッション、プラットフォーム）を扱い、後半ではゲームワールドの5つの次元（物理・時間・環境・感情・倫理）とリアリズムについて解説する。

第I部：ゲームコンセプト（Chapter 3）

1. アイデアの源泉

ゲームのアイデアはあらゆるところに潜んでいる。ただし、創造性は**能動的なプロセス**であり、待っているだけでは見つからない。

夢を描く（Dreaming the Dream）

大規模で豊かなゲームは「夢」から始まる。「こんなことができたら…」「こんな体験ができたら…」という想像が出発点となる。ダンジョン探索、フットボールチームの監督、ファッションデザイナーなど、プレイヤーがどんな夢を叶えたいのかを理解し、感じ取ることが重要である。

他のメディアからのアイデア

書籍、映画、テレビなどは優れたインスピレーション源となる。『Interstate '76』は1970年代の刑事ドラマから着想を得たし、『The Sims』はクリストファー・アレグザンダーの建築書『A Pattern Language』から影響を受けた。SF やファンタジーだけでなく、詩や歴史からもアイデアは見つかる。



Interstate '76 - テレビに着想を得たゲーム

他のゲームからのアイデア

既存のゲームを分析的にプレイすることは貴重な経験である。しかし、レオナルド・ダ・ヴィンチが警告したように、模倣ばかりでは類似性と凡庸さを生む。最も優れたゲームは前例のない新しい地平を切り開く。

キーポイント - アイデアは一つでは足りない。常に複数のアイデアを記録しておく - 他メディア・他ゲームを参考にしつつも、独自性を追求する - 夢を他者に伝えられるか（「なぜこのゲームを遊びたいのか？」）を常に問う

2. アイデアからゲームコンセプトへ

ゲームコンセプトとは、アイデアを**商品として議論できるレベル**にまで具体化したものである。最低限、以下の要素を含む必要がある：

要素	内容
ハイコンセプト文	2～3文でゲームの概要を記述
プレイヤーの役割	ゲーム内でプレイヤーが何者であるか

要素	内容
主要なゲームプレイモード	カメラモデル、インタラクションモデル、チャレンジの種類
ジャンル	既存ジャンルか、ハイブリッドか、新ジャンルか
ターゲットオーディエンス	想定するプレイヤー層
プラットフォーム	対応するゲーム機と特殊機器
ライセンス	利用する既存IP
競争モード	シングル・デュアル・マルチプレイヤー、競争・協力
プログレッション	ゲーム進行の概要、レベルやミッションのアイデア
ゲームワールド	世界観の簡潔な記述

キーポイント - ゲームコンセプトは単なるアイデアではなく、十分な詳細を含む設計の出発点である - これらの要素をまとめたものが「ハイコンセプトドキュメント」となる

3. プレイヤーの役割

「プレイヤーは何をするのか？」

コンセプト段階で最も重要な問いは「**プレイヤーは何をするのか？**」である。設定やキャラクターだけではゲームは作れない。「古代ローマを舞台にしたゲーム」というアイデアの正しい次のステップは「プレイヤーは古代ローマで何ができるのか？」と問うことだ。

デザインルール：プレイヤーアクションを最初に考えよ ストーリー、アバター、ゲームワールド、アートワークなど、他のすべてのデザインに先立って「プレイヤーは何をするのか？」に答えること。

役割の定義

プレイヤーの役割を明確にすることで、ゲームの目的とルールが理解しやすくなる。

『Monopoly』の役割は「不動産王」、『America's Army』では「実在の兵士」である。役割が変化するゲームでは、なぜ変化したのか、新しい状況にどう適応すればよいかをプレイヤーに伝える必要がある。

キーポイント - 役割を説明しやすいほど、出版社・小売店・顧客に理解されやすい - 具体的な「動詞」（買う、売る、建てる、戦うなど）でアクションを定義する

4. ジャンルの選択

ゲームにおけるジャンルとは、設定ではなく**チャレンジの種類**によって分類されるカテゴリである。

代表的なジャンル

ジャンル	主なチャレンジ
アクション	身体的チャレンジ、パズル、レース、戦闘
ストラテジー	戦略・戦術・兵站、経済・探索を含むことも
RPG	戦術・兵站・探索、経済（戦利品の売買）
リアルワールドシミュレーション	身体的・戦術的チャレンジ（スポーツ、乗り物）
建設・経営シミュレーション	経済的・概念的チャレンジ
アドベンチャー	探索・パズル解決

ジャンル	主なチャレンジ
パズル	論理・概念的チャレンジ

ハイブリッドゲーム

ジャンルを横断するゲームもある。最も成功した例は『ゼルダの伝説』シリーズのような**アクションアドベンチャー**である。ただし、ジャンル横断は「二つの層に訴求するつもりが、どちらにも響かない」リスクがある。混合する場合は、ゲームプレイ上の正当な理由が必要だ。

キーポイント - ジャンルは内容（SF、ファンタジー等）ではなく、チャレンジの種類で決まる - 既存ジャンル内でデザインする方が容易だが、創造性を制限しすぎてはならない

5. ターゲットオーディエンスの定義

多くのデザイナーは「自分が楽しいものは万人が楽しい」と思い込むが、これは危険な傲慢さである。

二項対立思考の危険

「男性向け」「女性向け」と単純に二分するのは統計的に誤りである。下図のように、ある仮想的なゲームへの興味度を男女で比較すると、平均値には差があっても大きな重複領域がある。少数派のオーディエンスを無視することは、商機を逃すことに等しい。



興味度の分布 - 男女の重複

包括性を目指す

無関係な機能を詰め込んで「万人受け」を狙うのではなく、**排他的な要素を排除すること**で潜在的な顧客層を広げるべきである。

デザインルール：排他的要素をゲームから排除せよ 多様なオーディエンスに到達するために、無関係な機能の追加ではなく、人々を遠ざける要素の除去を心がける。

コアゲーマー vs カジュアルゲーマー

プレイヤータイプにおける最も重要な区別は**コアゲーマーとカジュアルゲーマー**の違いである。

- ・ **コアゲーマー**：ゲームを趣味として深く追求。挑戦を好み、フラストレーションに強い。勝利の達成感を重視する。
- ・ **カジュアルゲーマー**：体験そのものの楽しさを重視。フラストレーションを感じる
と離脱する。迅速な達成感を必要とする。

両者に訴求できるゲームは稀だが、『GoldenEye 007』のように難易度設定で対応した成功例もある。

その他の考慮すべきグループ

- ・ **男性と女性**：差は思われているほど大きくない
- ・ **子供と大人**：運動能力・認知能力・注意力が大きく異なる。保護者の受容も重要
- ・ **障がいを持つプレイヤー**：字幕、コントラスト調整などの対応が可能
- ・ **他文化のプレイヤー**：ローカライゼーションの計画は初期段階から

キーポイント - 「誰を楽しませるのか？」を最初に定義してからプレイヤー中心設計を適用する - 排他的要素の排除 > 無関係な機能の追加

6. プログレッションとストーリー

長いゲームでは、プレイヤーに**進行感**を与える必要がある。レベル分割するか、ストーリーで牽引するか、あるいはその両方か。レベルは独立か、順序的か、分岐構造か。クリア条件は何か——ジャンルがこれらの判断を助ける。

ストーリーはゲームにコンテキストと目標を与えるが、コンセプト段階では概要で十分である。

デザインルール：ストーリーは後から コンセプト段階でストーリーに時間をかけすぎないこと。まずゲームプレイに集中する。

7. ゲーム機の種類

プラットフォーム	特徴
家庭用コンソール	テレビに接続、標準コントローラー、マルチプレイに適する。グラフィックは大胆

プラットフォーム	特徴
	に、UIはシンプルに
PC（スタンドアロン）	高解像度、キーボード+マウスで精密操作。ネットワーク対戦に強い。構成の多様性が開発の難題
PC（ブラウザベース）	クロスプラットフォーム対応。3D描画は制限されるがカジュアルゲームに適する
携帯ゲーム機	小画面・少ボタン。カートリッジ容量の制限がデザインの腕を試す
スマートフォン	ポータブルなネットワークプレイが可能。標準化の欠如が課題

キーポイント - プラットフォームの特性（入出力デバイス、処理能力、使用環境）がゲームの範囲を決定する - コンセプトに「どのマシンで動くか」の記述は不可欠

第II部：ゲームワールド (Chapter 4)

8. ゲームワールドとは何か

ゲームワールドとは、ゲーム内の出来事が展開する**人工的な宇宙・仮想空間**である。プレイヤーがマジックサークルに入り「別の場所にいる」と仮定するとき、その場所こそがゲームワールドである。

すべてのゲームがゲームワールドを持つわけではない。チェスには世界の暗示しかなく、三目並べには全くない。しかし多くのビデオゲームにおいて、ゲームワールドは没入と販売の両方に不可欠な要素である。

ゲームワールドの目的

1. **それ自体がエンターテインメント**：探索し、インタラクションする場を提供する
2. **販売の推進力**：箱の絵を見て「この世界に行きたい」と思わせる

一般に、プレイヤーがコアメカニクスに習熟するほどゲームワールドの重要度は下がる。しかし初回プレイヤーの関心を引きつけ維持するには、世界の魅力が不可欠である。

キーポイント - ゲームワールドは絵と音の集合ではなく、文化・美学・道徳観を含む多次元的な概念 - アートだけでは不十分。ワールドはコアメカニクスと連動して設計する

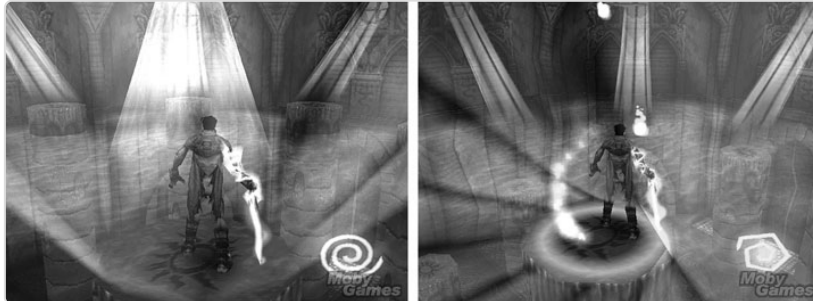
9. 物理的次元 (Physical Dimension)

物理的次元は、ゲーム空間のシミュレーション特性を定義する。

空間の次元数

次元数	説明	例
2D	左右・上下のみ。画面の2次元に直接対応	Super Mario Bros.
2.5D	見た目は3Dだが、実体は2Dレイヤーの重なり	StarCraft

次元数	説明	例
3D	真の3次元空間。「中にいる」感覚	Prince of Persia
4D	同一3D空間の「別バージョン」として実装	Legacy of Kain



StarCraft - 2.5Dの表現



Prince of Persia - 完全な3D環境

注意：3Dが常に最良とは限らない。『Lemmings 3D』は2D版の成功を3Dでは再現できなかった。次元数はエンターテインメント価値に寄与するかどうかで判断する。

スケール（縮尺）

スケールとは、ゲーム空間の絶対的な大きさとオブジェクト間の相対的な大きさを指す。スポーツゲームやFPSでは実寸に近いスケールが求められるが、俯瞰視点の戦略ゲームで

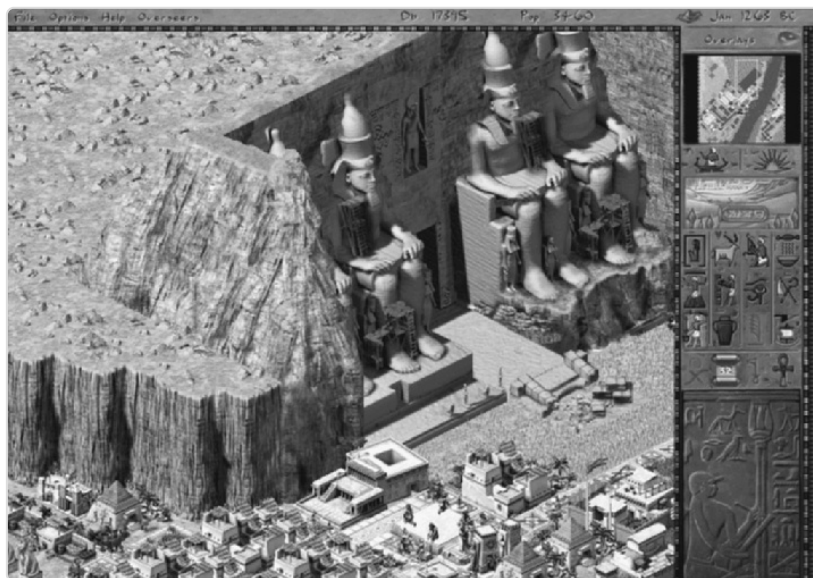
はオブジェクトを誇張して視認性を確保する必要がある。屋内と屋外でスケールを変えること、移動速度の歪みなども一般的なテクニックである。



Age of Empires - 建物と人のスケール歪み

境界（バウンダリ）

有限のメモリにより、ゲーム世界には必ず「端」がある。スタジアムやトラックのような自然な境界がない場合、島・山脈・砂漠などで世界の端を正当化する。フラットな世界を「ラップ」させて球体に見せかける手法（『Magic Carpet』 『Spore』）もある。



Spore - 球体として描画された世界

キーポイント - 空間の次元数はゲームプレイに貢献するかどうかで選択する - スケールの歪みはプレイアビリティとリアリズムのバランスで決まる - 境界は没入感を壊さないよう、世界観に即した方法で処理する

10. 時間的次元 (Temporal Dimension)

時間的次元は、ゲーム世界における時間の扱いを定義する。

主なパターン

- ・ **時間なし**：多くのアクション・ターン制ゲーム。世界はプレイヤーの操作まで静止する
- ・ **雰囲気としての時間**：昼夜の変化があるが、ゲームプレイには影響しない（効果は薄い）
- ・ **意味のある時間**：『Baldur's Gate』では夜間に店が閉まり、モンスター遭遇率が上がる
- ・ **可変時間**：『The Sims』ではキャラクターが眠ると時間が加速する
- ・ **非整合時間 (Anomalous Time)**：『The Settlers』では木の成長と鉄の精錬が同程度の時間で完了する。時計を表示しないことで不自然さを軽減



The Settlers - 時計を表示しない非整合時間

プレイヤーによる時間調整

スポーツゲームではクォーターの長さを短縮でき、フライトシミュレーターでは巡航中に2倍速・4倍速にできるなど、プレイヤーに時間のコントロールを委ねる設計も重要である。

キーポイント - 時間をゲームプレイに組み込むなら、視覚的な演出だけでなく機能的な影響を与える - 非整合時間はゲーム全体のバランスのために許容されるが、ファンタジーを壊さない工夫が必要

11. 環境的次元 (Environmental Dimension)

環境的次元は、ゲーム世界の外観と雰囲気を定義する。

文化的コンテキスト

ゲーム世界の文化とは、人類学的な意味での文化——信仰、態度、価値観、政治・宗教制度、社会組織——を指す。これらは衣服、家具、建築、武器などの人工物のデザインに反映される。



Cleopatra: Queen of the Nile - UIまで文化に統一

バックストーリー（ゲーム開始前の架空の歴史）も文化的コンテキストの一部であり、なぜその文化がそのようであるかの裏付けとなる。

物理的環境

物理的環境とは、ゲームが実際にどう見え、どう聞こえるかである。建物、乗り物、自然（鳥、植物、空、気候）、そして音（音楽、環境音、効果音）まで含む。スタンリー・キューブリックが『2001年宇宙の旅』で「美しく青きドナウ」を選んだように、音楽は世界観を決定づける力を持つ。



Grim Fandango - アステカ、アールデコ、死者の日の融合

ディテールとスタイル

ディテールは没入感を高めるが、常にコストが伴う。ルールは「ゲームプレイを損なわない範囲で最大限のディテールを」。

スタイル定義には二つの側面がある： 1. **世界の固有スタイル**（仏教寺院風の建築など）
2. **描写スタイル**（フィルムノワール風の描き方など）

両者が組み合わさってプレイヤーの体験を形成する。一貫したスタイルの維持が重要で、セクションごとにバラバラなアートスタイルは避けるべきである。



Naruto: Ultimate Ninja Storm - コミック風描写スタイル

使い古された設定と新しいインスピレーション

SF やファンタジーの使い古された設定（エルフ、ドワーフ、魔法使い、ドラゴン）に頼らず、多様な文化圏からインスピレーションを得るべきである。1200年頃のイスラム文明の優雅さ、ヴァルハラ、ピョートル大帝のロシア、アンコールワット、イースター島、マチュピチュなど、未開拓の源泉は無数にある。



使い古された中世ファンタジー設定の例

キーポイント - 文化的コンテキストはUIデザインにまで影響する - サウンドデザインはビジュアルと同時に考える - 使い古された設定を避け、多様な文化・芸術運動からインスピレーションを得る

12. 感情的次元 (Emotional Dimension)

感情的次元は、ゲームがプレイヤーに**喚起しようとする感情**を定義する。

感情の喚起手法

- ・ **達成感と敗北感**：リスクと報酬のメカニクスで増幅。難易度調整でフラストレーションを管理する
- ・ **野心と創造性**：建設・経営シミュレーション (SimCity、RollerCoaster Tycoon) が得意とする領域
- ・ **恐怖とサスペンス**：暗闇、突然の音、不意の出来事。ただし過度なゴア表現は逆効果。アルフレッド・ヒッチコックに学べ
- ・ **人間関係の感情**：愛、悲しみ、嫉妬、怒り。キャラクターへの感情移入とストーリーテリングが鍵



Final Fantasy VII - エアリスの死

「楽しさ」の限界を超えて

「楽しさ（fun）」は感情の範囲を狭める言葉である。ゲームはインタラクティブ・エンターテインメントであり、悲しい結末の映画がエンターテインメントであるように、より広い感情領域を探求できる。

デザインルール：非現実的な極端さを避けよ 「世界の運命があなたの手に」という大仰な設定より、個人の運命を描く方が感情的共鳴を生む。映画『カサブランカ』は三人の小さな問題で観客の心を掴んだ。

キーポイント - ゲームは「楽しさ」を超えた多様な感情を喚起できるメディアである
- ただし、まず「楽しさ」を作る技術を習得してから、より複雑な感情に取り組む -
感情の操作に安易な公式（「Xを感じさせればYをせよ」）は通用しない

13. 倫理的次元（Ethical Dimension）

倫理的次元は、ゲーム世界における**善悪の定義**を司る。デザイナーはゲーム世界の神であり、その道徳を確立する。

道徳的意思決定

多くのゲームの倫理は単純（悪者を倒し、善人を守る）だが、**倫理的曖昧さ**や**困難な選択**を組み込むことで、より深いゲーム体験を実現できる。「確実に死ぬ小隊を送り出して大隊を救うべきか？」のような問いが物語を豊かにする。

一部のRPGでは「不道德な行動は自由だが、その結果も引き受けよ」というルールを実装している。これは単に悪い行動を禁止するよりも成熟したアプローチである。

『America's Army』の特殊な道徳



America's Army - 各チームが自分を正義と認識する

米陸軍が配布した『America's Army』では、すべてのプレイヤーが自分のチームを米兵、相手をテロリストとして見る。同じ戦場で異なる「現実」を表示するという、ビデオゲーム史上でもユニークな倫理的処理が行われた。

ゲームにおける暴力

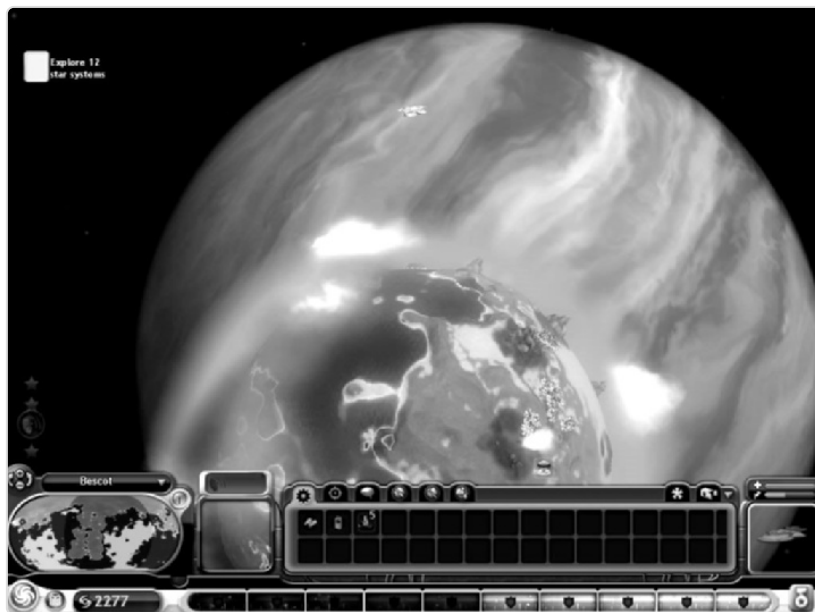
暴力表現そのものが問題なのではなく、**暴力の描写方法と許容される状況**が問題である。原則として、視覚的リアリズムが高いほど、倫理的リアリズムも高く設定すべきである。

キーポイント - デザイナーはゲーム世界の倫理を意識的に設計する責任がある - 倫理的ルールはゲーム開始前にプレイヤーに明示する - 視覚的リアリズムと倫理的リアリズムを連動させる

14. リアリズム

リアリズムは単一の値ではなく、ゲームのあらゆる要素に対して「完全にリアル」から「完全に抽象的」までの**ベクトル**として存在する。

- ・多くのFPSは武器の性能を精密にモデリングするが、プレイヤーは同時に10丁の武器を速度低下なしに運べる
- ・カートゥーン調のアクションゲームでは「どこまでリアリズムを入れるか」を問う、リアル系シミュレーションでは「現実のどの部分を省くか」を問う
- ・数百万ドルの訓練用フライトシミュレーターですら、完全な逆さまの状態は再現しない



Ratchet & Clank - 抽象的なスタイルのアクションゲーム

キーポイント - リアリズムはゲーム全体の単一属性ではなく、各要素ごとに設定される多変量的な性質 - すべてのデザイン判断はエンターテインメント価値に奉仕し、リアリズムの目標と調和すべき - 完全にリアルなゲームは存在しない。もし完全にリアルなら、それはゲームではなく人生である

まとめ

本講では、ゲームデザインの二つの根幹的なテーマを扱った。

ゲームコンセプトの段階では、アイデアを商品レベルの概念へと昇華させるために、プレイヤーの行動・役割・ジャンル・ターゲットオーディエンス・プラットフォーム・プログレッションを明確化する必要がある。特に「プレイヤーは何をするのか？」は最初に答えるべき根本的な問いである。

ゲームワールドの設計では、5つの次元（物理・時間・環境・感情・倫理）を体系的に検討することで、没入感のある一貫した世界を構築できる。リアリズムは各要素ごとに適切なレベルを設定し、エンターテインメント価値との調和を図る。

次回の授業では、ゲームプレイの核心であるコアメカニクスとチャレンジの設計について学ぶ。

本教材は Ernest Adams 著『Fundamentals of Game Design』Chapter 3-4 に基づく

ewpage

第4回：創造的プレイとキャラクター開発

対応章: Chapter 5 “Creative and Expressive Play” / Chapter 6 “Character Development”

第I部：創造的・表現的プレイ（Chapter 5）

ゲームをプレイすること自体が自己表現の一形態である。プレイヤーの判断にはプレイスタイル（慎重か大胆か、攻撃的か防御的か）が反映される。ビデオゲームでは、伝統的なゲーム以上に多彩な自己表現の手段を提供できる。本章では、ゲームに組み込む創造的プレイの類型を学ぶ。

1. 自己定義プレイ（Self-Defining Play）

プレイヤーがゲーム内で自分を表す **アバター** を選択・カスタマイズする行為を **自己定義プレイ** と呼ぶ。モノポリーでトークンを選ぶ行為もその一例である。プレイヤーは自分に似たアバターや、憧れのファンタジー像を選ぶことを楽しむ。特に女性プレイヤーはアバターの選択・デザインに強い関心を持ち、魅力を感じないアバターの使用を嫌う傾向がある。

人格表現の形式

自己定義プレイには3つの段階がある。

形式	説明	例
アバター選択	あらかじめ用意されたアバターから選ぶ	レースゲームでの車種選択
アバターカスタマイズ		

形式	説明	例
	外見や能力のパーツを差し替える	RPGでの装備変更、レースゲームの塗装やパーツ交換
アバター構築	ゼロからアバターを組み立てる	『The Elder Scrolls IV: Oblivion』の詳細なキャラクター作成



アバター構築の例

属性 (Attributes) の理解

アバターの特性を定義する値を **属性** と呼ぶ。属性は大きく2種類に分かれる。

- ・ **機能的属性 (Functional Attributes)** : ゲームプレイに影響する属性。
『D&D』の筋力・知力・敏捷性など。さらに以下に細分される。
 - **キャラクターライゼーション属性** : キャラクターの根本を定義し、ほとんど変化しない (例: 最大速度)
 - **ステータス属性** : 現在の状態を示し、頻繁に変化する (例: 現在のHP)
- ・ **装飾的属性 (Cosmetic Attributes)** : ゲームプレイに影響しないが、自己表現に寄与する属性。レースカーの塗装色など。

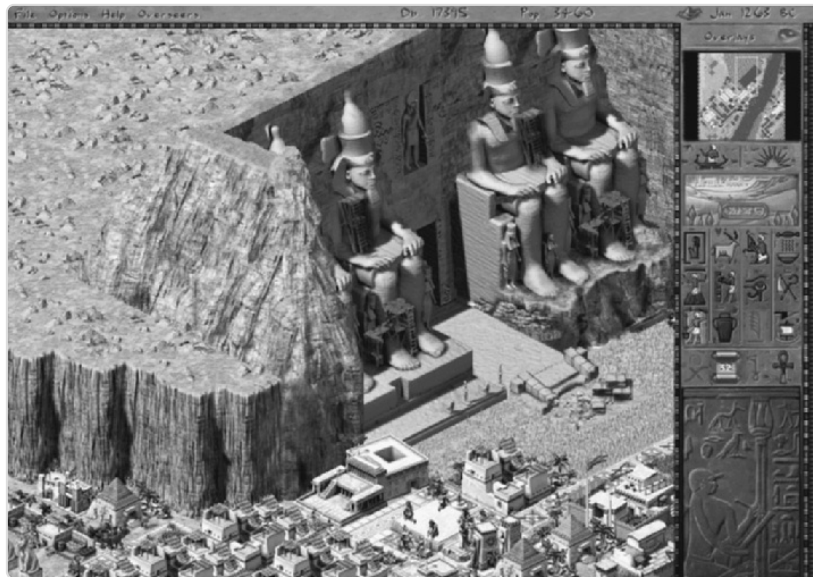
設計上の注意: 機能的属性をプレイヤーに設定させる場合、その選択がゲームプレイにどう影響するかを明確に伝えなければならない。影響の分からない選択を強いることは、プレイヤーにとってストレスとなる。

属性設定のベストプラクティス

- ・『D&D』方式で一定のポイントを属性に配分させ、バランスを保つ
- ・デフォルト設定を用意し、すぐに遊び始めたいプレイヤーに配慮する
- ・一度クリアした後に自由設定を解禁する方法もある

性別は機能的属性か装飾的属性か

ゲームデザインにおいては、性別を装飾的属性として扱うのが望ましい。現実世界の平均的な性差をゲームの制約として持ち込むと、ファンタジー体験を損なう。筋力や敏捷性などの機能的属性は性別とは独立に設定可能にすべきである。



カスタマイズの例

キーポイント - 自己定義プレイはプレイヤーに個性の投影機会を与える - 属性は「機能的」と「装飾的」に大別される - 装飾的属性は低コストで楽しさを高める - 性別は装飾的属性として扱い、機能的ペナルティを付けない

2. クリエイティブプレイ (Creative Play)

多くのゲームはプレイヤーに何かを設計・構築する機会を提供する。『Caesar』シリーズではローマの都市、『Spore』では生物を作る。人は設計や構築を楽しむものであり、これが建設・経営シミュレーションの主要な魅力となっている。

クリエイティブプレイは保存・共有できるようにすべきである。スクリーンショットの撮影、他プレイヤーとの共有、Webへのアップロードなどの機能が創作の楽しさを増幅する。

制約付きクリエイティブプレイ (Constrained Creative Play)

ルールが課す人為的制約の中で創造するプレイ形態。制約は創造の「構造」を提供する。

経済による制限: 『SimCity』では都市建設に資金が必要で、経済成長に伴い新施設が解禁される。経済管理が主な挑戦であり、創造力はその報酬として機能する。

物理的基準への適合: 『Spore』では生物を自由にデザインできるが、背骨を持つ陸上動物でなければならない。『RollerCoaster Tycoon』ではジェットコースターの設計が物理法則に準拠する必要がある。失敗時にはその理由をプレイヤーに伝えることが重要である。



RollerCoaster Tycoon 2 のコースター設計画面

美的基準への適合: 物理基準と異なり、美的基準には普遍法則がない。以下の実装方法がある。

1. **固定ルールによるテスト:** 色の組み合わせルールなど。ルールをプレイヤーが学べる仕組みが必須
2. **トレンドシステム:** 基準が時間とともに変動し、リサーチで部分的に把握可能

3. **オンライン投票**: コミュニティがプレイヤーの作品を評価する（『The Sims 2: H&M Fashion Runway』）

フリーフォームプレイとサンドボックスモード

制約なしに全機能を使える状態が **フリーフォームクリエイティブプレイ** である。通常は制約付きモードの特別版として **サンドボックスモード** が提供される。報酬は少ないが口だが、自由度が高く、プレイヤーに好評である。クリエイティブプレイを提供するゲームには、可能な限りサンドボックスモードを含めるべきである。

キーポイント - 制約付きプレイは「経済」「物理」「美的基準」で構造化できる - 失敗時はその理由を伝えるフィードバックが不可欠 - サンドボックスモードはコアメカニクスとの干渉が少なく、実装・調整が容易

3. ストーリーテリングプレイ (Storytelling Play)

ゲームの機能を使って自分自身の物語を創り、オンラインで配布するプレイ形態である。

- ・ **高機能型**: 『The Movies』（Lionhead Software）ではカメラ配置、録画、編集が可能で、Adobe Premiere Pro等の外部ツールへ映像を書き出せる
- ・ **簡易型**: 『The Sims』ではスクリーンショットにキャプションを付け、ストーリーボード形式でWebに公開できる
- ・ **最小構成**: プレイヤーの行動ログをテキストで生成し、編集してナラティブ化する

複雑なビデオ編集ソフトを必要としないシンプルな方式でも、十分な成功を収められることを『The Sims』が証明した。

キーポイント - ストーリーテリングプレイの複雑さは幅広いレンジで設計可能 - シンプルなスクリーンショット+キャプション方式でも高い効果がある

4. ゲーム改変 (MOD)

プレイヤーにゲーム自体の改変を許すことは、創造的自由の最高峰である。

- ・ **MOD (Modifications)** : ハードコアゲーマーに極めて人気が高い。Unreal エンジンやHalf-Life 2エンジンのMODは、ゲーム本体の継続的な販売にも貢献する
- ・ **レベルエディタ**: プレイヤーが独自のレベルを構築可能にするツール。公開する場合は、社内ツールと同等の堅牢性が求められる (例: StarCraft Campaign Editor、Half-Life 2のHammerエディタ)
- ・ **Bot**: プレイヤー自身がプログラムするAI対戦相手。FPSで練習相手として利用されることが多い



レベルエディタの例

キーポイント - MODはゲームの寿命を大幅に延ばす商業的メリットがある - レベルエディタは製品品質の堅牢性が必要 - MOD対応はプログラマ主導の課題だが、デザイナーも計画段階で関与すべき

第II部：キャラクター開発（Chapter 6）

「ハリー、私たちが何者であるかを示すのは、能力ではなく選択なのだ。」 ——
J.K.ローリング『ハリー・ポッターと秘密の部屋』

キャラクターデザインは、物語を語り感情的な反応を喚起するうえで不可欠な要素である。英雄、悪役、苦境にある無辜の人々、傍観者——これらのキャラクターなしにはゲーム世界は空虚な殻にすぎない。

5. キャラクターデザインの目標

優れたキャラクターデザインには3つの目標がある。

1. **魅力（Appeal）**：悪役であっても、人々を引きつける力を持つこと（例：ダース・ベイダー）
2. **信頼性（Credibility）**：外見と行動に一貫性があり、プレイヤーが信じられること
3. **共感・同一化（Identification）**：特にアバターの場合、プレイヤーがその靴に入りたいと思えること

キャラクターの各属性——身体、衣服、声、アニメーション、表情——は調和して役割を表現すべきである。ステレオタイプのキャラクターにも、他と区別する「何か」が必要である。

ビジネス上の意義：優れたキャラクターはゲームの知的財産として最も商業価値が高い。書籍、映画、玩具など多メディア展開の基盤となる。



キャラクターデザインの例

キーポイント - キャラクターは「魅力的」「信頼できる」「同一化可能」であるべき - 各属性の調和がキャラクターの明確さを生む - キャラクターIPは最大の商業資産となりうる

6. プレイヤーとアバターの関係

ゲーム業界で **アバター** とは、プレイヤーの操作下にある主人公キャラクターを指す。プレイヤーとアバターの関係は、アバターの設定度合いによって大きく異なる。

アバターの特定性スペクトラム

類型	特徴	例
非特定のアバター	デザイナーが何も規定しない「空の器」	ゴードン・フリーマン (Half-Life)、Myst
半特定のアバター	ある程度の外見・性格があるが詳細は不明	マリオ、ララ・クロフト、リンク
特定のアバター		

類型	特徴	例
	独自の過去・性格・人間関係を持つ	マックス・ペイン、ナンシー・ドリュー、エイプリル・ライアン

非特定の-avatarはプレイヤー自身の投影先となる。特定の-avatarとの関係は小説の主人公と読者の関係に近い。ただしゲームではプレイヤーが主人公を導ける点が異なる。特定の-avatarはプレイヤーの指示を拒否することもある（例：『The Longest Journey』のエイプリル・ライアン）。

操作方式の影響

- ・ **間接操作**（ポイント&クリック）：プレイヤーは非物質的な案内人のように感じる
- ・ **直接操作**（キャラクターを直接ステアリング）：プレイヤーは-avatarそのものになる感覚を得るが、-avatarは操り人形に近くなる

男女プレイヤーとキャラクターの関係

- ・ 男性プレイヤーはデフォルト-avatarを比較的容易に受け入れる
- ・ 女性プレイヤーは-avatarを自己の延長と見なし、カスタマイズ機能を重視する
- ・ ララ・クロフトが示したように、男性は女性-avatarで「男性的活動」を行うことに抵抗がない
- ・ 女性プレイヤーにとっては、過度に性的に描かれたキャラクターは同一化の障壁となる（例：Heather（Silent Hill 3）のようなリアルな女性像が好まれる）



ララ・クロフト

キーポイント - アバターの特定度合いがプレイヤーとの関係性を決定する - 直接操作は没入感を高めるが、アバターの独立性を弱める - 女性プレイヤー向けにはカスタマイズ機能が特に重要

7. ビジュアル外見 (Visual Appearances)

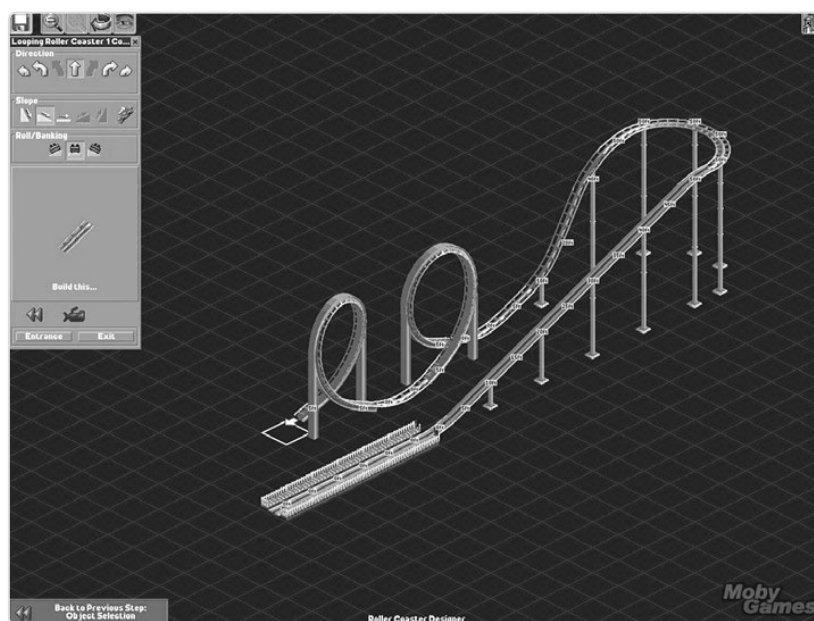
キャラクターの外見はプレイヤーの感情に大きな影響を与える。外見から着手するデザイン手法を **アートドリブン・キャラクターデザイン** と呼ぶ。単純でカートゥン的なキャラクター、または多メディア展開を想定する場合に適する。

キャラクターの体型分類

- ・ **ヒューマノイド**: 2本の腕・脚と1つの頭を持つ人間型。リアルまたはカートゥン比率
- ・ **非ヒューマノイド**: 乗り物・機械型、動物型、モンスター型 (R2-D2、ダーレクなど)
- ・ **ハイブリッド**: 人間と機械の融合体 (人魚、ボーク、サイロンなど)

カートゥンの特性の4類型

類型	特徴	例
クール	超然・反権威的、サングラス、のんびりした体勢	ラチェット（Ratchet & Clank）
タフ	身体的攻撃性、大柄・大きな動作	リュウ（Street Fighter）
キュート	大きな目・頭、丸みのある体、明るい色	マリオ
グーフィー	奇妙なプロポーション、ぎこちない動き	クラッシュ・バンディクー



カートゥンキャラクターの例

文化的注意: アートスタイルは文化によって大きく異なる。日本のアニメスタイル、ヨーロッパの漫画スタイルなど、販売対象国の文化的期待を十分に研究すべきである。

過度な性的誇張の問題

キャラクターの性的特徴を極端に誇張する **ハイパーセクシュアライゼーション** は、若い男性には訴求力があるが、以下の問題を伴う。

- ・ 類似キャラクターが飽和し、差別化が困難になる
- ・ 年齢層の高いプレイヤーを遠ざける
- ・ 女性プレイヤーに「歓迎されていない」というメッセージを送る
- ・ ゲーム自体の技術的・デザインの進歩を覆い隠す

ユーモアの文脈であれば許容される場合もあるが（『No One Lives Forever』のケイト・アーチャー）、外見だけでは質の低いゲームは売れない。

衣装・武器・象徴的オブジェクト・名前

- ・ 衣装と小道具はキャラクターを素早く識別させる重要な手段（インディ・ジョーンズの帽子とブルウィップ、ダース・ベイダーのマントとヘルメット）
- ・ 武器の選択はキャラクター性を語る（肉切り包丁＝粗暴、レイピア＝優雅）
- ・ ヘアスタイルや宝飾品は変化しにくく、視覚的識別子として有効
- ・ **カラーパレット** を設定することで、キャラクターの態度・気質を色で表現する（スーパーマンの赤と青＝明朗、バットマンの暗色＝渋い）

デザインルール: 視覚的特徴を盛り込みすぎない。2～3の特徴で十分であり、それ以上は滑稽に見える。

コンセプトアートとモデルシート

- ・ **コンセプトアート:** デザイン初期段階で「何を作るか」を伝える素描。短時間で描き、完成品ではなく説明と着想のためのもの
- ・ **モデルシート:** 1キャラクターの複数ポーズを1枚にまとめ、感情・態度を表情と姿勢で比較する伝統的アニメーション手法



コンセプトアート例

キーポイント - アートドリブンデザインはシンプルなキャラクターに適する - ハイパーセクシュアライゼーションは市場を狭める - カラーパレットと2〜3の視覚的特徴で明確な個性を構築する

8. キャラクターの深み (Character Depth)

外見だけでは伝えきれないキャラクターの内面を掘り下げる手法が **ストーリードリブン・キャラクターデザイン** である。キャラクターの役割・性格・行動を先に決め、それを基にアーティストが外見を開発する。

役割・態度・価値観

主要キャラクターには **キャラクター背景資料 (バックグラウンダー)** を作成する。出生地、家族、教育、趣味、恐怖症、政治信条、恋愛歴など、あらゆる側面を文書化する。

キャラクターの性格を伝える手段は3つ: 1. **外見** (前述のビジュアルデザイン) 2. **言語** (後述のオーディオデザイン) 3. **行動** (最も繊細な方法 —— 見せるのであって、語るのではない)

キャラクターの次元性

キャラクターの感情的複雑さを4段階で分類する（『ロード・オブ・ザ・リング』の例で解説）。

次元	説明	例
0次元	離散的な感情状態のみ （憎悪/恐怖など）、混合感情なし	オーク
1次元	1つの感情変数が時間とともに変化する	ギムリ（エルフへの敵意が徐々に友情に変わる）
2次元	複数の感情変数があるが相互に矛盾しない	デネソール（誇り・軽蔑・絶望はあるが道徳的葛藤なし）
3次元	複数の感情変数が矛盾を生じ、一貫性のない行動を引き起こす	フロド、ゴラム（指輪の誘惑による内的葛藤）

3次元キャラクターの矛盾する感情をどう見せるか ―― 躊躇、途中での心変わり、無意識の自己妨害 ―― はプログラマとアーティスト双方との協議が必要な高度な設計課題である。



キャラクターの次元性

キャラクターの成長

ゲームが単なる冒険の連続以上のものを目指すなら、キャラクター成長が不可欠である。

- ・ **アクションゲーム**: 新しい技・パワーアップ（物理的成長のみ）
- ・ **アドベンチャーゲーム**: 個人的・感情的な文学的成長
- ・ **RPG**: スキル、筋力・知力などの属性、および物語的な個人的成長

成長の表現方法は、画面上の数値表示（最も粗い方法）から、外見の変化、利用可能アクションの増加、言語・行動の変化（最も繊細な方法）まで幅広い。

キャラクター・アーキタイプ

ジョセフ・キャンベルの『千の顔を持つ英雄』が提唱した **英雄の旅** のパターンには、メンター、トリックスター、門番など、神話以来の普遍的なキャラクター類型が含まれる。クリストファー・ヴォグラーの『作家の旅』が脚本家向けに分かりやすく解説している。ただし、ゲームにアーキタイプを機械的に当てはめる必要はない。

キーポイント - ストーリードリブンデザインは複雑なキャラクターに適する - キャラクターの次元性（0～3次元）は感情的複雑さの設計指針となる - 「見せる、語らない」の原則がゲームでは特に重要 - キャラクター成長はジャンルに応じて物理的/感情的/属性的に設計する

9. オーディオデザイン (Audio Design)

キャラクターデザインの重要な構成要素として、効果音・音楽・音声がある。

効果音と音楽

- ・ 音はキャラクターの性格を伝える（喋らなくても）。打撃音、掛け声はプレイヤーの操作に対する確認フィードバックとなる
- ・ **深い音** は遅く強いキャラクター、**高い音** は軽く速いキャラクターを暗示する
- ・ 音の質感は視覚的テクスチャと調和すべき（金属的な外見には金属的な音）
- ・ 不調和はコメディ効果に利用できる

キャラクター固有の **音楽テーマ** は感情的インパクトを大きく高める。ジョン・ウィリアムズの『スター・ウォーズ』は好例：ダース・ベイダーの帝国マーチ（不協和なトランペット）、レイア姫の愛のテーマ、反乱同盟軍のメインテーマ。

- ・ 悪や奇異なキャラクターには短調
- ・ 善や英雄的キャラクターには長調
- ・ ユニゾン演奏と単調なリズムは強制的画一性を暗示

実装上の注意: 音楽・効果音・台詞/ナレーションは別々の音声ファイルとし、ゲーム内でミックスする。これによりローカライズ時の差し替えが容易になり、音楽と効果音を個別に音量調整できる（聴覚障害者への配慮として重要）。

声と言語

キャラクターの話し方は膨大な情報を伝達する。

要素	伝達する情報
語彙	年齢、社会階層、教育レベル
文法・構文	教育と階級（長く複雑な文は知性を連想させる）
アクセント	出身地、社会階層
話速と声調	興奮、退屈、不安、敵意など感情状態
声の癖	吃音、舌足らず、決まり文句（「えー…どうしたんだい？」）

『ザ・シンプソンズ』は言語によるキャラクター描写の優れた例である。ホーマーの限られた語彙は無教養を、リサの豊富な語彙と複雑な構文は学者肌を、バートの状況に応じた言語の切り替えは自覚的な享楽主義者を表現している。

キーポイント - 効果音はプレイヤーへのフィードバックでありキャラクター表現でもある - 音楽テーマはキャラクターの感情的印象を強化する - 音声ファイルは音楽・効果音・台詞を分離して管理する - 語彙・文法・アクセント・声調・癖の5要素でキャラクターの社会的背景が伝わる

まとめ：本講のポイント

1. **自己定義プレイ** はアバターの選択・カスタマイズ・構築を通じてプレイヤーの個性をゲーム世界に投影させる
2. **クリエイティブプレイ** は経済・物理・美的基準による制約で構造化でき、サンドボックスモードが自由な創作を可能にする
3. **ストーリーテリングプレイ** はシンプルな仕組みでも大きな成功を収めうる
4. **MOD・レベルエディタ** はゲームの寿命を延ばし、コミュニティを活性化する
5. **キャラクターデザイン** の目標は「魅力」「信頼性」「同一化」の3つ
6. **プレイヤーとアバターの関係** はアバターの特定度と操作方式に依存する
7. **ビジュアルデザイン** ではカラーパレットと少数の特徴で個性を構築し、過度な性的誇張を避ける
8. **キャラクターの深み** は0～3次元の枠組みで設計し、「見せて語らない」原則を守る
9. **オーディオデザイン** は効果音・音楽テーマ・声と言語の3層でキャラクターを立体化する

考えてみよう（ディスカッション用）

1. あなたが最も感情移入したゲームキャラクターは誰か。そのキャラクターの「次元性」は何次元だと考えるか。
2. サンドボックスモードと制約付きモードでは、どちらがプレイヤーの創造性をより刺激するか。
3. キャラクターの性的誇張がゲーム市場に与える影響について、肯定的・否定的両面から議論せよ。
4. 自分がデザインするとしたら、アバターの特定度（非特定/半特定/特定）をどう設定するか、その理由は何か。
5. ゲームキャラクターのオーディオデザインにおいて、声優のキャスティングとキャラクターテーマ音楽のどちらがより重要か。

参考文献 Adams, E. (2014). Fundamentals of Game Design (3rd ed.). New Riders. Chapters 5-6.

ewpage

第5回：ストーリーテリングとナラティブ

対応テキスト: Fundamentals of Game Design — Chapter 7: Storytelling and Narrative (pp.155–196)

1. ゲームにストーリーを入れる理由

ビデオゲームにはしばしば、ゲームプレイそのものを越えたフィクションのストーリーが含まれる。デザイナーがストーリーを追加する主な理由は以下の4つである。

1. **娯楽価値の向上** — ストーリーのないゲームは「競技」にすぎない。ストーリーは競技に文脈を与え、抽象的なゴールではなく「劇的に意味のあるゴール」へ向かう進行感と感情的満足をもたらす。
2. **より広い層へのアピール** — ストーリーがなければ買わないプレイヤーも多い。ストーリーは開発コストを増やすが、潜在顧客も増やす。
3. **長時間プレイの維持** — 短いゲーム（例: Bejeweled）にストーリーは不要だが、数十時間に及ぶゲームではスコア稼ぎだけではプレイヤーの動機が持たない。ストーリーは新鮮さと多様性を提供する。
4. **マーケティングへの貢献** — ゲームプレイは静的な広告で説明しにくいが、キャラクターやストーリーの場面は効果的な販促素材になる。

ストーリーを入れるかどうかは、ゲームの**長さ**、**キャラクターの有無**、**リアリズムの度合い**、**感情的な深さ**によって判断する。Space Invaders のように一行のバックストーリーで十分なゲームもあれば、Dreamfall のように小説並みのストーリーが不可欠なゲームもある。

デザインルール：ゲームプレイが常に最優先 (Gameplay Comes First)。 ストーリーがゲームプレイを覆い隠してはならない。

キーポイント

- ・ストーリーは娯楽・動機付け・差別化・マーケティングの4つの観点で有効
- ・ジャンルとゲームの性質に応じて、ストーリーの深さを選択する
- ・ゲームプレイを犠牲にしてストーリーを優先してはいけない

2. キーコンセプト

2.1 ストーリーの要件

よいストーリーは単なる「出来事の連なり」ではなく、最低限以下の3条件を満たす必要がある。

条件	意味
信頼性 (Credible)	観客がその世界の前提を受け入れれば信じられる。キャラクターが共感・感情移入できるものであること
一貫性 (Coherent)	出来事が無関係・恣意的でなく、全体として調和していること
劇的意味 (Dramatically Meaningful)	プレイヤーが気にかける人物や事柄に関わること

2.2 インタラクティブストーリー

インタラクティブストーリーは「プレイヤーがアクションを貢献 (contribute) することで対話するストーリー」と定義される。重要なのは、**プレイヤーがプロットの変更を求めなくても**、その行為自体がストーリー体験の一部を構成するという点である。

インタラクティブストーリーには3種類のイベントがある。

- ・ **プレイヤーイベント** — プレイヤーが直接行うアクション。プロットに影響するものは「ドラマティック・アクション」と呼ばれる
- ・ **インゲームイベント** — コアメカニクスが生成するイベント（罾の作動、NPCの行動など）

- ・ **ナラティブイベント** — プレイヤーが内容を変更できないイベント（カットシーン等）

プロットの方角を変える力は **エージェンシー (agency)** と呼ばれる。

2.3 ナラティブ

ナラティブとは、デザイナーがプレイヤーに語る・見せるストーリーイベント、すなわち **非インタラクティブな提示部分**を指す。

ナラティブの主な役割： - プレイヤーが制御できない出来事の提示 - ゲーム世界の背景・文化・歴史の伝達 - 目標達成時の報酬（ムービーやカットシーン）

よく使われるナラティブブロック： - オープニングシーケンス（例: Half-Life のトラム乗車シーン） - エンディングシーケンス - レベル間シーケンス（ブリーフィングなど） - カットシーン（プレイ中の短い非インタラクティブ演出）

デザインルール：すべてのナラティブはスキップ可能にすること。 再プレイ時、既知のナラティブを強制視聴させてはならない。

デザインルール：アバターの制御を奪うな。 ナラティブ中にアバターがプレイヤーの望まない行動をとることは避ける。

キーポイント

- ・ よいストーリーは信頼性・一貫性・劇的意味の3条件を満たす
- ・ インタラクティブストーリーはプレイヤーが行動を「貢献」するストーリー
- ・ ナラティブは非インタラクティブな提示部分であり、ゲームプレイとのバランスが重要

3. ドラマティックテンションとゲームプレイテンション

3.1 ドラマティックテンション

ドラマティックテンション（劇的緊張）とは、「何か大切なことが懸かっている」という感覚と「次に何が起こるか知りたい」という欲求である。物語の**クライマックス**でテンションが頂点に達し、そこからテンションは下降に転じる。クリフハンガー（未解決の危機で章やエピソードを終わらせる手法）はテンションを維持する代表的な技法である。

3.2 ゲームプレイテンション

ゲームプレイテンション（遊びの緊張）もまた「何か大切なことが懸かっている」感覚だが、その源泉はプレイヤーの「チャレンジを克服できるかどうかの不確実性」にある。

3.3 両者の誤った類推

この2つは表面的に似ているが本質的に異なる。

	ドラマティックテンション	ゲームプレイテンション
源泉	キャラクターへの感情移入と好奇心	チャレンジ克服の不確実性
ランダム性への耐性	低い（ランダムな出来事は興味を失わせる）	高い（ポーカーやテトリスのように長く維持できる）
反復への耐性	低い（同じ出来事の繰り返しは退屈）	高い（反復的ゲームプレイでも緊張は続く）

デザインルール：ランダム性と反復はドラマティックテンションを破壊する。 ナラティブイベントをランダムに発生させたり、同じナラティブを繰り返してはならない。

キーポイント

- ・ドラマティックテンションはキャラクターへの共感から、ゲームプレイテンションはチャレンジの不確実性から生まれる
- ・ドラマティックテンションはランダム性・反復に弱く、ゲームプレイテンションは強い
- ・両者を混同してはならない

4. ストーリーテリングエンジン

ストーリーテリングエンジンは、コアメカニクス・ユーザーインターフェースと並ぶゲームの3大構成要素の1つであり、**ナラティブイベントをゲームプレイに織り込む**役割を担う（ストーリーのないゲームには不要）。



ストーリーテリングエンジンとコアメカニクスの関係（Figure 7.1）

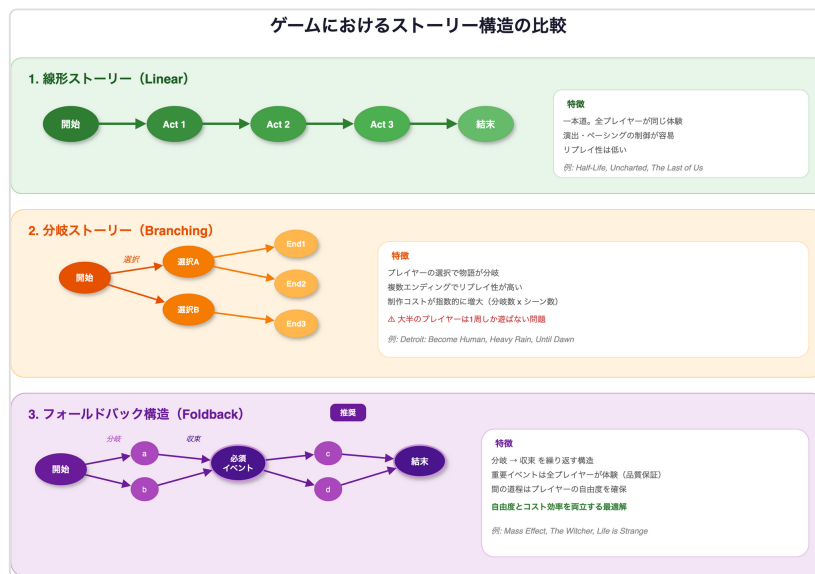
- ・ **コアメカニクス** はプレイヤーイベントとインゲームイベントを管理する
- ・ **ストーリーテリングエンジン** はナラティブイベントを管理し、ストーリーの進行状況を追跡する
- ・ 両者は**トリガー**を介して連携する。例: プレイヤーがレベルをクリア → コアメカニクスがストーリーテリングエンジンに通知 → ナラティブイベントが表示される → 終了後にコアメカニクスへプレイ再開を通知

ストーリーテリングエンジンはナラティブ再生中だけでなく、プレイ中も常にコアメカニクスからトリガーを受け取り、プレイヤーの重要な決定を記録している。

キーポイント

- ・ ストーリーテリングエンジンはナラティブとゲームプレイを1つのシームレスな体験に編み上げる
- ・ コアメカニクスとストーリーテリングエンジンはトリガーで双方向に連携する

5. 線形ストーリーと非線形ストーリー



ストーリー構造の比較 — 線形・分岐・フォールドバックの3パターン

5.1 線形ストーリー (Linear Stories)

プレイヤーがプロットやエンディングを変更できないストーリー。Half-Life や StarCraft が代表例。

利点： - コンテンツ量が少なく済む (1本のイベント列のみ) - ストーリーテリングエンジンが単純 - バグや矛盾が起きにくい - **より強い感情的インパクト** を実現できる (後述)

欠点： - プレイヤーにエージェンシーがない

5.2 非線形ストーリー (Nonlinear Stories)

プレイヤーの行動が将来のイベントに影響を与え、ストーリーの方向を変えられる構造。

5.2.1 分岐ストーリー (Branching Stories)

プロットが分岐点で複数の方向に分かれる。従来型のインタラクティブストーリーの古典的手法。



分岐ストーリーの構造 (Figure 7.2)

分岐の影響タイプ： - **即時影響 (Immediate)** — 決定が直ちにプロットに反映される (最も一般的) - **遅延影響 (Deferred)** — 序盤の決定がずっと後の分岐に影響する - **累積影響 (Cumulative)** — 一連の行動の蓄積が分岐を決定する (RPGの「善悪の評価」など)

デザインルール：結果を明確に伝えよ。 些細な決定に重大な結果を結びつけてはならない。遅延影響がある場合は事前にプレイヤーに警告する。

分岐ストーリーの重大な欠点： 1. **組み合わせ爆発 (Combinatorial Explosion)** — 分岐のたびにコンテンツ量が指数的に増大する。6回の分岐で21の分岐点と35の異なるブランチが必要になる 2. **キャラクターの生死などの重大イベントが独自のツリー全体を要求する** — 2人のキャラクターの生死で4通りのストーリーが必要 3. **全コンテンツを見るには繰り返しプレイが必要** — 悪いプレイを意図的にしなければ見られないコンテンツが生じる

5.2.2 フォールドバックストーリー (Foldback Stories)

分岐ストーリーと線形ストーリーの折衷案。プロットは分岐するが、やがて**不可避のイベント** (Inevitable Event) に収束し、再び分岐し、また収束する。The Secret of Monkey Island をはじめ多くのアドベンチャーゲームがこの形式を採用している。



フォールドバックストーリーの構造 (Figure 7.3)

- ・プレイヤーにはある程度のエージェンシーがあるが、特定のイベントは回避不能
- ・初回プレイではプレイヤーは自分の選択がすべてを決めていると感じる
- ・**現代ゲームで最も一般的な非線形ストーリー構造**

不可避イベントには「物語の転換点」を選ぶとよい。例：重要キャラクターの死、主人公が最終チャレンジに直面する瞬間など。

5.2.3 エマージェントナラティブ (Emergent Narrative)

Marc LeBlanc が提唱した概念で、プレイヤーのアクションとインゲームイベントだけからストーリーが生まれる手法。事前に書かれたナラティブブロックを持たず、ストーリーテリングエンジンも独立して存在しない。The Sims がその例に近い。

ただし現状ではコアメカニクスが自動的に信頼性・一貫性・劇的意味のあるストーリーを生成することは極めて困難であり、**研究段階の実験的技法**にとどまっている。

5.3 エンディング

- ・ **チャレンジ中心**のゲームではエンディングは1つで十分な場合が多い（うまくプレイすれば到達できる）
- ・ **選択中心**のゲーム（特に道徳的選択を含むもの）では複数のエンディングを用意すべき
- ・ 途中死亡などの「未熟なエンディング」は真のエンディングではなく、リソースを費やすべきでない

キーポイント

- ・ 線形ストーリーはコスト効率がよく感情的に強力だが、エージェンシーがない
- ・ 分岐ストーリーは自由度が高いが、組み合わせ爆発のコストが大きい
- ・ フォールドバックストーリーが業界で最も実用的な非線形構造
- ・ エンディングの数はゲームが提供する「劇的選択」の量に応じて決める

6. 粒度 (Granularity)

粒度とは、ゲームがプレイヤーにナラティブを提示する**頻度**のことである。

粒度	例	説明
粗い粒度	StarCraft, Wing Commander	ミッション間にのみナラティブを提示。20分～1時間以上に1回
細かい粒度		

粒度	例	説明
	The Secret of Monkey Island	パズルを解くたびにナラティブを提示。4～5分に1回
無限小の粒度（理想）	Half-Life（近似）	ナラティブとゲームプレイの区別がつかないシームレスな体験

キーポイント

- ・ 粒度はナラティブ提示の頻度を表す相対的な概念
- ・ アバターの行動にすべてのストーリーイベントを関連づけると、シームレスな体験に近づく

7. プロット進行メカニズム

ストーリーテリングエンジンにプロットを進めさせるトリガーには3つの主要なアプローチがある。

7.1 チャレンジ・選択としてのストーリー

ミッションやレベルの成功・失敗、あるいはプレイヤーの重要な意思決定がプロットを進める。StarCraft（成功時のみ進行）、Wing Commander（成功・失敗で異なる方向に進行）が該当する。

- ・ 粗い粒度のストーリーに適している
- ・ チャレンジに行き詰まるとストーリーが停止し、やや機械的な印象になる

7.2 旅としてのストーリー

アバターの移動がトリガーとなる。新しいエリアへの到着がプロット進行を引き起こす。多くのアドベンチャーゲームやRPGがこの形式。

利点： - 移動により常に新しい風景が提供され、新鮮さが自然に生まれる - プレイヤーがペースを制御できる

ジョセフ・キャンベルの「英雄の旅 (Hero's Journey)」は、このアプローチに特に適した物語構造として知られている。

線形ストーリーを旅として扱う場合、訪問済みエリアではもうドラマティックイベントが起きない。そのため、**一方通行のドア** (戻れない移動装置) がしばしば用いられる (例: The Secret of Monkey Island で大砲で島に上陸する場面)。

7.3 ドラマとしてのストーリー

時間の経過自体がプロットを進行させる。プレイヤーが何もしなくてもストーリーは進む。Night Trap や Facade がこの形式に該当する。

Night Trap では監視カメラを切り替えてモンスターの罠を作動させるのがプレイヤーの行動で、ストーリーは常に進行する。Facade ではプレイヤーの会話がストーリーの方向と結末に大きく影響する。

キーポイント

- ・ チャレンジ型：成功/失敗がトリガー。粗い粒度に適合
- ・ 旅型：移動がトリガー。アドベンチャー・RPGの定番
- ・ ドラマ型：時間がトリガー。リアルタイム進行する特殊な形式

8. インタラクティブストーリーの感情的限界

8.1 非線形ストーリーの感情的限界

プレイヤーに選択の自由を与えると、デザイナーが意図した最も感情的に強力な結末を体験しない可能性がある。例えばシェイクスピアのハムレットをゲーム化してプレイヤーに選択肢を与えると、「逃げる」「自殺する」など原作より劇的インパクトの薄い結末を選ぶかもしれない。

このため、非線形ストーリーでもエンディングだけは1つに固定するデザイナーは多い。プレイヤーはエンディングが変わると約束されていない限り、この妥協を受け入れる。

8.2 アバターベースゲームの感情的限界

アバターは一人称小説の語り手に似ており、「ゲーム終盤まで生き残るはず」とプレイヤーが暗黙に理解している。そのためアバターの死は「チャレンジ失敗」を意味するだけで感情的インパクトが薄い。

プレイヤーの感情を動かしたいなら、アバターではなくアバターの仲間を殺すべきである。

- ・ Planetfall: サイドキックのロボットが自己犠牲 — コンピュータゲーム初の感動的瞬間とされる
- ・ Final Fantasy VII: エアリスの死 — プレイヤーには防ぐ手段がなく、強い感情を呼び起こした

パーティベースのゲームはアバターベースより自由にキャストを退場させられる。

キーポイント

- ・ 非線形ストーリーでは最も強力な感情的結末を保証できない
- ・ アバターの死には感情的重みがないため、仲間キャラクターの死を活用する
- ・ 線形ストーリーは感情的パワーの点で非線形より優れる

9. スクリプト会話とダイアログツリー

9.1 スクリプト会話の仕組み

自然言語理解はAI研究の難題であるため、ゲームでは**スクリプト会話**を用いてNPCとの対話を実現する。プレイヤーは自由にテキストを入力するのではなく、**あらかじめ書かれた選択肢メニュー**から台詞を選ぶ。



会話メニューの例 — Neverwinter Nights 2 (Figure 7.4)

- ・ 選択肢には「外交的」「はったり」「威嚇的」などの態度が示される場合がある

- ・ NPCはプレイヤーの態度に応じて異なる反応を返す
- ・ 「もう一度聞く」 オプションで過去の会話を再確認できる

9.2 ダイアログツリーの構造

ダイアログツリーは分岐ストーリーに似た分岐データ構造で、各**ノード**に選択肢メニューがあり、各選択肢は「プレイヤーの台詞」と「NPCの応答」のペア（**エクスチェンジ**）を含む。



Neverwinter Nights 2 のダイアログ画面 (Figure 7.4 参照)

分岐ストーリーとは異なり、ダイアログツリーでは**矢印が前にも後ろにも**向くことができる（会話を繰り返すため）。ただし、分岐が収束しないと**組み合わせ爆発**が起きる点は同じである。

9.3 条件分岐とコンテキスト依存

ダイアログツリーでは、ゲームの状態（アバターのカリスマ値、外交スキルなど）に応じて以下が変化する。

- ・ NPCの応答が変わる（**条件分岐**: 「カリスマ > 12 なら応答A、それ以外は応答B」）

- ・プレイヤーに追加の選択肢が開放される（**条件エクステンジ**: 「外交スキル > 10 なら追加選択肢を表示」）

9.4 メニュー追加・削除方式

ダイアログツリーの別の実装アプローチとして、固定メニューを遷移する方式ではなく、各エクステンジを独立した項目として管理し、会話の進行に応じてメニューに**追加・削除**する方式がある。これにより「会話をいつでも終了」や「先ほどの話をもう一度」といった共通選択肢を効率的に管理できる。

キーポイント

- ・スクリプト会話は自然言語処理なしでNPCとの対話を実現する実用的手法
- ・ダイアログツリーは分岐データ構造で、プレイヤーの態度や能力値に応じた条件分岐が可能
- ・大規模なダイアログツリーでは組み合わせ爆発に注意が必要

10. その他の考慮事項

10.1 フラストレーテッド・オーサー症候群

本当は映画監督や小説家になりたかったデザイナーが陥る2つの典型的ミス。

1. **見かけだけの自由度** — 線形ストーリーなのにプレイヤーに大きな役割があるかのように偽装する（例: Critical Path）
2. **過剰なナラティブ** — プレイヤーを長時間座らせて観せるだけにする（例: The Longest Journey の20分間のモノログ）

デザインルール：映画監督ではなくゲームデザイナーであれ。 プレイヤーは行動するためにゲームを遊ぶ。ナラティブは3～4分以内に抑え、常にゲームプレイの方を多くする。

10.2 エピソード配信

マルチパートストーリーの3つの形式。

形式	特徴	テレビの例
アンリミテッドシリーズ	各エピソードが独立した完結ストーリー。順不同で楽しめる	刑事コロンボ
シリアル	プロットが複数エピソードにまたがる。各話末にクリフハンガー	昼ドラ（ソープオペラ）
リミテッドシリーズ	個別エピソードの完結話＋全体を貫くプロット。最終的に完結する	Babylon 5

本日のまとめ

1. **ストーリーはゲームプレイを補完する手段**であり、目的そのものではない
2. よいストーリーの3条件：**信頼性・一貫性・劇的意味**
3. **ドラマティックテンション**と**ゲームプレイテンション**は異なる概念であり、混同してはならない
4. **ストーリーテリングエンジン**がコアメカニクスとトリガーで連携し、ストーリーとゲームプレイを織り上げる
5. ストーリー構造は**線形**、**分岐**、**フォールドバック**、**エマージェントナラティブ**の4種類
6. **フォールドバックストーリー**が実務上最も広く使われる非線形構造
7. **粒度**はナラティブ提示の頻度を示し、シームレスな体験を目指すなら細かい粒度が理想
8. プロット進行のトリガーには**チャレンジ型・旅型・ドラマ型**の3方式がある
9. 非線形ストーリーとアバターベースには**感情的限界**がある
10. **スクリプト会話**と**ダイアログツリー**はNPCとの対話の標準的実装手法である

第6回：ユーザーインターフェース

1. ユーザーインターフェース（UI）とは何か

ユーザーインターフェース（UI）は、プレイヤーとゲームの内部処理（コアメカニクス）の間に位置し、ゲームを「見える・聞こえる・操作できる」ものにする仕組みである。UIの出来はプレイヤーがゲームに対して「満足か失望か」「楽しいか苛立たしいか」を判断する最大の要因の一つとなる。

UIの役割は大きく分けて3つある。

1. **入力の変換**：プレイヤーのボタン操作などの現実世界の行動を、インタラクションモデルに基づいてゲーム世界の行動に翻訳し、コアメカニクスに渡す。
2. **情報の提示**：コアメカニクスの内部データのうち、プレイヤーが必要とする情報を視覚・聴覚で提示する（フィードバック）。
3. **体験の創出**：単に情報を全て見せるのではなく、ゲームプレイとストーリーテリングの「体験」として適切にフィルタリングする。

用語の整理

用語	意味
視覚要素	画面上に表示される情報（メインビュー、インジケータ等）
音声要素	効果音、環境音、音楽、台詞など
制御要素	プレイヤーの入力を受け付ける仕組み（ボタン、メニュー等）
フィードバック要素	プレイヤーの行動結果や状態を伝える視覚・聴覚要素
コントローラボタン	入力デバイス上の物理ボタン

用語	意味
スクリーンボタン	画面上に表示され、マウスでクリックするボタン

設計上の注意：コアメカニクスを設計する際は、特定の入出力デバイスの性能に依存しない設計にすべきである。UIにハードウェア管理を任せ、コアメカニクスはハードウェア非依存に保つ。こうすれば移植時にUIだけを再設計すればよい。

キーポイント - UIはプレイヤーとゲーム内部の「橋渡し役」であり、入力変換・情報提示・体験創出の3機能を持つ - コアメカニクスはUI・ハードウェアから独立させて設計する

2. プレイヤー中心のインターフェースデザイン

ゲームのUIは一般的なソフトウェアのUIと異なり、プレイヤーに全ての情報を見せたり最大限の制御を与えたりするものではない。ゲーム体験として楽しいものにするために、情報と操作を適切に「媒介」する必要がある。

不要なイノベーションを避ける

デザインルール：標準が存在するなら、それを使え。既に実績のある操作体系があるのに、独自UIを無理に導入すると、プレイヤーは不慣れな操作に苛立ち、他にどれほど良い点があってもゲームを嫌いになる。

同ジャンルの優れたゲームを研究し、長年進化してきた標準的なフィードバック要素と操作方式を採用すべきである。もし新しいUIを導入するなら、チュートリアルレベルを作り、初心者と経験者の両方で徹底的にテストすること。

一般原則

原則	説明
一貫性	同じ機能には同じボタン・メニュー項目を割り当て、色・書体・レイアウトを統一する
適切なフィードバック	ボタン操作には即座に反応を返す。非活性ボタンでも音で応答する
プレイヤーが主導権を持つ	アバターを勝手に動かさない。プレイヤーは自分が操作している感覚を求めている
操作ステップの最小化	特殊技は最大3ボタン。メニュー階層は浅く保つ
操作の取り消し	ゲームバランスに影響しない限り、アンドゥを許可する
身体的負荷の軽減	頻繁な操作はアクセスしやすいボタンに割り当てる
短期記憶への配慮	常時必要な情報は常に表示。暗記を強制しない
関連情報のグループ化	関連するフィードバックと制御を画面上でまとめる
経験者向けショートカット	メニューの頻出コマンドにはキーボードショートカットを用意する

プレイヤーが知るべきこと

UIを設計する際は、「プレイヤーが問いかけるであろう質問」を基準に考える。

- ・ **今どこにいる？** → メインビュー＋ミニマップ＋環境音
- ・ **今何をしている？** → アバターの動作表示＋適切な動作音

- ・ **どんな課題に直面している？** → メインビューでの課題表示
- ・ **行動は成功した？失敗した？** → アニメーション＋効果音
- ・ **必要なリソースはある？** → 弾薬・資金・エネルギー等のインジケータ
- ・ **敗北の危険はある？** → HP・残り時間のインジケータ＋警告音
- ・ **進捗はどうか？** → スコア・達成率・チェックポイントの表示
- ・ **次に何をすべきか？** → ヒントや目標リスト
- ・ **結果はどうだった？** → レベルクリア時のスコア画面・報酬演出

デザインルール：負けたプレイヤーを嘲笑してはならない。プレイヤーは負けて既に落胆している。共感がプレイヤー中心設計の根幹にある。

キーポイント - 標準UIを尊重し、不要なイノベーションを避ける - 9つの一般原則（一貫性、フィードバック、主導権等）を遵守する - 「プレイヤーの問い」を基準にフィードバック要素を設計する

3. デザインプロセス

UIデザインは、ゲームデザインの「詳細化（elaboration）」段階の初期に行う。コンセプトフェーズが完了する前にUI設計を始めると、全体設計の変更でやり直しになるリスクがある。

ステップ1：ゲームプレイモードの定義

ゲームプレイモードとは、カメラモデル＋インタラクションモデル＋利用可能なゲームプレイ（課題と行動）の組み合わせである。

- ・ **主要モード**を最初に設計する（プレイヤーが最も長い時間を過ごすモード）
- ・ 次に他のモード（ストーリー配信、対話、ポーズメニュー等）を設計する
- ・ モード間の構造を**フローボード**で計画する
- ・ モード間で共通するUI要素を統一する

ステップ2：画面レイアウトの選択

メインビュー（ゲーム世界の表示）が画面上最大の要素であるべきである。メインビューの配置方法には2種類ある。

- ・ **ウィンドウ方式**：画面の一部にメインビュー、周囲にフィードバック・制御パネル
- ・ **フルスクリーン+オーバーレイ方式**：メインビューが画面全体を占め、UI要素がその上に重なる

ステップ3：情報の提示と操作の設計

1. プレイヤーに常時見せるべき情報を選定し、適切なフィードバック要素を選ぶ
2. プレイヤーの全行動に対して制御方式を設計する
3. 入力デバイスへのマッピングを行う

プロトタイプの構築

経験豊富なデザイナーは、最終仕様を書く前に必ずUIプロトタイプを構築しテストする。プレースホルダーのアートや音を使い、メニュー遷移やボタン応答を確認する。一度に全部作るのではなく、少しずつ構築・テスト・調整を繰り返す。

キーポイント - UIデザインは詳細化段階の初期に行う - 主要ゲームプレイモードから設計を始め、他モードへ展開する - プロトタイプの反復的な構築・テストが不可欠

4. 複雑さの管理

ゲームが高度化するにつれ、UIも複雑化する。複雑さを管理しなければ、プレイヤーはオプションを覚えきれず、画面が制御要素であふれてメインビューの余地がなくなる。

ゲームの簡素化

- ・ **抽象化**：詳細な要素を簡略化または削除する（例：燃料消費をシミュレーションしない → 燃料ゲージが不要になる）
- ・ **自動化**：プレイヤーの制御から外し、コンピュータに任せる（例：オートマチック変速 → 初心者モード/上級者モードで切替可能にする）

深さ vs. 広さ

- ・ **広いUI**：多くのオプションを同時に表示。学習コストは高いが慣れると効率的（例：航空機のコックピット）
- ・ **深いUI**：階層メニューで少しずつ選択肢を提示。学習コストは低いが操作に時間がかかる

理想的には両方を提供する。初心者向けの深いUI（メニュー）と、経験者向けの広いUI（ショートカットキー）を併用する。階層は最大3〜4段に抑え、頻繁に使う機能は1〜2ステップでアクセスできるようにする。

コンテキスト感応型インターフェース

現在の文脈で無意味なオプションを非表示にすることで複雑さを軽減する。例えば、マウスポインタをオブジェクトに向けると可能な操作に応じて形状が変わる（木に向けると斧の形になる等）。

曖昧さの回避

UIが正しく機能し見た目も良くても、ボタンやメニューの意味がわからなければ問題である。以下に注意する。

- ・ **アート偏重**：美しいが意味が伝わらないUI要素
- ・ **省スペースへの過度な圧縮**：小さすぎるアイコンは区別がつかない
- ・ **開発者の慣れ**：自分が作ったものの分かりやすさは自分では判断できない。必ず第三者にテストさせる

キーポイント - 抽象化・自動化でゲーム自体を簡素化するのが最優先 - 深い（メニュー型）と広い（一覧型）のUIを併用し、初心者と経験者の双方に対応する - コンテキスト感応で不要な選択肢を隠し、第三者テストで曖昧さを排除する

5. インタラクションモデル

インタラクションモデルとは、プレイヤーの入力とゲーム世界での行動の関係を定義するものである。

モデル	説明	例
アバター型	一人のキャラクター（アバター）を操作。アバターの周辺のみ影響を与えられる	アクションゲーム、FPS
マルチプレゼント型	ゲーム世界の複数箇所に同時に働きかけられる。俯瞰カメラが必要	チェス、RTS
パーティ型	小グループをまとめて操作。ポイント&クリック＋俯瞰視点が多い	RPG
出場者型	質問に回答し決定を下す。ナビゲーション不要	クイズゲーム
デスクトップ型	PCデスクトップを模倣。オフィス系のゲームに限定	経営シミュレーション

既存のモデルに収まるデザインが望ましい。独自モデルを作る場合は、操作を教えるための詳細なチュートリアルが必要になる。

キーポイント - インタクションモデルは5つの基本型がある - 標準的なモデルを採用することで学習コストを低減できる

6. カメラモデル

カメラモデルは、プレイヤーがゲーム世界をどのように見るかを定義する。映画のカメラになぞらえ、「仮想カメラ（virtual camera）」がゲーム世界を映すものとして設計する。

3D vs. 2D

2Dグラフィックスでは一人称・三人称視点は使えない。3Dグラフィックスの方が表現力は高いが、ツール・スキル・時間が不足しているなら、見栄えの良い2Dの方が見栄えの悪い3Dより常に望ましい。

一人称視点 (First-Person Perspective)

アバターの目線に固定されたカメラ。アバター自体は通常見えない。

利点	欠点
アバターのアニメーション開発コスト削減	アバターの外見カスタマイズの楽しみがない
カメラAIが不要	アバターのボディランゲージが見えず個性が伝わりにくい
照準が直感的	映画的カメラアングルによる演出ができない
環境との相互作用が容易	ジャンプなどの体操的動作のタイミングが取りにくい
	急な回転で酔いを引き起こしやすい

三人称視点 (Third-Person Perspective)

アバターの背後上方からカメラが追従する。アバターが見えるためキャラクター表現に優れるが、カメラ制御の設計課題が多い。

カメラの回転挙動（アバターが向きを変えた時）： - 常にアバターの背後に追従 → 速い回転で酔いの原因になる - 数秒遅れてゆっくり追従 → バランスが良い（Super Mario 64方式） - アバターが止まった時のみ追従 → 後退時に前方が見えない問題がある

障害物の問題：アバターが壁際に立つとカメラが壁の向こう側に行ってしまう。対策として、壁を半透明にする、カメラを上方に移動する、アバターの頭を半透明にする等の手法がある。

俯瞰視点 (Aerial Perspectives)

パーティ型やマルチプレゼント型のインタラクショナルモデルで使用される。

- ・ **トップダウン視点**：真上から見下ろす。地図的で親しみやすいが、建物の屋根と人の頭頂部しか見えない
- ・ **アイソメトリック視点**：斜め30度から見下ろす2D表示。建物の側面も見え、没入感が高い
- ・ **自由カメラ (Free-roaming)**：3Dエンジンによる俯瞰。上下移動・回転・ズームが自由にでき、遠近法も正確
- ・ **コンテキスト感応カメラ**：場面に応じて自動的に最適なアングルに切り替わる。映画的演出に優れるが実装コストが高く、高速アクション中に突然視点が変わるとプレイヤーを混乱させる

デザインルール：激しいアクション中はカメラを予期せず動かしてはならない。固定的で予測可能な視点を使うこと。



カメラモデルの例

キーポイント - カメラモデルはインタラクショナルモデルと密接に関連する - 一人称は没入感が高いが制約も多く、三人称はキャラ表現に優れるがカメラAIの設計が必要 - 俯瞰系は自由カメラが現代の主流。コンテキスト感応カメラは演出力が高いが設計負荷が大きい

7. 視覚要素

メインビューの表示方式

方式	特徴
ウィンドウ表示	メインビューを画面の一部に配置。情報量は多いが没入感が低下
不透明オーバーレイ	メインビュー全画面表示の上にUI要素を重ねる。消えた時の視界が広い
半透明オーバーレイ	背景が透けて見えるが、テキスト表示には不向き（背景と文字が混ざる）

フィードバック要素（インジケータ）

プレイヤーにゲームの内部状態を伝える視覚要素。ゲームのテーマに合ったものを選ぶ。

種類	タイプ	説明	例
数字表示	一般数値	大きな数値を省スペースで表示。一瞬の判読には不向き	スコア、弾薬数
針ゲージ	一般数値	一目で読めるが画面占有が大きい	速度計、タコメーター
パワーバー	一般数値	棒グラフ状。緑→黄→赤で危険度を直感的に伝達	HP、残り時間
小さな繰り返し	小整数	アイコンの個数で残数を表示（最大5つ程度）	残機、手榴弾

種類	タイプ	説明	例
色つきライト	記号	数ピクセルで二値・三値を表示	オン/オフ状態
アイコン	記号	少数の選択肢を小スペースで表現	季節、状態異常
テキスト	記号	抽象概念を表現可能。ただしローカライズコストが高い	アドバイザーの助言

ミニマップ

ゲーム世界のミニチュア版。2種類の表示規約がある。

- ・ **ワールド指向**：世界全体を北を上にして表示。現在のメインビュー範囲を矩形で示す
- ・ **キャラクター指向**：アバターを中心に据え、アバターの向きが常に上。レーダースクリーンとも呼ばれる

キャラクターポートレート

俯瞰視点で顔が見えにくい時に小ウィンドウで表情を表示する。Doomのアバターポートレートは被ダメージに応じて血まみれになり、HPフィードバック要素としても機能した。

テキストとローカライズ

- ・ テキストは画像に埋め込まず、必ずテキストファイルに分離して管理する
- ・ 最小文字高は12ピクセル（日本語は16ピクセルが望ましい）
- ・ 全て大文字のテキストは読みにくく、SHOUTING（叫び）に見える。警告時のみ使用する



UIの視覚要素例

キーポイント - メインビューを最大要素とし、フィードバック要素は目的に応じた形式を選択する - ミニマップはワールド指向とキャラクター指向の2種がある - テキストは画像から分離し、ローカライズを前提に設計する

8. 音声要素

効果音

ゲーム世界の行動や出来事に対応する音。UIのスクリーンボタンにもクリック音などの聴覚フィードバックを付与すべきである。危険の接近を音で先に知らせる映画的手法も有効である（足音、銃のコッキング音等）。

振動（ランブル）

コントローラの振動機能は、爆発などの大きなイベントや壁への衝突などの否定的イベントに使う。常用するとプレイヤーが無視するようになり、手にしびれが生じる。

環境音

メインビューが「ここはどこか」を視覚で伝えるように、環境音は聴覚で伝える。都市の交通音、ジャングルの鳥の声などである。一人称・三人称ゲームでは**ポジショナルオーディオ**（音源の空間的位置を再現する立体音響）を使うべきである。

音楽

音楽はゲームのトーンとペースを設定する。文化的メッセージが強いため、テーマとの整合に注意する。長時間の繰り返しに耐えられる楽曲を選び、ダイナミックレンジの広い曲は避ける。**アダプティブミュージック**（ゲーム状況に応じて動的に変化する音楽）や**レイヤリング**（複数の楽曲を同時に異なるボリュームで合成）などの技法がある。

台詞とナレーション

- ・繰り返される台詞はすぐに飽きられるため、同じ内容に複数のバリエーションを用意する
- ・脚本と演技の質は極めて重要。下手な声優は即座に没入感を破壊する

プレイヤーは音楽・効果音・会話の音量を個別に調整できるようにすべきである。

キーポイント - 効果音はUI操作にも付与し、音で危険を先行告知する - 環境音とポジショナルオーディオで空間認識を支援する - 音楽はテーマとの整合性を保ち、繰り返し耐性を確保する

9. 入力デバイス

ゲームに同梱される標準的な入力デバイスに対して設計すべきであり、特殊なデバイスを前提にすると市場が限定される。

デバイスの分類

分類軸	種類	説明
データ型	バイナリ / アナログ	オン・オフのみ / 連続値
次元	1D / 2D / 3D	送信するデータの数
値の種類	絶対値 / 相対値	

分類軸	種類	説明
		ニュートラル位置からの距離 / 前回位置からの移動量

主な入力デバイス

デバイス	次元	データ型	特徴
コントローラボタン/ キー	1D	バイナリ	単発・連射・継続の3つの使い方がある
感圧ボタン	1D	アナログ	押す強さで値が変わる
D-pad	2D	バイナリ	8方向のみ。粗い制御
ジョイスティック	2D	アナログ	細かい操舵に最適。自動復帰するため精密ポインティングには不向き
マウス	2D	相対アナログ	精密な位置指定。無限の移動範囲。ニュートラル位置がない
タッチスクリーン	2D	絶対アナログ	

デバイス	次元	データ型	特徴
			直感的だが長時間使用で指が疲れる
加速度センサー	3D	アナログ	コントローラの動き・傾きを検出（Wiiリモコン等）
GPS	3D	絶対値	屋外の位置情報。拡張現実ゲームに使用

キーポイント - 標準同梱デバイスを前提に設計する - デバイスの次元・データ型・値の種類を理解し、ゲームの操作に最適なものを選択する - マウスは精密操作に優れ、ジョイスティックは操舵に最適と、それぞれ得意分野が異なる

10. ナビゲーションメカニズム

ナビゲーションメカニズムは、プレイヤーの操作入力とアバターの画面上の動きの関係を定義する。この関係は一貫性と予測可能性が極めて重要であり、突然のカメラアングル変更などは操作ミスの原因となる。

画面指向ステアリング

ジョイスティックを上倒すとアバターが画面上方に移動する方式。

- ・ **トップダウン/アイソメトリック**：上下左右がそのまま画面の上下左右に対応（最もシンプル）
- ・ **サイドスクロール**：左右で移動、ジャンプは別ボタン

- ・ **3D (Crash Bandicoot方式)** : 上で画面奥へ、下で手前へ。サイドスクローラーに奥行きを加えた感覚

アバター指向ステアリング

ジョイスティック上でアバターが「現在向いている方向」に前進する方式。一人称ゲームでは唯一の選択肢。

- ・ **ジョイスティック/D-pad** : 上＝前進、下＝後退、左右＝回転（移動はしない。斜め入力で回転＋前進）
- ・ **マウス＋キーボード (FPS標準)** : マウスで視線方向を制御、WASD（またはIJKL）キーで移動。ストレイフィング（横移動）が可能

飛行

3次元移動には2Dデバイスでは足りないため、追加デバイスが必要。

- ・ **固定翼機** : ジョイスティックでピッチ・ロール、スロットル（スライダー/キー）で速度
- ・ **ヘリコプター** : ジョイスティックで回転・前後移動、別コントロールで上昇・下降
- ・ **宇宙船** : 固定翼機と同様だが、左右でヨー（水平回転）する場合もある

ポイント＆クリックナビゲーション

俯瞰視点やコンテキスト感応カメラで使用。プレイヤーが目的地をクリックするとアバター/ユニットがパスファインディングで自動的にそこへ移動する。RTSやパーティ型RPGで多用される。ウェイポイント（中間目標点）を設定できるようにすると、プレイヤーが経路をある程度制御できる。

キーポイント - 画面指向とアバター指向の2大方式があり、カメラモデルに合わせて選択する - FPSではマウス＋キーボードによるアバター指向ステアリングが標準 - 俯瞰視点ではポイント＆クリック＋パスファインディングが効果的

11. カスタマイズ

プレイヤーが入力デバイスを自分好みに設定できる機能は、設計コストに対して効果が非常に高い。

必須のカスタマイズ機能

- ・ **マウスボタンの左右入替**：左利きプレイヤーへの対応
- ・ **マウス/ジョイスティックの上下反転**：上で見上げ or 上で見下ろしの好みに対応
- ・ **キー/ボタンのリマッピング**：プレイヤーが好みのキー配置に変更できる機能。必須アクションが未割当のまま終了しようとした場合は警告する
- ・ **音量の個別調整**：音楽・効果音・音声をそれぞれ独立して調整可能にする

自由度の原則

同じ次元・同じデータ型を持つデバイス間であれば、基本的にプレイヤーが相互に入れ替えられるようにすべきである。ただし、自動復帰の有無や移動範囲の制限が異なるデバイス間では、一部の機能や利便性が失われることがある。

キーポイント - カスタマイズ機能は低コストで高い満足度をもたらす - マウスの左右入替、上下反転、キーリマッピングは必須機能として実装する

本講のまとめ

1. **UIの本質**：プレイヤーとゲーム内部の媒介であり、体験を創出するもの
2. **プレイヤー中心設計**：標準を尊重し、9つの一般原則を守り、プレイヤーの問いに答える
3. **デザインプロセス**：モード定義 → レイアウト → フィードバック/制御設計 → プロトタイプ
4. **複雑さの管理**：抽象化・自動化・深さと広さのバランス・コンテキスト感応
5. **インタラクションモデル**：アバター型・マルチプレゼント型・パーティ型・出場者型・デスクトップ型
6. **カメラモデル**：一人称・三人称・俯瞰（トップダウン/アイソメトリック/自由カメラ/コンテキスト感応）
7. **視覚要素**：メインビュー方式、インジケータの種類、ミニマップ、テキスト設計
8. **音声要素**：効果音・振動・環境音・音楽・台詞の設計原則
9. **入力デバイス**：次元・データ型・値の種類で分類し、操作に最適なデバイスを選ぶ
10. **ナビゲーション**：画面指向/アバター指向/ポイント&クリックの3方式
11. **カスタマイズ**：低コスト高効果。キーリマッピングと入力反転は必須

ewpage

第7回：ゲームプレイ

1. はじめに

ゲームプレイとは、ゲームがプレイヤーに提示する**チャレンジ（課題）**と、それを克服するための**アクション（行動）**の関係で構成される。ゲームにはゲームプレイ以外のアクションも含まれるが、ゲームプレイの本質はチャレンジとアクションの相互関係にある。

本章では、ゲームを面白くする方法、チャレンジの階層構造、スキル・ストレス・難易度の概念、よく使われるチャレンジの種類、アクションの設計、そしてゲームの保存について学ぶ。

2. ゲームを面白くする方法

2.1 実行力はイノベーションに勝る

ゲームの面白さの大部分は、創造性やアイデアではなく**実行の質**によって決まる。退屈、フラストレーション、不快感の多くは、悪いアイデアではなく**悪い実行**から生まれる。以下に、面白さへの貢献度が高い順に要素を示す。

1. **基本的なミスの回避**（最重要）：バグのあるプログラム、低品質なUI、粗悪な音楽やアートはプレイヤーの楽しさを台無しにする
2. **チューニングとポリッシュ**：細部への配慮と完璧さの追求が、良いゲームと凡庸なゲームを分ける
3. **ゲーム前提の想像力豊かなバリエーション**：基本要素を活かした楽しい体験の構築（レベルデザイナーの仕事）
4. **真のデザインイノベーション**：面白さの源泉の約5%に過ぎない

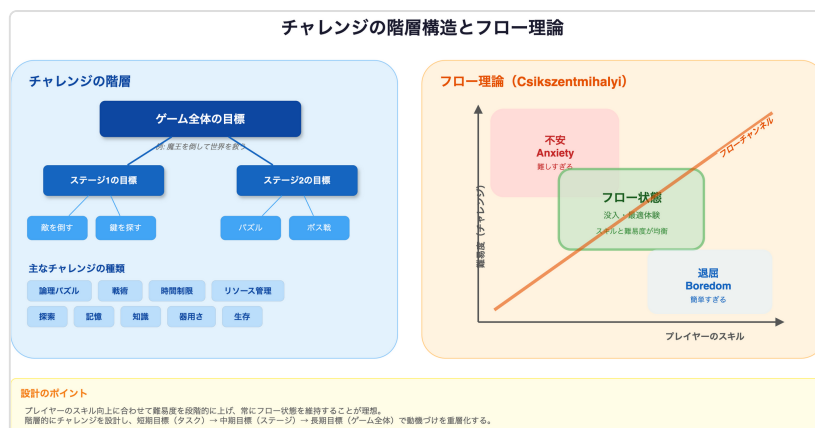
2.2 ファンファクターを見つける原則

面白さを生む確実な公式は存在しないが、以下の原則を守ることによりリスクを低減できる。

- ・ **ゲームプレイ最優先**：グラフィックやストーリーよりも、プレイヤーが楽しめる行動を優先する
- ・ **機能は完成させるか、省く**：壊れた機能を搭載するのは、機能を省くより遥かに悪い
- ・ **プレイヤー中心の設計**：すべての決定をプレイヤーの視点から検証する
- ・ **ターゲットを知る**：異なるプレイヤー層が求めるものは異なる
- ・ **面白くない部分は抽象化・自動化する**：現実世界を模すゲームでも退屈な部分は省く
- ・ **ビジョンに忠実であれ**：機能追加で市場を広げようとする、本来の品質が損なわれる
- ・ **調和・優雅さ・美しさを追求する**：美的完成度は楽しさを一段引き上げる

キーポイント - ゲームの面白さの大部分はイノベーションではなく、バグの排除やチューニングなどの実行力から生まれる - 壊れた機能は省いた機能より有害。プレイヤー中心の設計を徹底すること

3. チャレンジの階層構造



チャレンジの階層構造とフロー理論 — 目標階層 + Csikszentmihalyiのフロー状態図

3.1 階層の基本

ほとんどのゲームで、プレイヤーは複数のチャレンジに同時に直面する。これらは**階層構造 (Hierarchy of Challenges)** として整理される。

階層レベル	例
最上位	ゲーム全体のクリア
ミッション	現在のレベルのクリア
サブミッション	レベル内の個別目標
アトミックチャレンジ	目の前の敵を倒す、鍵を見つける等

アトミックチャレンジとは、それ以上分割できない最小単位の課題である。アトミックチャレンジがサブミッションを構成し、サブミッションがミッションを、ミッションがゲーム全体のクリアを構成する。



図：スキルとストレスの関係を示すグラフ。ゲームジャンルごとに要求されるスキルとストレスの度合いが異なる

3.2 プレイヤーへのチャレンジの伝え方

チャレンジには、ゲームが直接伝える**明示的チャレンジ (Explicit)** と、プレイヤーが自ら発見する**暗黙的チャレンジ (Implicit)** がある。

- ・ **最上位と最下位**は明示的に伝える（勝利条件とアトミックチャレンジの対処法）
- ・ **中間レベル**は暗黙的に残す（プレイヤーが探索・観察・推理で発見する部分が面白さになる）

デザインルール：プレイヤーがどのような手段で勝利条件を達成しても、それを認めること。想定外の方法での勝利もズルではない。

3.3 複数のクリア方法を用意する

優れたゲームは勝利への道を複数用意する。一つの「正解」しかないゲームは、デザイナーの意図を読む作業になってしまう。例えば戦争ゲームで「旗を奪取する」という勝利条件に対して、ステルスアプローチ、攻撃的アプローチ、防御的アプローチなど異なる戦略を選べるようにする。

3.4 同時発生するアトミックチャレンジ

プレイヤーが複数のアトミックチャレンジに同時に直面する場合、注意力が分散する。時間制限下で同時に複数の課題に対処する必要があると、ゲームのストレスが増大する。SimCityのように、警察・電力・病院・水道など多数のサービスを同時にバランスさせるゲームでは、終わりのないジャグリング行為が求められる。

キーポイント - チャレンジは階層構造で整理される。最上位の勝利条件から最下位のアトミックチャレンジまで - 勝利条件とアトミックチャレンジは明示的に、中間レベルは暗黙的に設計する - 同時に直面するアトミックチャレンジが多いほどストレスが増す

4. スキル・ストレス・難易度

4.1 固有スキル (Intrinsic Skill)

固有スキルとは、時間制限なしでチャレンジを克服するために必要なスキルレベルである。時間的プレッシャーを取り除いた状態で測定する。

- ・アーチェリー：的を遠くする・小さくすると要求スキルが上がる（時間は関係ない）
- ・数独：難易度は手がかりの数で決まる（制限時間ではない）
- ・トリビア：知識の有無であり、時間を増やしても変わらない

4.2 ストレス

ストレスとは、時間的プレッシャーがプレイヤーの課題達成能力に与える影響の度合いである。制限時間が短いほどストレスが高まる。

- ・**テトリス**：固有スキルは低い（時間があれば簡単）が、時間的プレッシャーによりストレスが高い
- ・**ゴルフ**：高いスキルを要求するが、時間的プレッシャーは低い

ストレスは身体的にも影響し、心拍数やアドレナリンを上昇させる。ゲームのペーシングを調整してプレイヤーに休息を与えることが重要である。

4.3 絶対的難易度 (Absolute Difficulty)

絶対的難易度 = 固有スキル + ストレス の組み合わせで決まる。

難易度レベルを設定する際には以下を考慮する：

- ・ターゲット層に応じて調整（若年層はストレス耐性が高い、高齢層はスキルは高いが反射神経は低下）
- ・スキルとストレスの逆相関を保つ：スキル要求が高いほど時間を与え、スキル要求が低いほど時間を短くする

キーポイント - 固有スキル：時間制限がなくとも必要な能力レベル - ストレス：時間的プレッシャーの影響 - 絶対的難易度はこの2つの組み合わせ。難易度調整では両方を意識する

5. よく使われるチャレンジの種類

5.1 身体的協調チャレンジ

手と目の協調を中心とした身体能力のテスト。

サブカテゴリ	内容	代表例
速度・反応時間	素早い入力・即座の反応	Track & Field, Quake
正確性・精密さ	ステアリング、射撃等	レーシング、FPS、Fable
物理の直感的理解	制動距離、加速率等の体得	レーシング、ビリヤード、ダーツ
タイミング・リズム	正しいタイミングでの入力	Dance Dance Revolution, Guitar Hero
コンビネーション技	複雑なボタン入力の連続	格闘ゲーム

難易度調整のポイント：身体的協調チャレンジでは、主に**時間的プレッシャーの増減**で難易度を調整する。正確性については物理エンジンの許容誤差を調整する方法もある。

5.2 論理・数学チャレンジ

- ・ **形式論理パズル**：パズルの定義内にすべての情報が含まれ、推論のみで解ける（例：ルービックキューブ）
- ・ **数学的チャレンジ**：確率的推論が求められる（例：ポーカー、Microsoft Hearts）



図：Microsoft Hearts。不完全情報下での確率的推論が求められるカードゲーム

デザインルール：**試行錯誤でしか解けないパズルは避ける**。プレイヤーが実験から推論できるようにすること。

5.3 レースと時間プレッシャー

レースは「他者より先に何かを達成する」チャレンジ。時間プレッシャーは戦略的思考を阻み、直接的・力任せの解法を促す。時間プレッシャーを加える場合は、固有スキルの要求を下げてバランスをとること。

5.4 事実知識チャレンジ

トリビア・クイズゲーム以外では、勝利に必要な知識はゲーム内で提供すべきである。外部知識を要求する場合は事前に明示すること。

5.5 記憶チャレンジ

ゲーム内で見聞きした情報の記憶を試す。アドベンチャーやRPGでよく使われる。難易度調整は、記憶時間と想起までの間隔で行う。

5.6 パターン認識チャレンジ

視覚的・聴覚的なパターンや行動パターンを見抜く能力を試す。典型例はアクションゲームの敵の行動パターン分析。パターンを短く・単純にすれば易しく、長く・複雑にすれば難しくなる。

5.7 探索チャレンジ

探索はそれ自体が報酬となるが、チャレンジなしの探索は単なる観光である。

種類	内容
空間認識	

種類	内容
	複雑な空間の把握。マップ提供で難易度を下げられる
鍵付きドア	進行を阻む障害物。鍵探し・パズル解き・戦闘等で開く
トラップ	プレイヤーを害する装置。発見・回避・解除が楽しさ
迷路・非論理空間	類似した部屋の構造を把握する。手がかりのない迷路は悪い設計
テレポーター	突然の場所移動。予測可能・可逆にすれば易くなる
隠しオブジェクト	アイテムの搜索。イースターエッグは勝利に不要だが発見の喜びを与える

5.8 コンフリクト（対立）

力の直接対決を伴うチャレンジ。戦闘だけでなく、チェッカーのような非暴力ゲームも含む。

- ・ **戦略（Strategy）**：状況分析に基づく計画立案。完全情報ゲーム（チェス等）では純粋戦略、不完全情報下では応用戦略が必要
- ・ **戦術（Tactics）**：計画の実行と予期せぬ事態への対応
- ・ **兵站（Logistics）**：資源の生産・輸送・管理。RTS における武器生産、RPG における所持品管理
- ・ **生存と敵の削減**：自軍の維持と敵の排除
- ・ **脆弱なユニットの防衛**：チェスの王のような守るべき存在の保護
- ・ **ステルス**：発見されずに行動する能力。Thief: The Dark Projectが代表例

5.9 経済チャレンジ

資源の生成・移動・保管・交換・消費のシステムが経済である。FPS の弾薬・体力管理のような単純なものから、SimCity のような全体が経済で構成されるゲームまで幅広い。

- ・ **資源の蓄積**：富やポイントの獲得（Monopoly等）
- ・ **バランスの達成**：複数資源の均衡維持（The Settlers等）
- ・ **生き物の世話**：個体の欲求を限られた資源で満たす（The Sims, Spore等）

5.10 概念的推論とラテラルシンキング

- ・ **概念的推論**：外部知識と推論力を組み合わせて解く（推理ゲーム等）
- ・ **ラテラルシンキング**：明らかな解法が使えない状況で、物事の別の用途を発想する（アドベンチャーゲーム等）

ラテラルシンキングパズルでは、一般常識の範囲を超えない知識に留めること。ヒントや手がかりを用意することも重要。

キーポイント - チャレンジの種類は多岐にわたる：身体的協調、論理、知識、記憶、パターン認識、探索、対立、経済、概念的推論 - 各チャレンジの難易度は、固有スキルの要求と時間的プレッシャーの調整で制御する - 試行錯誤でしか解けないパズルは避け、プレイヤーが推論できる設計にする

6. アクション

6.1 アクションとは

アクションはゲームの「動詞」である。プレイヤーの入力がUIを通じてゲーム世界で引き起こす直接的なイベントを指す。「走る」「跳ぶ」「撃つ」「買う」「建てる」がアクションである。一方、プレイヤーの行動の結果として間接的に起きるイベントはアクションではない。

チャレンジには階層構造があるが、**アクションには階層構造がない**。アクションは入力デバイスに直接結びつく低レベルの操作として定義される。プレイヤーはこれらの基本アクションを組み合わせ、上位のチャレンジに対処する。

6.2 アクションの設計手順

1. **プレイヤーの役割から出発**：役割に関連する動詞をリストアップする
2. **アトミックチャレンジから逆算**：各チャレンジをどうやって克服するかを具体的に書き出す
3. **上位チャレンジの確認**：定義済みのアクションで上位チャレンジも達成できるか検証する
4. **ゲームプレイ以外のアクション**も検討する
5. 各ゲームプレイモードごとにこのプロセスを繰り返す

少数のアクションで多様なチャレンジに対応できるゲームが優れている（例：ルービックキューブは「面を90度回転」の1アクションのみ）。

6.3 ゲームプレイ以外のアクション

種類	内容
自由な遊び	課題と無関係だが楽しい行動（観光、クラクションを鳴らす等）
創造・自己表現	アバターのカスタマイズ、建設等
社交	マルチプレイヤーでの会話、グループ形成
ストーリー参加	NPC との対話、プロット選択
ソフトウェア制御	カメラ操作、一時停止、セーブ、音量調整等

キーポイント - アクションはゲームの「動詞」で、入力デバイスに直結する低レベル操作 - 少数のアクションで多様なチャレンジに対応する設計が理想的 - ゲームプレイ以外のアクション（自由な遊び、創造、社交等）も重要

7. ゲームの保存

7.1 保存の意義

ゲームの保存は、ある時点のゲーム世界のスナップショットを記録し、後からその時点に戻ってプレイを再開する機能である。保存が必要な主な理由は以下の3つ。

1. **中断と再開**：40時間のゲームを一気にプレイすることは現実的でない
2. **致命的ミスからの回復**：アバターの死亡等からの復帰
3. **代替戦略の探索**：異なるアプローチを試す機会を提供

7.2 没入感とストーリーテリングへの影響

保存は没入感を損なう側面もある。ゲーム世界の外側で行われる行為であり、「過去に戻れないのが現実」という前提を破壊する。分岐ストーリーでは、やり直しによってドラマチックな緊張感が失われる。しかし、保存の利点はこれらの欠点を上回る。

7.3 保存方式の比較

方式	特徴	没入感への影響
パスワード	レベルクリア時にパスワード付与。ストレージ不要の環境向け	低い
ファイル/スロット保存	任意時点で名前付きスロットに保存。最も一般的	高い（ファイル管理画面が世界観を壊す）
クイックセーブ	ボタン一つで即時保存。FPS等で多用	低い（ゲーム世界を離れない）
自動保存/チェックポイント	通過時に自動保存。プレイヤーの操作不要	最小（ただしプレイヤーの制御が限定的）

7.4 保存を制限すべきか

一部のデザイナーは「保存がゲームを簡単にしすぎる」として保存を制限するが、これは**怠惰な設計**である。保存を禁止するのではなく、より良いチャレンジを設計すべきである。レベル終盤のミスで最初からやり直しを強いることは、プレイヤーの時間を浪費しフラストレーションを生む。

キーポイント - 保存は中断・再開、ミスからの回復、戦略探索のために不可欠 - 保存方式ごとに没入感への影響が異なる。クイックセーブは没入感を保ちやすい - 保存の制限は楽しさを加えず難しさだけを加える。怠惰な設計の典型

8. 本章のまとめ

本章では、ゲームプレイを構成する中核概念を体系的に学んだ。

1. **面白さの源泉**：イノベーションよりも実行の質。バグの排除、チューニング、プレイヤー中心の設計が重要
2. **チャレンジの階層構造**：ゲーム全体の勝利条件からアトミックチャレンジまでの階層。明示的/暗黙的チャレンジのバランスが鍵
3. **スキル・ストレス・難易度**：固有スキルと時間的プレッシャー（ストレス）の組み合わせが絶対的難易度を決定する
4. **チャレンジの種類**：身体的協調、論理、知識、記憶、パターン認識、探索、対立、経済、概念的推論など多様な種類がある
5. **アクション**：ゲームの「動詞」。少数の基本アクションで多様なチャレンジに対応する設計が理想
6. **ゲームの保存**：プレイヤーの利便性を最優先し、保存方式は没入感とのバランスで選択する

ゲームデザイナーの仕事は、これらの要素を組み合わせ、ターゲットプレイヤーにとって最も楽しい体験を構築することである。

第8回：コアメカニクス

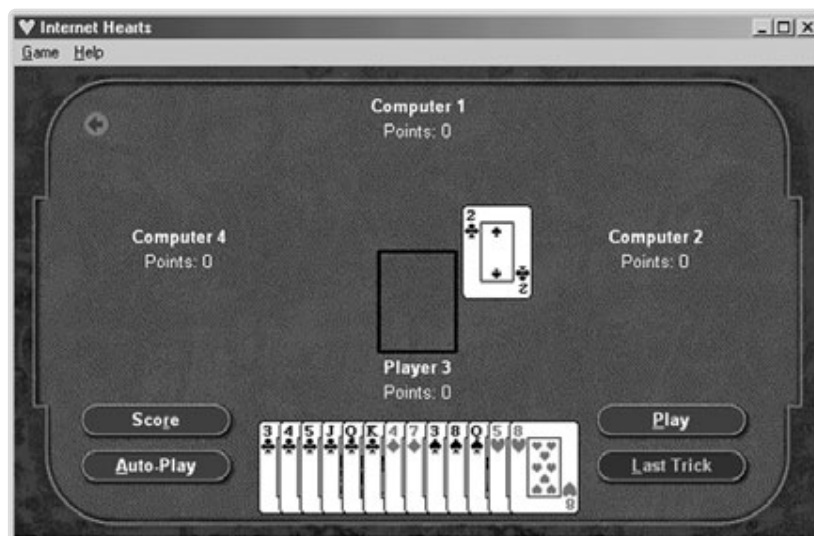
“すべてのルールの中で最も偉大なルールとは、ただ人を楽しませることではないか？” —— モリエール

1. コアメカニクスとは何か

1.1 定義

コアメカニクスとは、ゲームのルールと内部動作を正確に定義するデータとアルゴリズムの総体である。

デザインの初期段階では「沼を通り抜けるのに時間がかかりすぎるとペナルティがある」といった曖昧な記述で十分だが、コアメカニクスの段階では「アバターが沼に入ると黒いキノコが毒ガスを放出し、3秒ごとに1ゲームワールドインチの速度で画面下部から上昇、3分後にアバターの顔の高さに達したら死亡する」というレベルまで具体化する必要がある。



Internet Hearts のゲーム画面：コアメカニクスがルールを実装し、UIを通じてプレイヤーに提示する例

1.2 ルールからコアメカニクスへ

モノポリーの印刷ルールは3ページに満たないが、コンピュータゲームとして実装するには、各物件の価格、家賃テーブル、ボードのレイアウト、チャンスカードの全効果など、はるかに詳細な仕様が必要となる。これがルール（概要）とコアメカニクス（完全な仕様）の違いである。

1.3 コアメカニクスの所在

コアメカニクスの実装は開発フェーズによって変化する。

段階	実装形態	変更方法
初期設計	デザインドキュメント（自然言語）	ドキュメント編集
プロトタイプ	スプレッドシート／紙プロトタイプ	値の調整
開発段階	プログラムコード＋データファイル	コード変更

プレイヤーはコアメカニクスを直接目にすることはない。画面上の何かを指して「これがコアメカニクスだ」とは言えない。プレイヤーはゲームの振る舞いからメカニクスの機能を推測するのみである。

設計原則：ゲームを設計せよ、ソフトウェアを設計するな コアメカニクスはゲームエンジンが「何をするか」を定めるが、「どのようにするか」はプログラマに委ねる。

1.4 コアメカニクスの機能

プレイ中、コアメカニクス（ゲームエンジンとして実装されたもの）は以下を担う。

- ・ **内部経済の運用**：ゲーム経済の基盤となるリソースの生産・分配・消費を管理する
- ・ **能動的チャレンジの提示**：レベルデザインの指定に従い、UIを通じてプレイヤーにチャレンジを出す
- ・ **プレイヤーアクションの受理と処理**：入力を受け、ゲーム世界への影響を計算する

- ・ **勝利・敗北の検出**：終了条件の判定と結果の適用
- ・ **NPCのAI制御**：ノンプレイヤーキャラクターや人工的な対戦相手の行動制御
- ・ **ゲームモードの切り替え**：現在のモードを管理し、UIに更新を通知する
- ・ **ストーリーテリングエンジンへのトリガー送信**：プロットに影響するイベント発生時の通知

1.5 リアルタイムとターン制の違い

リアルタイムゲームでは、メカニクスはプレイヤーの行動有無にかかわらず動き続ける「生きた世界」を定義する。AI駆動のキャラクターは自律行動し、罾は発動条件をチェックし続ける。

ターン制ゲームでは、プレイヤーがターンを終えるまでメカニクスは待機し、行動後にその効果を計算する。プロセスを設計する場合でも、次のプレイヤーのターンのために安全に中断できるポイントを含める必要がある。

キーポイント - コアメカニクスはゲームのルールと内部動作をデータ+アルゴリズムで正確に定義したもの - プレイヤーはメカニクスを直接見ないが、ゲーム体験のすべてがメカニクスから生まれる - リアルタイムでは連続プロセス、ターン制ではイベントの連鎖として設計する

2. キーコンセプト：リソース・エンティティ・属性

2.1 リソース (Resources)

リソースとは、ゲームが移動・交換でき、数値として扱える物体や素材の「型（タイプ）」を指す。特定のビー玉ではなく「ビー玉という概念」がリソースであり、プレイヤーのポケットの中の15個のビー玉は「リソースのインスタンス」である。

リソースは物理的なもの（弾薬、金貨）に限らず、人気度・毒への耐性など抽象概念も数値化してリソースとして扱える。ゲームデザイナーの仕事の一部は、カリスマや好戦性といった「測定不能なもの」をプログラムが操作できる数値に変換することである。

純粋に装飾的なアイテム（プレイヤーが何もできず、何も起こらない花など）はリソースではない。

2.2 エンティティ (Entities)

エンティティとは、リソースの特定のインスタンス、またはゲーム世界の要素の状態を指す。

種類	説明	例
単純エンティティ	単一の値で完全に記述できる	スコア（数値型）、信号機の状態（記号型：赤/黄/緑）
複合エンティティ	複数の属性で構成される	風（速度＋方向）、アバター（名前・HP・種族・所持品等）
ユニークエンティティ	ゲーム内に1つしか存在しない	アバター本体、フットボールのボール

2.3 属性 (Attributes)

属性とは、あるエンティティに属し、それを記述するエンティティである。アバターの「HP」「種族」「所持品」はすべてアバターの属性であり、各属性自体が単純エンティティまたは複合エンティティとなりうる。

エンティティを設計する際は、初期値（数値型なら範囲と初期量、記号型なら取りうる状態一覧と初期状態）を定め、チューニング段階で調整する。

2.4 メカニクス (Mechanics)

メカニクスは、ゲーム世界とその中のすべての振る舞いを文書化したものであり、以下の4要素で構成される。

1. **関係 (Relationships)** : エンティティ間の依存関係。数式で表す場合もある
(例：キャラクターレベル = 経験値 / 1000)

2. **イベント (Events)** : 条件に発動され、1回実行される変化 (「金の卵を拾うと2点獲得」)
3. **プロセス (Processes)** : 開始後、停止されるまで継続する一連の活動
4. **条件 (Conditions)** : イベントの発生やプロセスの開始・停止を定義する (if～then、whenever～、until～)

ゲーム全体を通じて動作するメカニクスを**グローバルメカニクス**と呼ぶ。複数のゲームプレイモードを持つゲームでは、モード切替を管理するグローバルメカニクスが必須となる。

2.5 数值的関係と記号的関係

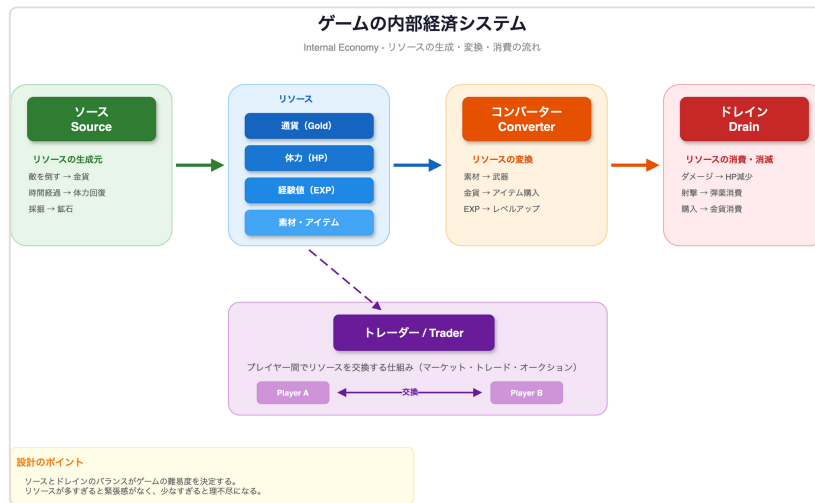
数值的関係は算術演算で定義される (例: 小麦粉1袋+水4バケツ → パン50斤)。数式的设计では、ゼロ除算の回避やドミノ効果の予測が重要である。

記号的関係は数学的操作ができない状態値 (赤/黄/緑など) 間の関係を定義する。各状態について、他のエンティティとの相互作用を個別に記述する必要がある。二値エンティティは**フラグ**と呼ばれる。

統合: 記号エンティティの状態変化を数値条件で定義できる (例: 「燃料が2ガロン未満になったら警告灯をON」)。逆に、記号を数値に変換するテーブル (ギア比テーブルなど) を作成し、計算に組み込むこともできる。

キーポイント - リソースは「型」、エンティティは「インスタンス」。この区別が設計の基盤 - エンティティは単純・複合・ユニークに分類される - メカニクスは関係・イベント・プロセス・条件の4要素で構成される - 数值的関係と記号的関係を適切に組み合わせることで多様なゲーム動作を実現する

3. 内部経済 (Internal Economy)



内部経済システム — ソース・リソース・コンバーター・ドレイン・トレーダーの資源フロー

内部経済とは、リソースとエンティティが数量的に生産・消費・交換されるシステムである。内部経済の複雑さはジャンルによって大きく異なる。

3.1 ソース (Source)

リソースがゲーム世界に新たに出現する仕組みを**ソース**と呼ぶ。

- ・シューティングゲームの**スポーンポイント**：敵・武器・弾薬・回復アイテムを生成する場所。位置、生成するリソースの種類、頻度を定義する
- ・自動生産：工場を建設するとリソースが自動的に生成される。生産レート of 定義が必要
- ・**限定ソース**と**無限ソース**：モノポリーの「Go」マスは無限の資金源。家・ホテルの在庫は限定ソース

3.2 ドレイン (Drain)

リソースがゲーム世界から永久に消滅する仕組みを**ドレイン**と呼ぶ。

- ・発砲による弾薬の消費、敵の死亡によるHP消滅
- ・建設管理シミュレーションの**劣化 (Decay)**：建造物が時間経過で損傷し、修理にリソースを要する

設計上の注意：リソースが消えるときには必ず理由をプレイヤーに説明すること。
プレイヤーは無料で得ることは気にしないが、失うときには理由を求める。

3.3 コンバーター (Converter)

1つ以上のリソースを別の種類のリソースに変換する仕組みを**コンバーター**と呼ぶ。生産レートと入出力比率の定義が必要。

- ・『The Settlers』の風車：穀物1袋 → 小麦粉1袋（20秒、毎分3袋）
- ・製鉄所：鉱石1 + 石炭1 → 鉄棒1、ただし木炭使用時は木炭3必要

3.4 トレーダー (Trader)

リソースの所有権を移転させる仕組みを**トレーダー**と呼ぶ。コンバーターと異なり、アイテム自体は変化しない。

暴利問題への対策：プレイヤーが安く買って高く売る行為を無制限に繰り返せると、ゲームが破綻する。対策として、再販価格を購入価格以下に制限する、売買間にクールダウンを設ける、トレーダーの資金を有限にする、などがある。

3.5 有形リソースと無形リソース

- ・ **有形 (Tangible)**：物理的に存在し、保管・輸送が必要（シューターの弾薬など）
- ・ **無形 (Intangible)**：空間を占めず、輸送不要（多くのゲームにおける金銭）
- ・ **混合型**：『Age of Empires』では輸送中は有形（敵に奪われる）、貯蔵後は無形（破壊されない）

3.6 フィードバックループとデッドロック

フィードバックループ：生産メカニズムが自身の生産物を入力として必要とする構造。

例：『The Settlers III』では石が石切り小屋の建設に必要で、石切り小屋が石を生産する。

デッドロック：フィードバックループの中でリソースが枯渇し、生産が完全停止する状態。モノポリーの「Go」で毎週200ドルもらえるのは、物件を買えない→家賃収入がない→物件を買えない、というデッドロックを防ぐためである。

設計原則：デッドロックを打破する手段を必ず提供せよ

3.7 静的均衡と動的均衡

- ・ **静的均衡**：生産量と消費量が常に一定で、リソースの量が変化しない状態
- ・ **動的均衡**：生産と消費がサイクルで変動するが、一巡すると同じ状態に戻る

均衡状態ではプレイヤーへの圧力が減るため、多くのゲームでは意図的に均衡を崩す要素を導入する（農場の老朽化、道路の劣化など）。成長を達成するにはプレイヤーの能動的な介入が必要となるよう設計することが重要である。

キーポイント - 内部経済はソース→コンバーター→トレーダー→ドレインのリソースフローで構成される - フィードバックループはデッドロックのリスクを伴う。必ず脱出手段を用意する - 均衡と不均衡のバランスがゲームプレイの緊張感を生む

4. コアメカニクスとゲームプレイの関係

4.1 受動的チャレンジと能動的チャレンジ

種類	定義	メカニクスの役割	例
受動的 (Passive)	静的な障害物	チャレンジ自体にメカニクス不要	壁を乗り越える
能動的 (Active)	動的な仕組みを持つ	エンティティとメカニクスの定義が必要	パズルを解いてドアを開ける、敵と戦闘

コアメカニクスは能動的チャレンジに必要なエンティティとメカニクスをレベルデザイナーに提供する「LEGOブロック」の役割を果たす。

4.2 アクションとコアメカニクス

プレイヤーのアクションはメカニクスをトリガーする。

例：しゃがみアクション 1. プレイヤーがしゃがみボタンを押す 2. UIがしゃがみメカニクスをトリガー 3. アバターの「姿勢」属性（記号型）が「直立」から「しゃがみ」に変化 4. アバターの「頭部の高さ」属性（数値型）が低下 5. グラフィックエンジンが視点を更新

アナログ入力（マウス移動量など）を伴うアクションでは、UIからデータを受け取り、属性値を適切に変更するメカニクスが必要となる。

キーポイント - 受動的チャレンジはメカニクス不要、能動的チャレンジはメカニクス必須 - プレイヤーアクションはUIを通じてメカニクスをトリガーし、エンティティの状態を変更する

5. コアメカニクスのデザイン手法

5.1 デザインの目標

シンプルさとエレガンス

最も優れたゲームは最小限のルールで興味深い多様性を生み出す。モノポリーのルール自体はシンプルだが、チャンスカードが「所得税還付：20ドル受け取る」などバリエーション豊かな文脈を添えることで面白さを増している。

パターンの発見と一般化

個別ケースを列挙するのではなく、共通パターンを見つけて一般化する。例えば「沼ヒルは水から出ると毎分10HP減少」「サラマンダーは火から出ると毎分5HP減少」という個別ルールは、「環境依存クリーチャーは固有環境から離れると一定レートでHP減少」という一般ルールに統合できる。

紙上での完璧を求めない

すべてを頭の中で計算するのは不可能。まずドラフトを書き、プロトタイプで検証し、反復的に改善する。明確で正確な記述の方が、正確だが曖昧な記述よりも重要である。

適切な詳細度

- ・ 詳細すぎる：デザイナーが作業量で潰れる
- ・ 不足すぎる：プログラマが推測で実装し、バグの原因になる
- ・ **適切な水準**：慣習的なシナリオは一般的な言語で記述し、独自のメカニクスほど詳細に記述する

5.2 デザインのステップ

ステップ1：既存の設計成果を見直す

以前のデザインドキュメントを読み返し、名詞と動詞を抽出する。 - 名詞 → エンティティまたはリソースの候補 - 動詞 → メカニクスの候補 - if / when / whenever → 条件の候補

特に以下を精査する： - 「プレイヤーは何をするのか？」への回答 - ゲーム構造のフローボード - 各ゲームプレイモードの計画 - ストーリーの概要と分岐条件 - レベルの進行計画と勝利・敗北条件

ステップ2：エンティティとリソースのリスト化

抽出した名詞を分類し、以下を決定する： - リソースか、独立エンティティか、他エンティティの属性か - 単純か複合か。数値型か記号型か - 初期値と取りうる範囲

ステップ3：メカニクスの追加

動詞のリストとエンティティの一覧を手にも、「somehow（何らかの方法で）」と書かれた部分を正確な仕様に置き換える。

- ・ リソースのフローを考える（ソース、ドレイン、コンバーター、トレーダー）
- ・ 各エンティティについて「何が影響するか」「何に影響するか」「自律行動はあるか」「プレイヤーは操作できるか」を問う
- ・ チャレンジとアクションの対応関係を定義する
- ・ グローバルメカニクスを特定する

キーポイント - パターンを見つけて一般化し、少ないルールで多くの状況をカバーする - 反復的改善を前提とし、プロトタイプで検証する - 名詞→エンティティ、動詞→メカニクス、条件語→条件として設計を進める

6. 乱数とガウス曲線

6.1 乱数の基本

コンピュータが生成する乱数は通常0以上1未満の実数値である。確率0.1のイベントは、生成した乱数が0.1未満のときに発生する（10%の確率）。命中精度0.8の武器は、乱数 < 0.8 なら命中する。

6.2 疑似乱数 (Pseudo-Random Numbers)

乱数生成アルゴリズムは**シード値**によって数列が決まる。同じシードなら毎回同じ乱数列が得られるため、チューニングやバグ修正に極めて有用。製品版出荷時にはシステムクロックなどからシードを取得し、毎回異なる体験を提供する。

6.3 モンテカルロ・シミュレーション

複雑なメカニクスの挙動を検証するために、ランダムな入力で大量の試行（例：1000回）を行い、結果を分析する手法。弱いチームが強いチームに勝つ頻度が妥当かどうか、入力と結果の相関が正しいかどうかを確認できる。Excelなどのスプレッドシートに組み込みツールが用意されている。

6.4 一様分布と非一様分布

一様分布 (Uniform Distribution)：すべての値が等確率で出現する。1個のサイコロを振る場合と同じ。選択肢のランダムな並べ替えなどに使う。

非一様分布：複数のサイコロの合計値のように、中央値付近が出やすく極端な値が出にくい分布。オリンピックの弓射手のように「的の中心に当たりやすい」シミュレーションに向く。

6.5 ガウス曲線（正規分布）

複数のサイコロの合計値は**ベル型（ガウス）曲線**を描く。

- ・ 6面サイコロ2個：7が出る確率（ $6/36 = 16.7\%$ ）は2が出る確率（ $1/36 = 2.8\%$ ）の6倍
- ・ 6面サイコロ3個：10や11が出る確率（ $27/216$ ）は3や18が出る確率（ $1/216$ ）の27倍

これは「3面から18面の16面サイコロを1個振る」のとは全く異なる。16面サイコロでは18が出る確率は6.25%だが、3個の6面サイコロの合計で18が出る確率は0.46%に過ぎない。

ガウス分布は水滴の落下パターンや知能の分布など自然界の多くの現象を記述する。「大多数は中央に集まり、極端な値はまれ」という効果が必要なときに使う。

キーポイント - 疑似乱数は同じシードで再現可能。チューニングとデバッグに不可欠 - 一様分布は等確率の選択に、ガウス分布は「中央に集まる」効果に使う - モンテカルロ・シミュレーションで複雑なメカニクスの妥当性を統計的に検証できる

授業のまとめ

概念	要点
コアメカニクス	ゲームのルールと内部動作をデータ＋アルゴリズムで正確に定義したもの
リソース	数量として扱える物体・素材の「型」
エンティティ	リソースのインスタンスまたはゲーム世界の状態。単純/複合/ユニークに分類
属性	エンティティに属し、それを記述するエンティティ
メカニクス	

概念	要点
	関係・イベント・プロセス・条件の4要素で構成
内部経済	ソース・ドレイン・コンバーター・トレーダーによるリソースフロー
デザイン手法	名詞→エンティティ、動詞→メカニクスの抽出。パターンの一般化と反復的改善
乱数	一様分布とガウス分布を目的に応じて使い分ける

本資料は Ernest Adams 著『Fundamentals of Game Design』Chapter 10 に基づく講義用テキストです。

ewpage

第9回：ゲームバランスング

学習目標

本講義では、ゲームデザインにおける「バランス」の概念を体系的に理解する。具体的には以下の項目を習得する。

- ・ バランスの取れたゲームが備えるべき特性を説明できる
- ・ 支配的戦略（ドミナントストラテジー）の問題と回避方法を理解する
- ・ 運（チャンス）の要素を適切に組み込む手法を学ぶ
- ・ PvP・PvEそれぞれにおける公平性の考え方を区別できる
- ・ フロー理論に基づく難易度管理と動的難易度調整（DDA）を理解する
- ・ ポジティブフィードバックの仕組みと制御方法を把握する
- ・ チューニングしやすいゲーム設計の原則を理解する

1. バランスの取れたゲームとは

ゲームバランスとは、ゲームが適度な難しさで、プレイヤーにとって公平であり、プレイヤーの技量が勝敗を左右する最大の要因となっている状態を指す。バランス調整は単一の作業ではなく、設計とチューニングの複合的なプロセスである。

バランスの考え方は、**PvP（プレイヤー対プレイヤー）** ゲームと **PvE（プレイヤー対環境）** ゲームで大きく異なる。

1.1 すべてのゲームに共通する特性

- ・ **意味のある選択肢を提供すること**：どの戦略も妥当な勝利の可能性をもち、ある1つの戦略だけが圧倒的に有利になっていない
- ・ **運が技量が無意味にしないこと**：長期的には、優れたプレイヤーが劣るプレイヤーより成功する

1.2 PvPゲームに求められる特性

- ・プレイヤーがゲームを**公平**だと感じる
- ・序盤に遅れをとったプレイヤーにも**巻き返しの合理的な機会**がある
- ・**膠着（ステイルメイト）が発生しにくい**（特に実力差がある場合）

1.3 PvEゲームに求められる特性

- ・プレイヤーがゲームを**公平**だと感じる
- ・難易度が**一貫しており、急激な変動がない**

キーポイント - ゲームバランスはPvPとPvEで要件が異なる - 共通して「意味のある選択」と「技量の重視」が基盤である - 公平性の意味はゲームの種類によって異なる

2. 支配的戦略（ドミナントストラテジー）の回避

2.1 支配的戦略とは

支配的戦略とは、相手が何をしようと常に最善の結果を生む戦略のことである（ゲーム理論の用語）。支配的戦略が存在すると、プレイヤーは他の選択肢を試す理由がなくなり、ゲームの楽しさが損なわれる。

実例：『Command & Conquer: Red Alert』の「タンクラッシュ」は、ソ連側プレイヤーが序盤に大量の戦車を生産して敵基地を急襲するもので、あまりに強力なため、多くのプレイヤーが「タンクラッシュ禁止」のハウスルールを追加した。

2.2 推移的關係とコストによるバランス

プレイヤーの選択肢に**推移的關係**（AがBより強く、BがCより強ければ、AがCより強い）が存在すると、最強の選択肢が支配的戦略になる。

これを是正するには、強い選択肢に高い**直接コスト**を課す方法がある。例えば、レースゲームで最速の車に最高の価格を設定すれば、速度の優位性が費用面のハンデで相殺される。

さらに巧みな方法として**シャドウコスト**（隠れたコスト）がある。例えば、最速の車の燃料タンクを小さくしてピットストップを増やす、高威力の武器の発射速度を遅くするなど、プレイを通じて初めて気づくような二次的な不利益を設定する。

ただし、完全に隠されたシャドウコストはPvEゲームでは不公平と感じられることがあるため注意が必要である。

2.3 非推移的關係（じゃんけんモデル）

非推移的關係とは、AがBに勝ち、BがCに勝ち、CがAに勝つというループ構造である。これは「じゃんけん（Rock-Paper-Scissors: RPS）」として知られ、支配的戦略を防ぐ古典的手法である。

実例： - 『Virtua Fighter 3』：攻撃が投げに勝ち、投げが防御に勝ち、防御が攻撃に勝つ
- 『The Ancient Art of War』：騎士が蛮族に強く、蛮族が弓兵に強く、弓兵が騎士に強い

単純なRPSモデルだけでは現代のゲームには不十分だが、各選択肢に異なる報酬や状況依存の優劣を加えることで、より深い戦略性を生み出せる。

2.4 直交的ユニット差別化

Harvey Smithが提唱した**直交的ユニット差別化（Orthogonal Unit Differentiation）**とは、各ユニットが単に同次元（速度や火力など）で強弱が異なるのではなく、**まったく異なる次元の能力**を持つべきだという考え方である。

例えば、近接武器を持つユニットと治療能力を持つユニットは直接比較できない。この多様性がプレイヤーの選択をより面白くし、さまざまなユニットを組み合わせる必要性を生んで支配的戦略を防ぐ。チェスの駒がその好例で、クイーンは最強だが単独では盤面を支配できず、各駒の協力が必要である。

2.5 PvEゲームにおける支配的戦略

PvEゲームでは、少数のアクションが多くの課題を克服できるよう設計されることが多いが、これは意図しない**エクスプロイト**（設計上の弱点を突く無敵の行動）を生むリスクがある。プレイテストで可能な限り多くのアクションの組み合わせを試すことが唯一の対処法である。

キーポイント - 支配的戦略はゲームの選択肢を無意味にする最大の問題である - 直接コストやシャドウコストで推移的關係をバランスする - じゃんけんモデル（非推移的關係）は支配的戦略防止の基本手法 - 直交的な能力差別化がユニットの多様性と戦略の深さを生む

3. 運の要素の組み込み

ゲームにおける運（チャンス）の役割は、チェッカーのように運がゼロのものからクラップスのように運がすべてのものまでさまざまである。技量が勝敗の主因であるべきだが、運を取り入れる場合は以下のガイドラインに従う。

原則	説明
運を控えめに使う	勝利につながるアクションの大半が技量に依存するよう設計する
小さなリスク・報酬で頻繁に使う	ポーカーのように、1回の勝負ではなく多数の勝負の積み重ねで実力差が反映されるようにする
確率を利用した意思決定を許す	プレイヤーが経験を通じて確率を学び、有利な判断ができるようにする
リスク量の選択を許す	プレイヤーが各挑戦でどれだけ賭けるかを自分で決められるようにする

要約すると、**プレイヤーが明示的に大きなリスクを選ばない限り、運によって大きな問題が決まるべきではない。**

キーポイント - 運は技量を補完するものであり、置き換えるものではない - 小さなリスクの頻繁な繰り返しが、長期的に技量を反映させる仕組みになる - プレイヤーにリスクの選択権を与えることが公平感につながる

4. PvPゲームの公平性

4.1 対称型ゲームによるバランス

対称型ゲームでは、すべてのプレイヤーが同じリソース・同じ選択肢・同じ勝利条件でスタートする。チェス、バックギャモン、モノポリーなどがこの例にあたる。一方のプレイヤーにしたことは他のプレイヤーにも同様にすればよいため、公平性の確保は容易である。

ただし、ビデオゲームのプレイヤーは対称型ゲームを単調と感じる傾向がある。

4.2 非対称型ゲームのバランス

非対称型ゲームは、プレイヤーごとに異なるルール・ユニット・勝利条件を持つ。このため支配的戦略が生まれやすく、バランス調整がはるかに困難になる。

ポイント配分システムは非対称ゲームをバランスする有効な手法である。各プレイヤーに同数のポイントを与え、速度・防御力・攻撃力などに自由に配分させる。ただし、配分先の属性は**直交的**（互いに独立した効果を持つ）であること、また各属性への1ポイントの投資効果が均等であることが重要である。

『StarCraft』の好例：3種族（テラン、プロトス、ザーグ）はそれぞれまったく異なる能力を持つが、同等の戦闘力のユニット生産にかかるコストはほぼ等しい。直接コストとシャドウコストの両方が巧みに組み合わせられ、10年以上にわたりトーナメントで愛される卓越したバランスを実現した。

4.3 永続型ワールドのバランス問題

MMOなどの永続型ワールドは、長期プレイヤーが蓄積するリソースと経験により**本質的に非対称**である。新規プレイヤーの保護（PvP回避の選択肢、新規向けコンテンツの提供）が不可欠であり、新しいリソースが常に流入するゼロサムでない経済が求められる。

4.4 不正なAI（チーティングAI）

シングルプレイヤーPvPで、AIが人間に勝てない場合にAIに有利なルール（霧の先を見通す、ユニットを強化する等）を密かに適用することがある。プレイヤーは発覚時に強い不満を感じるため、**最終手段としてのみ使用し、できれば難易度設定として公開する**のが望ましい。

キーポイント - 対称型は公平だが単調になりがち、非対称型は面白いが調整が難しい - ポイント配分システムは非対称バランスの基本手法 - 永続型ワールドは新規プレイヤー保護が最重要課題 - AIの不正は最終手段であり、透明性が重要

5. PvEゲームの公平性

PvEゲームでは対戦相手がいないため、公平性はゲーム環境そのものの設計に関わる。プレイヤーが公平と感じるための条件は以下の通りである。

1. **難易度に突然のスパイクがないこと**
2. **警告なしに突然ゲームオーバーにならないこと**（「死んで覚える」設計は現在では不公平とみなされる）
3. **膠着状態（ステイルメイト）が発生しないこと**（重要アイテムの取り逃しで進行不能になるなど）
4. **十分な情報なしに重大な決断を強いないこと**
5. **勝利に必要なすべての知識がゲーム内にあること**（トリビアゲームを除く）
6. **ジャンルに通常含まれない種類の課題を課さないこと**

キーポイント - PvEの公平性は「環境の誠実さ」に帰着する - プレイヤーに必要な情報と回避手段を常に提供すべきである - 進行不能状態の防止はメカニクス設計段階から検討する

6. 難易度管理

6.1 フロー理論

心理学者チクセントミハイは、人が課題に取り組む際、自身の能力と課題の難しさが釣り合ったときに最も生産的で楽しい「**フロー状態**」に入ることを見出した。課題が難しすぎれば**不安**、易しすぎれば**退屈**を感じる。

ゲームバランスの目的は、プレイヤーを可能な限りこのフロー状態に保つことである。

6.2 難易度の4つの要素

知覚難易度（プレイヤーが実際に感じる難しさ）は、以下の要素で決まる。

要素	デザイナーの制御
固有スキル要件（ISR）	制御可能
ストレス（時間的圧迫）	制御可能
提供パワー （プレイヤーに与える力）	制御可能
ゲーム内経験 （プレイ中に得た慣れ）	制御可能
過去の経験・先天的才能	制御不能

これらの関係は以下の式で表される。

$$\text{知覚難易度} = \text{絶対難易度} - (\text{提供パワー} + \text{ゲーム内経験})$$

- ・ **絶対難易度**：固有スキル要件とストレスから決まる、基準との比較による難しさ
- ・ **相対難易度**：絶対難易度から提供パワーを引いたもの（レベル1の騎士とレベル5の騎士では同じ敵の相対難易度が異なる）

6.3 難易度の進行設計

バランスの取れた難易度進行を作るためのガイドライン：

1. 時間とともに**絶対難易度を上げる**
2. プレイヤーへの**提供パワーもやや低い率で上げる**
3. ゲーム内経験の蓄積速度より**相対難易度の上昇をやや速くする**
4. プレイテストで**難易度のスパイクや急落を探して修正**する
5. 各レベルの冒頭は**前レベルの終盤より低い難易度から始め**、レベル内で上昇させる（ノコギリ歯パターン）

このノコギリ歯型は、プレイヤーに達成感を与つつも各レベルの最初に息をつかせる良いペーシングを生む。

6.4 難易度モード

シングルプレイヤーゲームでは、イージー・ノーマル・ハードなどの**難易度モード**を提供すべきである。これにより、デザイナーが制御できない「過去の経験」と「先天的才能」の差をプレイヤー自身が補正できる。

設計ルール：「イージーモード」と名付けたなら、本当に初心者にとって簡単でなければならない。

6.5 動的難易度調整 (DDA)

動的難易度調整 (Dynamic Difficulty Adjustment) は、プレイヤーのパフォーマンスをリアルタイムで評価し、自動的にゲームの難しさを変化させるシステムである。

ゲーム	DDA手法
Max Payne	敵の強さとエイム補助をパフォーマンスに応じて透明に調整
Half-Life 2	アバターの体力・弾薬に応じてクレートの中身を変更
Burnout 2	AI車の性能を自動調整してプレイヤーとの接近レースを維持
Crash Bandicoot	同じセクションで何度も失敗するとシールドを追加提供
Madden NFL 09	トレーニングセンターでのテスト結果に基づき難易度を設定
God of War	頻繁に死亡すると低難易度モードへの変更を提案

DDAを設計する際の原則： - 通常の難易度モードの**代替ではなく補助**として使う - **任意（オプション）**にする - ゲームを**難しくする方向に使う方が簡単**（易しくするには失敗原因の特定が必要） - プレイヤーが稼いだものを**奪わない** - **気づかれないほど微妙**に行うのが最善（Max Payneは好例、God of Warは悪例）

キーポイント - フロー理論がゲーム難易度管理の理論的基盤 - 知覚難易度 = 絶対難易度 - （提供パワー + ゲーム内経験） - ノコギリ歯パターンで各レベルの難易度をペーシングする - DDAは強力だが微妙な実装と任意性が求められる

7. ポジティブフィードバックの理解

7.1 ポジティブフィードバックとは

ポジティブフィードバックとは、プレイヤーの成功がゲーム状態を変化させ、次の成功をより容易にする循環構造のことである。これはプレイヤーの経験蓄積とは別の、コアメカニクス上の現象である。

達成 → 報酬を得る → **パワー**に変換 → 次の**達成**が容易に → …

例：モノポリーで独占（モノポリー）を達成すると高い家賃を徴収でき、その資金ですらに物件を購入できる。チェスでは相手の駒を取ると相対的なパワーが増す。

7.2 ポジティブフィードバックの利点

- ・ **膠着を防ぐ**：リードしたプレイヤーが止められなくなることで、ゲームが確実に終結に向かう
- ・ **成功に有意義な報酬を与える**：スコアという装飾的な報酬ではなく、ゲームプレイに実質的に影響する力を得られる

7.3 ポジティブフィードバックの制御

ポジティブフィードバックが強すぎると、ゲームが早く終わりすぎたり、遅れたプレイヤーが巻き返せなくなったりする。以下の6つの方法で制御する。

1. **成功への報酬としてのパワーを抑える**：将棋では奪った駒を弱い形で再利用させる、戦争ゲームでは工場を破壊はできても奪取はできないようにする
2. **ネガティブフィードバックを導入する**：達成にコストを伴わせる（例：『Dungeon Keeper』で敵クリーチャーを味方にすると食料・金・住居が必要になる）
3. **課題の絶対難易度を段階的に上げる**：RPGで敵の強さをプレイヤーの成長に合わせて上昇させる
4. **リーダーへの共謀を許可する**：3人以上のゲームで他プレイヤーが協力してリーダーを抑える仕組み（例：『Diplomacy』）
5. **フィードバックサイクルと無関係な勝利条件を設定する**：チェスの勝利条件は「駒を最も多く取る」ではなく「チェックメイト」
6. **運の効果で報酬量を変動させる**：RPGで敵からの戦利品をランダムに変動させ、ときに少ない報酬にする

7.4 フィードバックの6パターン（Figure 11.7の解説）

パターン	特徴
1. 短距離走	フィードバックなし。速い方が勝つ
2. 不均衡なルール	Bが圧倒的に有利で即座に勝利。バランス不良の例
3. 膠着	どちらもリードを維持できず勝敗がつかない
4. フィードバック過剰	ルールは均衡だが、わずかなリードが急速に拡大して唐突に決着
5. 激しい逆転の繰り返し	強力なネガティブフィードバック。リーダーが不利にな

パターン	特徴
	り乱高下する（例：マリオカート）
6. 理想的な進行	リードが入れ替わりつつ、最終的に実力が上のプレイヤーが勝つ

パターン6が理想であり、プレイヤー双方に勝利の可能性を感じさせつつ、最終的に技量が反映される。

キーポイント - ポジティブフィードバックは膠着防止と達成感の源泉 - 制御されないフィードバックはゲームを早期に決着させる - 6つの制御手法を状況に応じて組み合わせる - 理想的な進行は「逆転可能性のある技量反映型」

8. バランスチューニングの容易化

8.1 停滞（スタグネーション）の回避

停滞とは、プレイヤーが次に何をすべきかわからず行き詰まる状態である（進行不能のステイルメイトとは異なる）。情報不足が主な原因であり、以下で対処する。

- ・ **受動的対処**：さりげないヒントを目に見える場所に配置する
- ・ **能動的対処**：プレイヤーが目的なく徘徊していることを検知し、誘導する

プレイヤーが攻略サイトやチートに頼らなければ進めない状態は設計上の欠陥である。

8.2 些末事（トリビアリティ）の回避

プレイヤーが大局的な判断に集中すべきときに、些末な管理作業（金の保管場所を決めるなど）を強いるのは避けるべきである。実質的な影響のない選択や、1つしか合理的選択がない場面はコンピュータに任せる。

好例：『Sid Meier's Alpha Centauri』は、惑星全体の統治から個別ユニットの管理まで、どのレベルを自分で操作するかをプレイヤーが選べる。

8.3 メカニクス的一般化

コアメカニクスをできるだけ**一般化**（汎用化）して設計する。例えば、戦闘メカニクスをユニットの種類に依存せず共通のルールで処理すれば、各ユニットのパラメータを数値テーブルの調整だけでチューニングでき、個別のコードを書き直す必要がない。

一般化されたメカニクスの利点： - **プログラミングの簡素化**：コア処理の実装に集中できる - **チューニングの効率化**：パラメータの変更だけでバランスを調整可能 - **一貫性の確保**：全エンティティが同じルールに従うためバランス問題が局所化しにくい

キーポイント - 停滞はヒントと検知システムで防ぐ - 些末な決定はコンピュータに委任する - メカニクス的一般化がチューニング工程を大幅に効率化する

本講のまとめ

テーマ	核心
バランスの定義	公平・適度な難しさ・技量重視
支配的戦略	コスト設定、RPS、直交的差別化で防止
運の組み込み	小リスク・高頻度・選択権の付与
PvP公平性	対称設計 or 慎重な非対称バランス
PvE公平性	情報提供・突然の不利の排除
難易度管理	フロー理論・ノコギリ歯進行・DDA
ポジティブフィードバック	膠着防止と達成感。6手法で制御
チューニング容易化	停滞・些末事の排除、メカニクス的一般化

参考文献 - Csikszentmihalyi, M. (1991). Flow: The Psychology of Optimal Experience. - Sirin, D. (2000). "Rock, Paper and Scissors in Strategy Games." - Smith, H. (2003). "Orthogonal Unit Differentiation." GDC Lecture. - Adams, E. (2008). "Difficulty Modes and Dynamic Difficulty Adjustment." Gamasutra. - 本講義の内容は Ernest Adams『Fundamentals of Game Design』Chapter 11 に基づく。

ewpage

第10回：レベルデザインの原則

はじめに

レベルデザイナーは、ゲームが展開する空間だけでなく、プレイヤーの瞬間ごとの体験や感情的な文脈の多くを創り出す。本章では、レベルデザインの一般原則、空間レイアウトの類型、設計プロセス、そして陥りやすい落とし穴について学ぶ。

1. レベルデザインとは何か

レベルデザインとは、ゲームデザイナーが用意したコンポーネントを使い、プレイヤーに直接提供される体験を構築するプロセスである。ゲームデザイナーとレベルデザイナーは別の役割であり、大規模な開発チームでは通常、異なる人物が担当する。

レベルデザイナーが作り出す要素は、**ゲーム空間**（何がどこにあるかの具体化）、**初期条件**（敵の数やリソース配置）、**チャレンジ群**（挑戦の内容と順序）、**終了条件**（勝敗の基準）、**ストーリーとの連携**、**美的演出・雰囲気**の6つに大別される。

キーポイント - レベルデザインは「体験の構築」であり、空間設計だけでなくチャレンジ配置・物語展開・雰囲気づくりを含む - ゲームデザイナーが「何を」決め、レベルデザイナーが「どこに・どのように」を決める

2. 主要デザイン原則

2.1 普遍的なレベルデザイン原則

以下の原則はジャンルを問わず、あらゆるゲームのレベルデザインに適用される。

1. **序盤をチュートリアルにする** — 操作や仕組みを実践で学ばせる

2. **ペーシングを変化させる** — 緊張と緩和を交互に配置する
3. **リソース消費後に補充を用意する** — 困難なチャレンジの直後に回復手段を置く
4. **概念的な矛盾を避ける** — 世界観に合わない不合理な配置をしない
5. **短期目標を常に明確にする** — 「次に何をすべきか」を常にわかるようにする
6. **リスク・報酬・結果を明示する** — 「死んで覚える」設計は悪い慣行である
7. **技術・知性・献身を報酬で称える** — 多様な形で良いプレイに報いる
8. **報酬は大きく、罰は小さく** — 過度な罰はゲーム離脱を招く
9. **前景を背景より優先する** — 背景を複雑にしすぎない
10. **AI敵は善戦して負けるために存在する** — クリア不能なレベルは悪い設計
11. **複数の難易度設定を用意する** — 幅広いプレイヤー層に対応する

2.2 ジャンル固有の原則

各ジャンルには特に重視すべき原則がある。アクションゲームでは**ペースの緩急**、ストラテジーでは**計画への報酬**、RPGでは**キャラクター成長と自己表現**、スポーツでは**現実らしさの徹底**、アドベンチャーでは**チャレンジとストーリーの調和**、パズルでは**考える時間の確保**が、それぞれ最重要原則となる。

キーポイント - 普遍的原則の核心は「明確な目標」「適切なペーシング」「報酬と罰のバランス」「リソース補充」 - ジャンルごとに重点は異なるが、「プレイヤー中心設計」は共通

3. レイアウトの種類

移動を伴うゲームでは、空間のレイアウトがプレイ体験に大きな影響を与える。代表的なレイアウトパターンは以下の通りである。

3.1 オープンレイアウト

プレイヤーの移動がほぼ制約されない、屋外のような自由空間。『Battlefield 1942』のような戦争ゲームで多用される。RPGでもフィールド探索時に使われるが、屋内やダンジョンではネットワーク型に切り替わることが多い。

3.2 リニア（線形）レイアウト

プレイヤーが決められた順序で空間を体験する構造。分岐がなく、次のエリアか前のエリアにしか移動できない。「レール上」とも呼ばれ、横スクロールアクションやレールシューターに伝統的。線形ストーリーとの相性がよい。ただし一方通行のドアを設置する場合、後戻りできないエリアに必須アイテムを置かないよう注意が必要である。

3.3 パラレル（並列）レイアウト

線形レイアウトの発展形。プレイヤーはレベルの始点から終点へ進むが、途中で複数の並行経路を選択できる。ある経路はリスクが高い代わりに報酬も大きく、別の経路はストーリー情報が豊富、といった差別化が可能。隠しショートカットを用意する場合は、必ず何らかの手がかりを設けること（手がかりなしの隠し通路は試行錯誤を強いる悪い設計）。

3.4 リング（環状）レイアウト

経路が出発点に戻る構造。レースゲームで主に使われ、プレイヤーは同じ空間を周回する。ショートカットは通常ルートより短いが比例して難しくする必要があり、このバランス調整がレベルデザイナーの重要な仕事となる。

3.5 ネットワークレイアウト

空間同士が多様な方法で接続される構造。探索自体が大きなゲームプレイとなる。全空間が相互接続されていればデスマッチに適するが、順序立てたストーリーテリングには不向き。

3.6 ハブ&スポークレイアウト

中央のハブ（安全な拠点）から複数のスポーク（線形経路）が伸びる構造。プレイヤーはスポークの先端でチャレンジと報酬を得て、ハブに帰還する。スポークの開放を段階的にすれば難易度を制御できる。『Spyro the Dragon』シリーズが代表例。

3.7 レイアウトの組み合わせ

多くのゲームでは複数のレイアウトを組み合わせる。典型的なのは、大きな線形フレームワークの中にネットワーク型の探索空間を配置する構造であり、大型RPGやアドベンチャーゲームに多い。

キーポイント - 6つの基本レイアウト（オープン・リニア・パラレル・リング・ネットワーク・ハブ&スポーク）を目的に応じて選択・組み合わせる - レイアウト選択はストーリー構造、チャレンジの提示方法、プレイヤーの自由度に直結する

4. 雰囲気・進行・ペーシング

4.1 雰囲気（アトモスフィア）の構築

レベルデザイナーは以下の要素を組み合わせ、レベルの美的一貫性とムードを作り出す。

- ・ **ライティング・カラーパレット** — 光の配置・色彩で安全感や危険感を演出する
- ・ **天候・大気効果** — 霧、雨、雪、風が心理的印象を生む
- ・ **視覚・音響エフェクト** — 武器の反動、魔法の火花、タイヤのスキール音などで感情を喚起する
- ・ **音楽・環境音** — リズムがペースを、環境音が場所と時間の感覚を支える

4.2 進行（プログレッション）の設計

ゲーム全体を通してレベル間で以下の要素が段階的に変化すべきである。

1. **メカニクスの豊富さ** — 序盤は単純に、後半は複雑に
2. **プレイ時間** — 後半のレベルほど長くなる傾向
3. **付随的報酬と環境変化** — カットシーン、トロフィー、アンロックコンテンツ、風景の変化
4. **実用的報酬** — 新しい乗り物、装備、スキル、技術
5. **難易度** — 原則として徐々に上昇する
6. **プレイヤーアクション** — 新しいアクションを段階的に追加する
7. **ストーリー展開** — プレイの進行とともに物語も前進する
8. **キャラクター成長** — 能力面だけでなく、文学的な意味での人間的成熟も含む

4.3 ペーシングの設計

ペーシングとは、プレイヤーがチャレンジに遭遇する頻度のことである。

緩急の交替が基本原則 — ゲームのペースも交響曲やアクション映画と同様、速い区間と遅い区間を交互に配置する。ストレスの高いチャレンジの後にはリソース補充や探索の時間を挟む。『指輪物語』では冒険団が危機を乗り越えるたびに休息地を見つけるが、これはゲームのペーシング設計と同じ構造である。

全体のペース — レベル全体としてはペースが一定か、終盤に向けて加速すべきである。レベル末のボス戦は伝統的な手法であり、勝利と報酬をもたらす。線形レイアウトほどペーシング制御がしやすく、自由度の高いジャンルでは制御が難しくなる。

4.4 チュートリアルレベル

現代のゲームでは、チュートリアルレベルで操作方法を実践的に教える。設計の要点は、機能を順序立てて導入すること、未紹介機能は無効化しておくこと、UI要素を視覚的にハイライトすること、失敗ペナルティをなくすこと、そして**チュートリアルを任意参加にする**ことである。

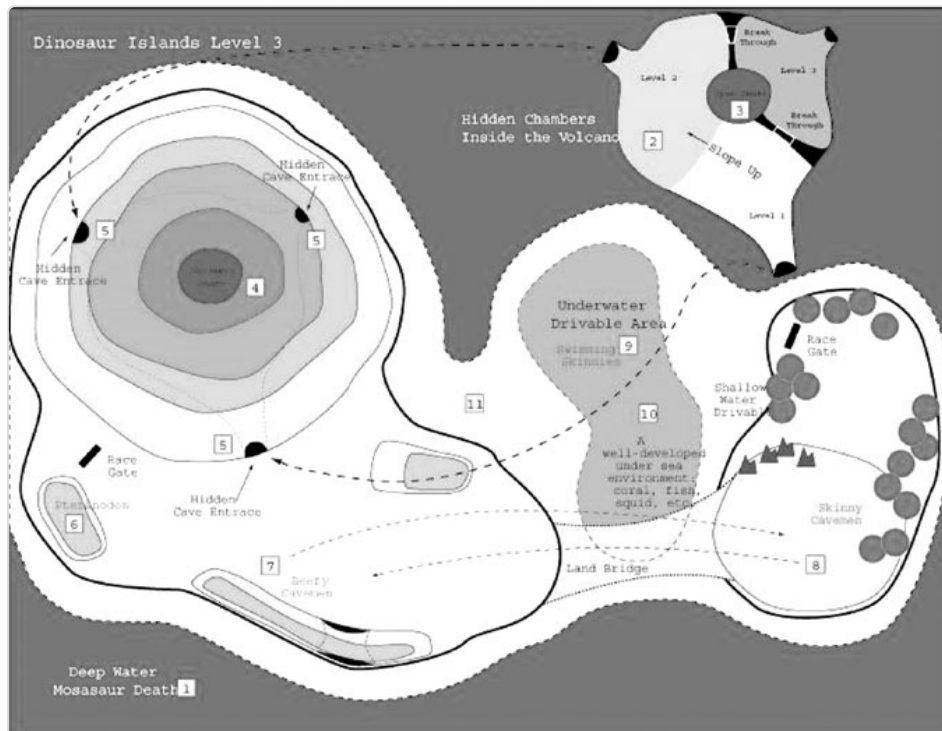
キーポイント - 雰囲気づくりではライティング・色彩・音響・天候を総合的に設計する - 進行は8つの要素（メカニクス、時間、報酬、難易度、アクション、ストーリー、キャラクター成長等）で段階的变化を設計する - ペーシングの核心は「緊張と緩和の交替」であり、リソース補充のタイミングと密接に関連する

5. レベルデザインプロセス

レベルデザインは反復的なプロセスであり、以下の11段階で進行する（Knowles & Ganetakos, 2004に基づく）。

ステップ1：ゲームデザインからの引き継ぎ

ゲームデザイナーからレベルの設定・雰囲気・主要ゲームプレイ・イベントの概要を受け取り、以下を作成する。 - トリガーイベント、プロップ（配置物）、NPCのリスト - レベルの概略マップ



図：Pseudo Interactive社による未発売ドライビングゲームのレベルスケッチ。主要な地形・プロップ・NPCの配置を示している。

ステップ2：計画フェーズ

鉛筆と紙でイベントシーケンスを詳細化し、以下の4領域を文書化する。

- ・ **ゲームプレイ** — レイアウト、チャレンジエリア、ペーシング、終了条件、リソース配置、NPC配置、隠しエリア、ランドマーク、セーブポイント等
- ・ **アート** — スケール決定、プロップ数・テクスチャ・特殊視覚効果のリストアップ（スコープ管理が重要）
- ・ **パフォーマンス** — ジオメトリの複雑さ、描画距離、同時処理ユニット数の制限をプログラマーと協議
- ・ **コード要件** — レベル固有の特殊イベント、独自AI、専用ツール等の事前特定

ステップ3～5：プロトタイピング → レビュー → ロックダウン

3Dモデリングツールで仮モデル（基本ジオメトリ、仮テクスチャ、仮プロップ、AI経路、トリガーポイント）を構築し、各チームのフィードバックを受ける。スケール・ペーシング・パフォーマンス等を確認し、合意を得た時点でレベルデザインをロック（重大な問題以外の変更禁止）する。

ステップ6～8：アートチームとの往復

ロックしたプロトタイプをアートチームに引き渡し、本番アートワークを制作してもらう。完成後に受け取ってレビューし、問題を修正する。

ステップ9～11：統合 → テスト → チューニング

アート・コード・オーディオを統合して完成レベルを組み立て、バグ修正を行う。その後QA部門がアルファテスト、さらにベータテストを実施する。

キーポイント - レベルデザインは「計画 → プロトタイプ → レビュー → ロックダウン → アート制作 → 統合 → テスト」の反復プロセス - 計画段階の文書化とロックダウンが、開発効率とコスト管理の鍵

6. レベルデザインの落とし穴

6.1 スコープを正しく設定する

初心者レベルデザイナーが最も犯しやすいミスは、大きすぎるレベルを作ろうとすることである。計画段階でプロップ・NPC・特殊イベントをリスト化し、チームの人員・予算・期間に見合ったスコープに収める必要がある。休暇や病欠も見込んでおくこと。小さすぎるレベルは拡張しにくい、大きすぎて未完成よりは遥かにましである。

6.2 概念的な矛盾（非論理的配置）を避ける

リアリスティックなゲームで不合理な配置をすると「頭を使うプレイヤーを罰する」ことになる。例えば『James Bond: Tomorrow Never Dies』では、潜入場面で油ドラム缶を爆破すると医療キットが出現するが、これは現実世界の前提に反し、合理的に考えるプレイヤーほど不利になる悪い設計である。

6.3 非典型的なレベルは任意参加にする

通常と大きく異なるレベルを作ること自体は構わないが、クリア必須にすると、その種のチャレンジが苦手なプレイヤーがゲームを完遂できなくなる。隠しレベルやサイドミッションとして提供すべきである。

6.4 すべてを一度に見せない

最初のレベルですべてのチャレンジ・環境・アクションを見せてしまうと、残りのゲームに新鮮味がなくなる。レベルごとに新要素を段階的に導入し、「もっと見たい」とプレイヤーに思わせ続けることが重要である。

キーポイント - スコープ管理は最優先事項：チームの能力に見合ったサイズで設計する - 世界観に矛盾する配置は新規プレイヤーを混乱させ、難易度を不当に上げる - 非典型レベルは面白い試みだが、必須にしない - コンテンツの出し惜しみ（段階的開示）がプレイヤーのモチベーション維持に効果的

まとめ

本章では、レベルデザインの全体像として、(1)定義と構成要素、(2)11の普遍的原則とジャンル固有原則、(3)6種のレイアウト類型、(4)雰囲気・進行・ペーシングの設計手法、(5)11段階の制作プロセス、(6)スコープ管理や概念的矛盾といった落とし穴を学んだ。レベルデザインは、ゲームデザインのビジョンをプレイヤーの具体的体験へ変換する、技術・芸術・心理学を統合する分野である。

参考文献：Ernest Adams 『Fundamentals of Game Design』 Chapter 12

ewpage

第11回：アクションゲームとストラテジーゲーム



ゲームジャンル総覧 — Part2で扱う全10ジャンルのマップ

1. アクションゲーム

1.1 アクションゲームとは

定義: アクションゲームとは、プレイヤーに提示される課題の大部分が身体的スキルと反射神経のテストであるゲームのことである。パズル要素や戦術的な戦闘、探索の要素を含むこともある。

アクションゲームは優れた手と目の協調性を必要とし、素早い反応が求められる。最も高速なアクションゲームは「ツイッチゲーム (twitch game)」と呼ばれ、ほぼ反射レベルでの操作が要求される。こうしたゲームでは戦略や計画を立てる余裕がなく、ストレスレベルは高いが、個々の課題に必要な固有スキルは低くなる傾向がある。

一方で、すべてのアクションゲームが生みの速度だけに依存するわけではない。正確な照準、リズムやタイミング、コンボ技（複雑なコマンドの連続入力）といった身体的スキルを要求するものもある。

アーケードゲームの多くがアクションゲームであるのは、短時間で未熟なプレイヤーを淘汰することで収益を上げる仕組みだからである。シンプルなコアメカニクスとゲームプレイは、携帯機やモバイル、ブラウザゲームにも適している。

1.2 アクションゲームのサブジャンル

シューター (Shooters)

シューターでは、プレイヤーは遠距離武器を使って行動する。照準がキースキルであり、弾薬が限られている場合は特に重要である。

2Dシューター: トップダウンまたはサイドビューの環境で展開される。敵はアバターに射撃したり近接攻撃を仕掛ける。多数の敵を可能な限り速く撃つ「シューテムアップ (shoot-'em-up)」が典型的。2Dシューターでは通常、弾薬管理は不要で、物理法則も非現実的（弾丸は直線で等速移動、キャラクターは慣性無視で即座に方向転換）である。



図: 古典的2Dシューターの例。Robotron: 2084 (1982年) は優れたゲームプレイにより長年人気を保った

3Dシューター: HaloやHalf-Lifeシリーズに代表される。2Dシューターよりリアルで、重力・音・影・衝突判定などが現実に近い形で実装される。一人称視点 (FPS) と三人称視点の両方がある。



図: Maze Warから現代の3Dシューターまで、グラフィックスの劇的な進化

3Dシューターの主な分類:

分類	特徴	代表例
レールシューター	分岐の少ない一本道。 始点から終点まで戦い抜く	Half-Life
タクティカルシューター	現実的な武器・状況。 ステルスとカバーが重要	Ghost Recon
サバイバルホラー	リアルなグラフィックで恐怖を演出。探索が大きな役割	Silent Hill, Resident Evil
アリーナゲーム	マルチプレイヤーのデスマッチ・チーム戦に特化	Quake III: Arena

プラットフォーム (Platform Games)

プラットフォームは、アバターがさまざまな高さのプラットフォームを飛び移りながら障害物を避け、敵と戦うゲームである。超人的なジャンプ力を持ち、高所から落下してもダメージを受けない（危険物や底なし穴を除く）。物理法則は非常に非現実的で、空中で方向転換が可能。Super Mario Bros.が代表例であり、暴力表現が穏やかで子供に適したものが多い。

格闘ゲーム (Fighting Games)

格闘ゲームは探索もシューティングもパズル解きも含まないが、反応速度とタイミングに高い身体的スキルを要求する。アジアの武術をモデルにした誇張された動きで手相手の戦闘をシミュレートする。

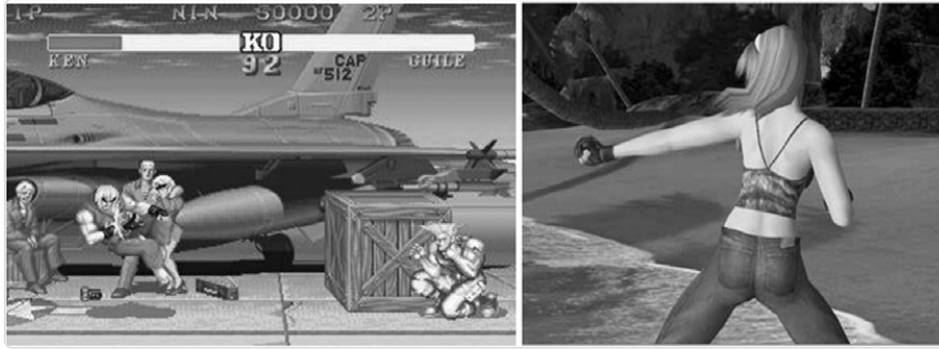


図: Street FighterとDead or Alive 3。格闘ゲームの進化を示す

格闘ゲームの重要な特徴: - **コンボ技**: 特定のボタン・ジョイスティック操作を高速で正確に入力することで発動する強力な攻撃。難易度が高いほど威力が大きい - **コンボメーター**: コンボ入力の進捗を視覚的に表示するインジケーター - **じゃんけんの戦略**: 特定の防御は特定の攻撃をブロックするが他はブロックしない。プレイヤーは相手の次の動きを予測しようとする

その他のサブジャンル

- ・ **ファストパズルゲーム**: Tetrisに代表される、時間制限のある抽象的パズルゲーム。カジュアルゲーマーに人気
- ・ **アクションアドベンチャー**: アクションとアドベンチャーのハイブリッド。Zeldaシリーズ、Indiana Jones等
- ・ **音楽・ダンス・リズムゲーム**: Dance Dance Revolution、Guitar Hero等。リズム感を試すゲーム

キーポイント: アクションゲームのサブジャンルは多様であり、シューター、プラットフォーム、格闘ゲーム、パズル、リズムゲームなど幅広い。共通するのは身体的スキルへの依存であるが、求められるスキルの種類はジャンルによって異なる。

1.3 ゲームの進行構造

レベル設計

アクションゲームのレベル進行は**直線的**であることが多い。全レベルをクリアすればゲームクリアとなる。レベルはテーマごとにグループ化され、各テーマの最後には**ボス (Big Boss)** が待ち構えている。

ペーシングの計画が特に重要であり、Mike Lopezは9段階のプロセスを提唱している: 1. ブレインストーミング (場所・興奮の瞬間の列挙) 2. 優先順位の設定 3. ストーリーフレームワークの作成 4. キーイベントの強度評価と配列 5. プロットポイントの評価と配列 6. 高強度イベント間の時間設定 7. 全体のトレンド評価 8. レベル構築開始 9. レビューと反復

チェックポイント

プレイヤーのアバターが死亡した際に再出現する地点。Sonic the Hedgehogでは、街灯がチェックポイントとして機能し、通過すると白から赤に変わる。

レベル出口・ワープ・テレポーター

- ・ **レベル出口**: 次のレベルへの遷移地点。隠されているか敵に守られていることが多い
- ・ **レベルワープ**: 数レベル先にスキップする隠し出口
- ・ **テレポーター**: 同じレベル内の別の場所にジャンプする遷移点

キーポイント: レベル設計においてはペーシング (緩急の配分) が最も重要な原則である。チェックポイントやワープは、難易度の調整と探索のモチベーションを両立させる仕組みである。

1.4 課題の種類

アクションゲームの課題は主に身体的スキルのテストである。

- ・ **受動的障害物**: 壁や深淵など、移動を妨げるが直接的に攻撃はしないもの

- ・ **固定的危険物**: 電気フェンス、振り子の刃など、近づくと攻撃してくるが移動しないもの
- ・ **能動的危険物（敵）**: 移動しながらアバターを攻撃する存在

ウェーブ: 敵が集団で出現する仕組み。スクリプト化されたウェーブ（毎回同じ構成）と、アルゴリズムで生成されるウェーブがある。

ビッグボス: テーマレベル群の最後に登場する強敵。通常の方法では倒せず、特殊な武器・攻撃法・タイミングが必要。

ワイルドカード敵: ウェーブの予測可能性を崩すためにランダムに出現する敵（AsteroidsのUFO等）。

モンスタージェネレーターとスポーンポイント: 新たな敵を生成する仕組み。Gauntletのように破壊可能なジェネレーターは、プレイヤーに「敵を倒すか、発生源を壊すか」という戦略的選択を与える。

1.5 コアメカニクスの特徴

アクションゲームのコアメカニクスは**シンプルで明快**であるべきである。プレイヤーは忙しすぎて複雑な内部経済を研究する余裕がない。

要素	説明
ライフ	通常3～5回の死亡猶予。敵との衝突で1ライフ失う。パワーアップやスコア到達で追加獲得
エネルギー（HP）	ヒットポイントまたはヘルス。危険な遭遇で減少し、枯渇すると死亡
パワーアップ	永続型（ゲーム終了まで持続）と一時型（時間やヒット回数で消滅）がある。強力なほど持続時間は短くすべき
コレクティブル	スコア加算やシークレットレベル解放に使うボーナスアイテム

要素	説明
タイムリミット	レベルクリアの制限時間、災害へのカウントダウン、パワーアップの有効期限など
スコア	プレイヤーの達成度の記録。ハイスコアテーブルで競争を促進

特殊アクション

- ・ **スマートボム**: 周囲の全敵を一掃する強力なアクション。使用回数が限られるため、使いどころの判断が重要
- ・ **ハイパースペースエスケープ**: ランダムな場所に瞬間移動。危険回避になるが、さらに悪い状況に陥る可能性もある

キーポイント: アクションゲームのコアメカニクスは少数のリソース（ライフ、エネルギー、スコア等）の単純な関係で構成される。パワーアップやコレクティブルは「リスクと報酬」のバランスを生み出す重要な要素である。

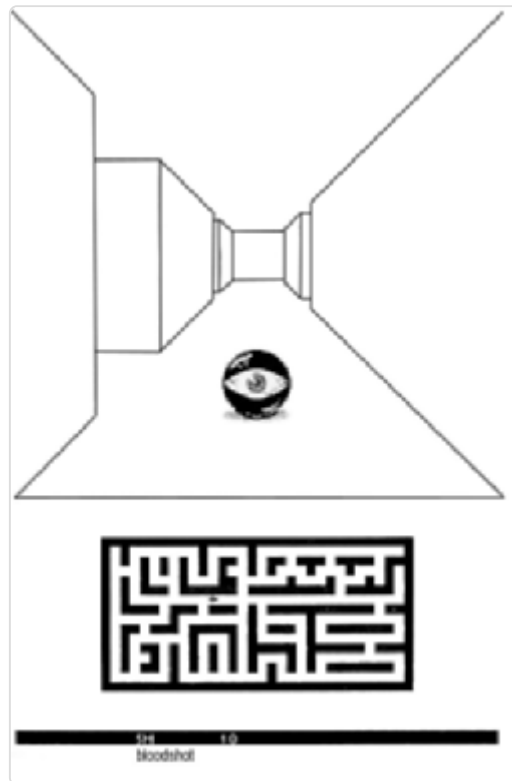
1.6 カメラモデルとUI

カメラモデル

2Dカメラ: サイドビューまたはトップダウンビュー。スクロール方式には以下がある: - **連続スクロール**: 一定速度で自動スクロール。プレイヤーは止められない - **可変スクロール**: プレイヤーの操作に応じてスクロール。ペースをコントロール可能 - **パララックススクロール**: 背景を前景より遅くスクロールさせ、奥行き感を演出

3D一人称視点 (FPS): 高速シューティングに最適。射撃対象がよく見えるが、アバター周囲の視野は狭い（約30度、人間の自然な視野120度の約4分の1）。

3D三人称視点: アバターの背後上方からの視点。アバターの周囲が見えるが、遠距離の視認性はFPSに劣る。Resident Evil 4はアバターの右肩越しカメラという革新を導入した。

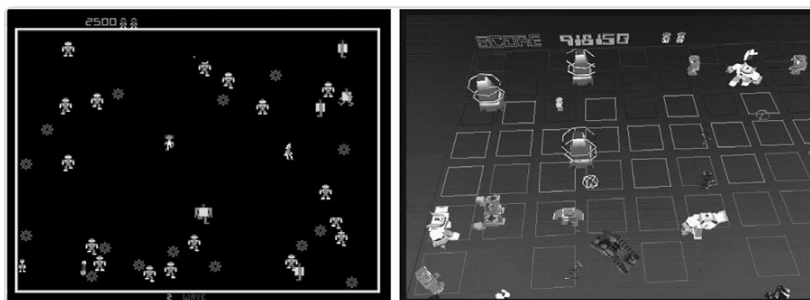


カメラモデルの例

コンテキスト依存カメラ: AIによりシーンに応じて最適なアングルを選ぶ。Icoが好例。ただし高速アクション時には混乱の原因になりうる。

ユーザーインターフェース設計の原則

- ・ 必要な情報だけを表示する（HUDはミニマルに）
- ・ 重要情報は常に画面上に表示する
- ・ 数値よりグラフィカルインジケータ（パワーバー等）を使う
- ・ 危険な状況では点滅等で注意を引く
- ・ マップはシンプルに。複雑な場合はポーズ画面で表示
- ・ オーディオフィードバックを豊富に使う



UIの例

キーポイント: カメラモデルの選択はゲームプレイに直接影響する。2Dは視認性に優れ高速な展開が可能、3Dは環境の活用と驚きの演出に優れる。UIは「可能な限りシンプルに、しかし必要以上に単純にしない」ことが原則。

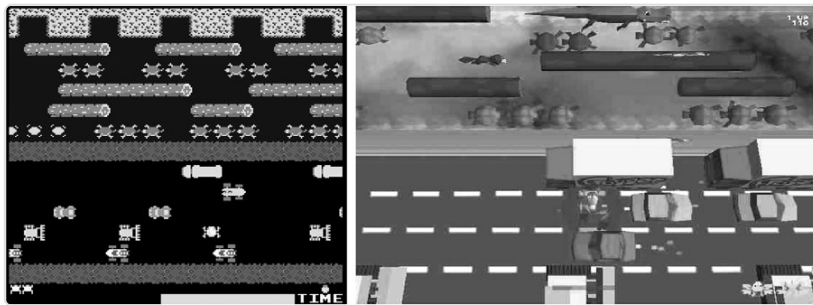
2. ストラテジーゲーム

2.1 ストラテジーゲームとは

定義: ストラテジーゲームとは、提示される課題の大部分が戦略的対立であり、プレイヤーがゲーム中のほとんどの場面で多様な行動・手を選択できるゲームである。勝利は優れた計画と最適な行動の選択によって達成され、偶然の要素は大きな役割を果たしてはならない。

ストラテジーゲームは世界最古のゲームジャンルの一つであり、囲碁（紀元前約2200年）やウルの王朝ゲーム（紀元前約2500年）に遡る。PCゲームとして発展し、コンピュータの計算力を活かした複雑なルールセットの管理が可能となった。

他のジャンルに比べて**対称性が高い**ため、バランス調整が比較的容易である。両陣営に同一またはそれに近いリソースと行動が与えられる。



ストラテジーゲームの例

2.2 ターンベース vs リアルタイム

項目	ターンベース	リアルタイム (RTS)
時間管理		すべてが同時進行。素早い判断が必要

項目	ターンベース	リアルタイム (RTS)
	プレイヤーが好きなだけ考慮可能	
テンポ	低速～中速	中速～高速
代表例	Civilization, XCOM, チェス	StarCraft, Age of Empires, Warcraft
特徴	深い思考と最適解の探求	マルチタスクと迅速な意思決定
マルチプレイヤー	同時ターン制で待ち時間短縮可能	自然に同時プレイ

純粋なストラテジーゲーム（対立のみで経済や物理的課題のないもの）はターンベースである傾向が強い。RTSはターンベースの後に登場し、時間的プレッシャーを追加した。

2.3 課題の種類

戦略的対立

ストラテジーゲームの中心的課題。戦闘は**ユニット**と呼ばれる個別の戦闘員の間で行われる。ユニットは古代エジプトの戦士から宇宙船まで多岐にわたる。戦闘ユニットの他に、輸送・偵察ユニット、建物型の**工場**（新しいユニットを生産する施設）もある。

ゲームの戦術は設定のリアリズム度合いに応じて、側面攻撃、奇襲、補給線の切断、悪天候の活用など多岐にわたる。

戦闘の代替手段

高度なストラテジーゲームでは、外交・危機管理・諜報が含まれる。Civilization IIIでは外交的交渉が可能で、同盟を結んだり戦争を回避したりできる。Balance of Powerは外交ストラテジーの傑出した例で、冷戦時代の米ソ対立をテーマに、核戦争のリスクを背景にした政治的駆け引きを実現した。

探索の課題

多くのストラテジーゲームは**戦場の霧 (Fog of War)**を採用している。これは未探索の領域を黒く、探索済みだが部隊のいない領域を薄暗く表示する手法である。プレイヤーは偵察部隊を送り出して地形を明らかにする必要があり、防御措置を怠ると不意打ちを受ける。



図: Civilization IIIにおける戦場の霧。明るい領域が現在可視、薄暗い領域が既探索だが未パトロール、黒い領域が未探索

経済的課題

チェスのように初期ユニット固定のゲームもあるが、多くの現代ストラテジーゲームは**資源の獲得と管理**を含む。Age of Empiresでは食料・木材・石材・金の4資源を採集・管理し、工場の建設やユニットの生産に使用する。

資源ラッシュの防止: マップを対称的にレイアウトし、各陣営の近くに同程度の資源を配置する。遠方の資源鉱山はボーナス程度に留め、経済的優位が戦略的スキルを圧倒しないようバランスを取る。

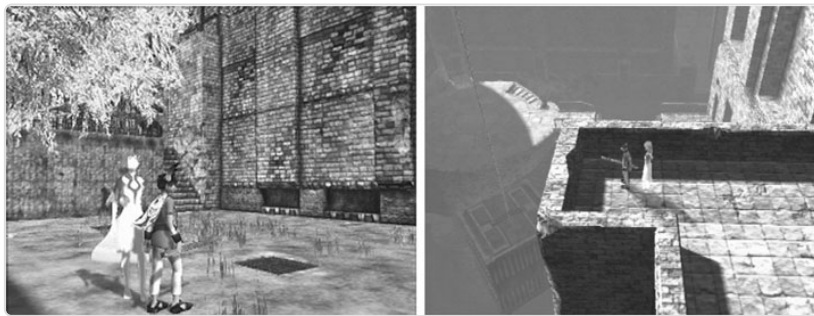
キーポイント: ストラテジーゲームの課題は戦略的対立を中心に、探索・経済・外交が複合的に絡み合う。戦場の霧は緊張感を生み、経済管理はプレイヤーに多層的な意思決定を求める。

2.4 プレイヤーの行動

ストラテジーゲームにおけるプレイヤーの役割は指揮官・管理者であり、ユニットに命令を出すことが主な行動である。代表的なコマンド:

- ・ **移動**: 指定地点への移動（ウェイポイント経由も可能）
- ・ **攻撃**: 敵ユニットへの攻撃（射程内に入るまで前進し、後退すれば追撃）
- ・ **停止**: 移動・攻撃の中止
- ・ **防衛 (Hold)** : 現在位置を保持し、射程内の敵のみ攻撃。追撃しない
- ・ **陣形**: 戦術的に有利なフォーメーションの編成
- ・ **生産**: 資源を消費して新ユニットを作成

その他: 後退 (Retreat) 、突進 (Dash) 、パトロール (Patrol) など一般的。



StarCraftのコマンドインターフェース

2.5 コアメカニクス

ユニット設計

ユニットは**タイプ**に分類され、同タイプのユニットは共通の属性を持つ。主要な数値属性:

属性	説明
HP (ヘルスポイント)	ダメージを受けると減少。0で破壊
武器	

属性	説明
	ショットパワー（1発あたりのダメージ）、発射速度、弾薬数
射程	武器がダメージを与えられる最大距離
命中精度	0（命中しない）～1（必中）の値
回避能力	敵の射撃を避ける能力。0～1の値
移動速度	地形の難易度により実際の速度が変動
旋回速度	新しい方向を向く速さ。瞬時では側面攻撃が無効化される
視界範囲	戦場の霧を使用する場合に関係

じゃんけん（RPS）モデル: The Ancient Art of Warのように、騎士>蛮族>弓兵>騎士という単純な相性関係。シンプルだが複雑なゲームには不向き。現代のゲームでは数値属性から相性関係が自然に生まれるように設計する。

無敵ユニットの禁止: 武器の射程と最大速度の両方が全敵ユニットを上回るユニットは設計してはならない。ゲームバランスが崩壊する。

特殊能力

- ・ **ステルス:** 敵に見えなくなる。武器使用時には姿を現すよう設計するのが通例
- ・ **飛行・航行:** 地上ユニットが通れない地形を移動可能
- ・ **修理:** 自己修復またはメディカルユニットによる回復
- ・ **輸送:** 他のユニットを高速で運搬。通常は非武装
- ・ **建設・生産:** 建物の建設や移動ユニットの製造
- ・ **リーダーシップ:** 周囲のユニットに戦闘ボーナスを付与

ランチェスターの法則

線形法則: 白兵戦では、軍の強さは兵力数に単純比例する。

二乗法則: 遠距離射撃戦では、軍の強さは兵力数の**二乗**に比例する。3倍の兵力は9倍の戦力を意味する。これはユニットの**生産速度のバランス**が極めて重要であることを示してい

る。一方がわずかでも多くユニットを生産できれば、その優位性は数の差の二乗で拡大する。

キーポイント: ユニット設計では数値属性のバランスが決定的に重要。ランチェスターの法則は、生産速度のわずかな差が勝敗を大きく左右することを教えている。特殊能力は対抗手段とセットで設計すべきである。

2.6 アップグレードとテクノロジーツリー



図: Civilization IIIのテクノロジーツリーの一部。左から右へ研究が進む

ゲームの進行に伴い、新しいユニットや能力を追加する仕組み:

アップグレードの種類: - **個別ユニットアップグレード:** チェッカーの「キング」のように特定のユニットに適用 - **ユニットタイプアップグレード:** StarCraftのシージタンクのように、同タイプの全ユニットに適用 - **グローバルアップグレード:** Civilizationのフーバーダムのように、社会全体に影響

永続 vs 一時アップグレード: ターンベースゲームでは永続型（残りのゲーム全体に有効）、RTSでは一時型（そのレベルのみ有効）が多い。

テクノロジーツリー: 多数のアップグレードを分岐するツリー構造に組織化したもの。例えば「蒸気機関」を研究すると「鉄道」「蒸気船」「動力工場」の選択肢が開放される。プレイヤーに戦略的な選択を強いる重要な仕組みである。

2.7 ロジスティクス（兵站）

実際の戦争では兵站は極めて複雑だが、ゲームでは簡略化される。

- ・ **補給と消耗品:** 食料・燃料は通常無視。弾薬は追跡する場合と無制限の場合がある。原則として「些末な事に煩わされない」
- ・ **補給線:** 部隊と物資の輸送ルート。切断は強力な戦略だが、ゲームでの実装は複雑
- ・ **抽象化:** Warcraftでは食料は農場で生産されるが、全部隊に魔法のように配布される。リアリズムより利便性を優先
- ・ **道路建設:** Civilization IIIでは道路ネットワークが資源分配を制御。道路で接続された町は資源を共有
- ・ **影響マップ:** 補給拠点の一定範囲内にいるユニットにのみ補給が届く仕組み。StarCraftのProtossパワービーコンが好例

キーポイント: ロジスティクスはリアリズムとプレイアビリティの妥協点を見つけることが重要。完全な再現は煩雑すぎるため、道路建設や影響マップなどの抽象化手法が有効である。

2.8 ゲームワールドの設定

ストラテジーゲームのコアメカニクスは様々な設定に移植可能だが、設定ごとに注意点がある。

設定	利点	注意点
歴史	プレイヤーが背景知識を持つ。没入感が高い	歴史に詳しいプレイヤーの厳しい目。第二次世界大戦は飽和状態

設定	利点	注意点
現代	リアルな武器と状況	実在の紛争を扱うと論争を招く。規模の問題（歩兵 vs ジェット機）
SF（未来）	自由な発明が可能	用語が直感的でない。規模の問題。技術の一貫性の維持が困難
ファンタジー	魔法による独自の戦闘。親しみやすい	北欧神話に偏りがち。他文化の民話から着想を得るべき



ゲームワールドの設定例

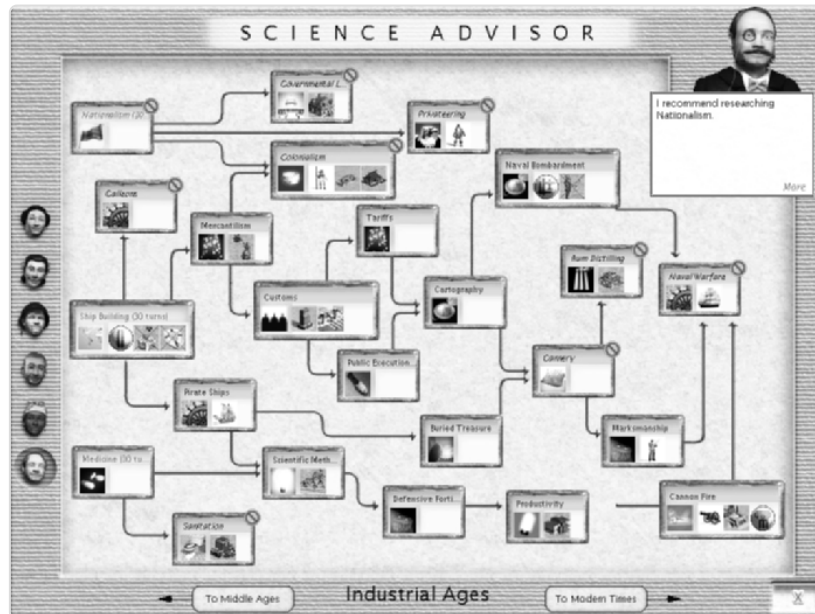
2.9 AI対戦相手

シングルプレイヤーのストラテジーゲームでは、人工知能（AI）が対戦相手を務める。主な手法:

ゲームツリー探索: ターンベースゲーム（チェッカー、チェス等）で使用。将来の手を探索し最も有利な手を選ぶ。選択肢が多いゲーム（囲碁等）では計算量が指数関数的に増大するため限界がある。

ニューラルネット: 脳のニューロンの動作を模倣し、成功につながるプレイパターンを学習する。学習に時間がかかり、複雑なパターンには不向き。

階層的有限状態マシン (hFSM)： 最も成功している手法。複数のFSM（有限状態マシン）が階層構造を成し、上位が下位に命令を出す。実際の軍隊の指揮系統に似た構造で、「この丘を取れ」という上位目標が「援護射撃」「陽動」といった下位タスクに分解される。



AIの概念図

hFSMの利点: - **創発的行動:** 明示的にプログラムされていない行動が生まれる - **トップダウン設計:** 大規模戦略を小規模戦略に分解して設計可能 - **汎用性:** 戦闘だけでなく経済目標にも適用可能

設計ルール: AIにユニットをマイクロマネジメントさせてはならない。階層の各レベルでインテリジェントな行動を設計すれば、インテリジェントな結果が自然に生まれる。

キーポイント: AI対戦相手はストラテジーゲームの品質を左右する重要要素。hFSMが最も実用的な手法であり、階層的な命令系統をモデル化することで、大規模な戦場でも合理的な行動を実現できる。

まとめ：2つのジャンルの比較

項目	アクションゲーム	ストラテジーゲーム
主な課題	身体的スキル（反応・照準・タイミング）	知的スキル（計画・意思決定・資源管理）
テンポ	高速（ツイッチゲームでは反射レベル）	低速～中速（RTSでも戦略的思考が中心）
プレイヤーの役割	アバターの直接操作	指揮官として間接的にユニットを管理
コアメカニクス	シンプル（ライフ・スコア・パワーアップ）	複雑（ユニット属性・生産・テクノロジーツリー）
バランス調整	非対称で困難	対称性が高く比較的容易
偶然の要素	少ない（スキル依存）	制限すべき（戦略依存）だが命中率等で存在

両ジャンルはゲームデザインの基本原則を異なる角度から照らし出す。アクションゲームはプレイヤーの即時的な身体反応を中心に据え、ストラテジーゲームは長期的な計画と意思決定を中心に据える。いずれにおいても、バランスの取れた課題設計とプレイヤーにとって意味のある選択の提供が良いゲームデザインの核心である。

第12回：RPGとスポーツゲーム

はじめに

本講では、ゲームジャンルの中でも特に奥深い2つのジャンル——**RPG（ロールプレイングゲーム）**と**スポーツゲーム**について学ぶ。RPGは多様な遊び方と豊かな物語体験を提供するジャンルであり、スポーツゲームは現実のスポーツを忠実にシミュレートしつつ、ビデオゲームならではの楽しさを追求するジャンルである。両者はまったく異なる方向性を持つが、いずれもコアメカニクス設計が極めて重要である点で共通している。

第1部：RPG（ロールプレイングゲーム）

RPGの定義

RPGとは、プレイヤーが1人または複数のキャラクターを操作し、コンピュータが管理する一連のクエストを通じて冒険するゲームである。クエストの達成が勝利条件であり、キャラクターの能力成長がジャンルの核心的特徴である。典型的な課題には、戦術的戦闘、物流管理、経済成長、探索、パズル解決が含まれる。

コンピュータRPG（CRPG）は、『ダンジョンズ&ドラゴンズ』に代表されるテーブルトップRPGから発展した。テーブルトップRPGでは、ゲームマスター（GM）が冒険を演出し、プレイヤーは想像力を駆使してあらゆる行動を提案できる。CRPGではコンピュータがGMの役割を担い、ルールの実装やゲーム世界の表示を行う。コンピュータは即興的なルール変更ができないため柔軟性では劣るが、美しいグラフィックスにより初心者にも親しみやすい。

CRPGの本質的な要素は、**クエスト（物語）**と**キャラクター成長**の2つである。

RPGの他ジャンルとの比較

RPGは多くのジャンルと要素を共有しており、ハイブリッド作品も多い。

比較対象	共通点	相違点
ウォーゲーム	戦闘ルールが存在	RPGは少数の異質なキャラクター、ウォーゲームは大量の同質なユニット
アクションゲーム	一部のRPGに物理的挑戦あり	RPGは売買や会話など非アクション要素が豊富
アドベンチャーゲーム	豊かなストーリーと探索	RPGはキャラクターを数値属性で定義し、戦闘が中心

代表的なハイブリッド作品として、The Elder Scrollsシリーズ（アクションRPG）、Final Fantasyシリーズ（アドベンチャーRPG）がある。

ゲームの特徴

テーマと進行

CRPGの多くは「あなただけが世界を救える！」という前提で展開される。しかしこれは使い古されたクリシェである。以下のような代替テーマも検討すべきである：

- ・ 愛する人の殺害犯を見つけて罰する
- ・ 隠された出自の秘密を学ぶ
- ・ 誘拐された王子/王女を救出する
- ・ 散逸した魔法のアイテムを再組立する
- ・ 虚偽の告発から自らの潔白を証明する

Planescape: Torment は、世界を救うのではなくアバターの過去を発見するという独創的なテーマを採用した優れた例である。

RPGの進行は通常、長大なクエストがエピソードに分割され、各エピソードの終わりにボス戦のような大きな挑戦が待つ線形構造をとる。メインクエストに加え、プレイヤーが自由に受諾・拒否できる**サイドクエスト**がゲームに深みを与える。



RPGの典型的な進行構造

ゲームプレイモード

CRPGは他のどのジャンルよりも多くの活動を提供する。主要なゲームプレイモードは以下の4つである：

1. **探索と戦闘** – パーティを操作して世界を探索し、敵と戦う。伝統的なパーティベースRPGではアイソメトリック視点が多い。
2. **会話** – NPCとの対話をダイアログツリーで管理する。適切な質問により有用な情報や経験値を得られる。
3. **取引** – 町の商人とアイテムの売買を行う。値段交渉が可能な場合もある。
4. **インベントリ管理** – キャラクターが持ち歩くアイテムを整理する。重量制限やスペース制限がある。



Baldur's Gate II: アイソメトリック視点での探索と戦闘



Dungeon Siege IIのインベントリモード

設計上の注意： インベントリ管理にプレイヤーが過剰な時間を費やすのは好ましくない。矩形パッキング問題（アイテムをグリッドに詰め込む作業）は退屈になりがちであり、自動整理機能やアイテムの回転機能を実装すべきである。

キーポイント：RPGのゲーム特徴

- ・ RPGの進行はエピソード型のクエスト構造で、各エピソードの終わりに大きな挑戦がある
- ・ メインクエストとサイドクエストの組み合わせがゲームに深みを与える
- ・ 探索・戦闘・会話・取引・インベントリ管理の5つの活動がRPGの中核をなす
- ・ インベントリ管理のUIはプレイヤーの負担を最小限にする設計が重要

コアメカニクス

ダイスロールと確率

RPGの行動判定はダイスロール（コンピュータによるシミュレーション）に基づく。例えば、戦士が扉を破壊するとき、腕力17に対して3個の6面ダイスを振り、合計が腕力以下なら成功する。このメカニクスがRPGの戦闘やその他の活動の基盤となる。

設計者は確率分布を理解する必要がある。3個の6面ダイスで18を出す確率は0.5%未満であり、難易度設定は確率に基づいて慎重に行わなければならない。

キャラクター属性

キャラクター属性は**特性属性（Characterization Attributes）**と**状態属性（Status Attributes）**に大別される。

特性属性（変化頻度が低い）：

属性種別	説明	例
種族（Race）	キャラクターの体型や外見を決定	人間、ドワーフ、エルフ
性別		－

属性種別	説明	例
	体型や関係性に影響 (能力制限は設けるべきでない)	
クラス	特定の行動やスキルの 専門化を決定	戦士、魔法使い、 盗賊、聖職者
身体属性	移動・戦闘時の能力を 決定	筋力、器用さ、耐 久力
精神属性	学習・推理の能力を決 定	知性、正気度
道徳属性	善悪の傾向を決定	善、悪、中立
社会属性	他者との関係構築能力	カリスマ、リー ダーシップ

状態属性（頻繁に変化する）：

- ・ **HP（ヒットポイント）**：キャラクターの現在の体力
- ・ **経験値（XP）**：成長の度合いを測る指標。敵の撃破やクエスト達成で獲得
- ・ **キャラクターレベル**：XPが一定値に達するとレベルアップし、属性を強化できる

設計ルール：HPが低いキャラクターの戦闘能力を減少させてはならない。先制攻撃が圧倒的に有利になり、ゲームバランスが崩れる。

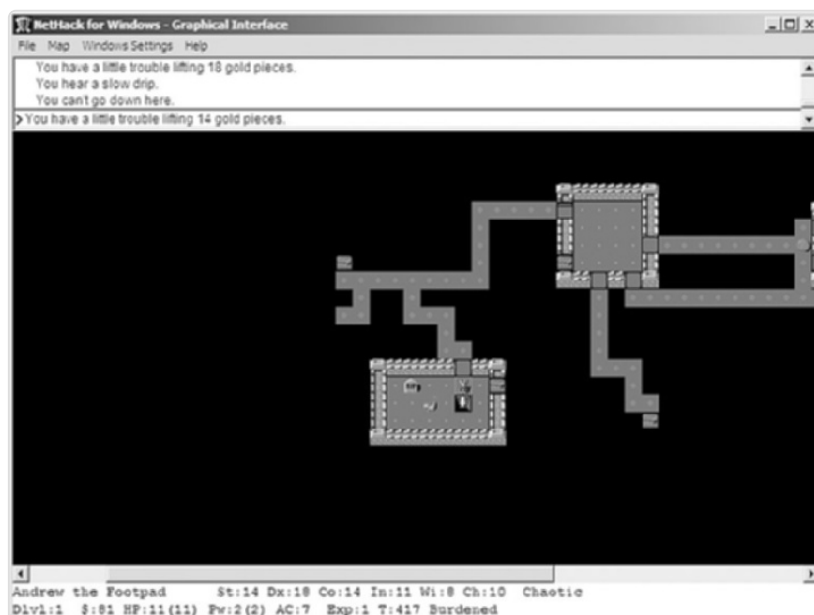
魔法システム

魔法（またはサイキックパワー、先端技術など）は、RPGの特徴的な要素である。一般に魔法使い専用のクラスに制限され、物理戦闘と魔法戦闘の補完関係がパーティ編成の動機となる。

魔法の制限方法として現在主流なのは**マナシステム**である。各呪文にマナコストが設定され、キャラクターのマナが尽きるまで自由に呪文を組み合わせ使用できる。マナはポーション消費や時間経過で回復する。

スキルツリー

キャラクターは時間の経過とともに新しいスキルを習得できる。スキルは通常、ツリー構造（スキルツリー）で整理され、基礎スキルを習得することで上位スキルが解放される。



Diablo IIのスキルツリー

スキルの習得方法には、経験値による自動獲得と、メンターNPCからの有料学習がある。戦略ゲームのユニットアップグレードとは異なり、RPGのスキルはキャラクター個人に永続的に帰属する。

キーポイント：RPGのコアメカニクス

- ・ダイスロールベースの確率判定がすべての行動の基盤
- ・キャラクター属性は特性属性（変化頻度低）と状態属性（変化頻度高）に分かれる
- ・経験値とレベルアップがキャラクター成長の中心的仕組み
- ・マナシステムによる魔法の制限が現在の主流
- ・スキルツリーにより段階的な能力獲得を実現

ゲームワールドとストーリー

設定（セッティング）

CRPGの世界はファンタジーやSFの設定が多い。その理由は：

1. 魔法や先端技術により、現実では不可能な体験を提供できる
2. 多様な敵やモンスターを登場させられる
3. キャラクターの非現実的な急速な成長をより自然に見せられる

ただし、現代を舞台にしたRPG（警察官やスパイが新しいスキルを獲得していく物語）も十分に成立する。設定がどのようなものであれ、RPGはスローペースのゲームであるため、**探索して楽しい魅力的な世界**を構築することが不可欠である。

ストーリー設計

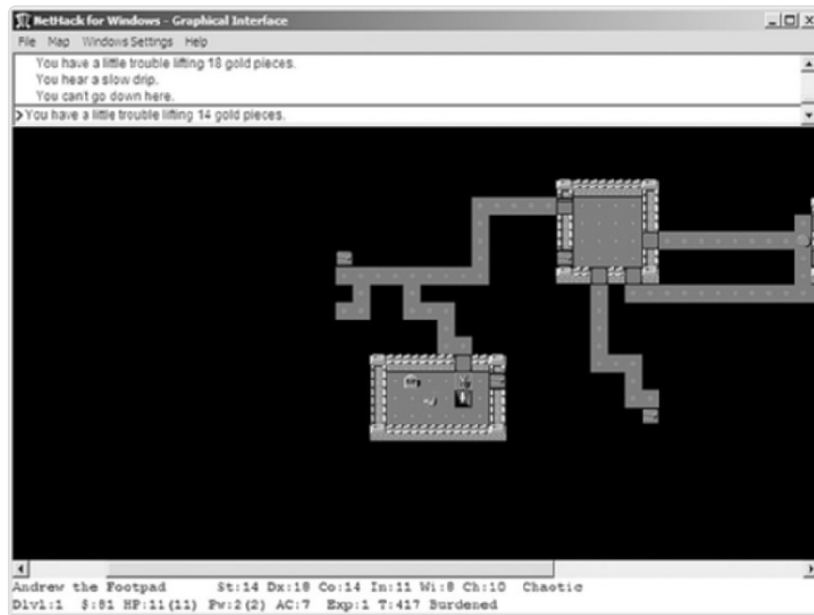
CRPGのストーリーは映画よりもはるかに長く、長編小説に匹敵する。設計の手順は以下の通り：

1. **全体クエスト（エンディング）を決定する** – 序盤のクエストは最終目的と異なることが多い（偽の目的からの転換）
2. **エピソードごとの旅程を設計する** – 各エピソードは異なる地域で展開
3. **プロットの転換点を配置する** – 失われた親族の出現、敵が味方に転じる展開など
4. **サイドクエストを追加する** – メインクエストの間接的支援となるものが望ましい
5. **オープニングの演出を設計する** – 謎と誘引のバランスが重要

Fallout のオープニングは優れた例である。地下シェルターの浄水チップを探すという単純な任務から始まるが、次第に複雑化し、最終的にシェルター全体の存亡をかけた冒険へと発展する。

プレゼンテーション層

- ・ **インタラクションモデル**：多くのCRPGはパーティベース。少数のゲーム（Diablo など）はアバターベース
- ・ **カメラモデル**：アバターベースでは一人称/三人称視点、パーティベースではアイソメトリックまたは俯瞰視点が一般的



Elder Scrolls III: Morrowindの三人称視点



Knights of the Old Republic IIの戦闘インターフェース

CRPGのユーザーインターフェースは他のジャンルよりも複雑で、ミニマップ、体力表示、戦闘オプション、パーティポートレート、モード切替ボタンなどが同時に表示される。

キーポイント：ゲームワールドとストーリー

- ・ ファンタジー/SF設定は急速なキャラクター成長を自然に見せる効果がある
- ・ ストーリーは単純なクエストではなく、謎解きと転換点を含む長編構造にする
- ・ サイドクエストはメインクエストを間接的に支援する内容が望ましい

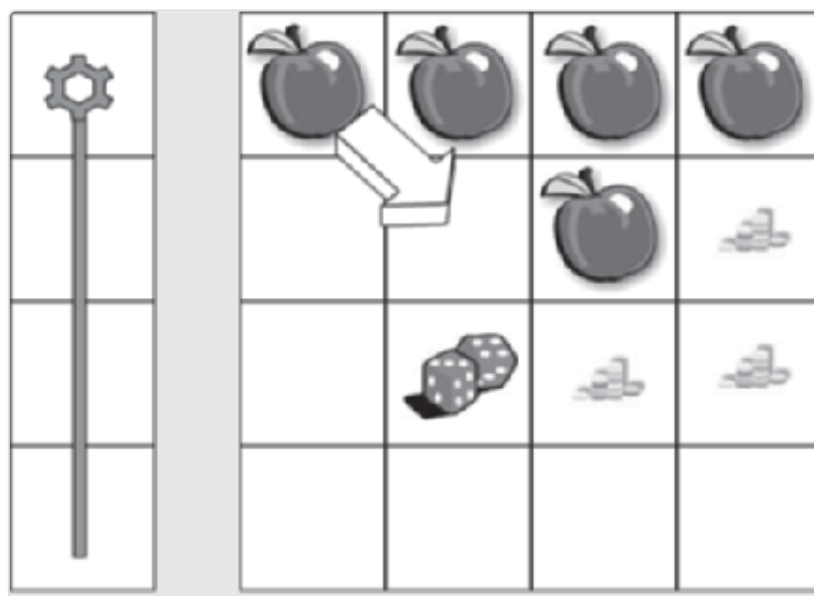
- ・ オープニングは謎と動機のバランスが鍵

第2部：スポーツゲーム

スポーツゲームの定義

スポーツゲームとは、現実または架空のスポーツの何らかの側面をシミュレートするゲームである。試合のプレイ、チームやキャリアの管理、またはその両方を含む。試合プレイは身体的・戦略的チャレンジを用い、マネジメントは主に経済的チャレンジである。

スポーツゲームは他のジャンルとは異なり、プレイヤーが**すでによく知っている世界**を再現する。そのため、現実との直接的な比較が避けられず、ファンは逸脱を欠陥と見なす。一方で、Mutant League Football のようなファンタジー系や、Blitz シリーズのような誇張系の作品も存在する。



スポーツゲームの例

ゲーム構造とプレイヤーの役割

スポーツゲームの主要なゲームプレイモードは**試合プレイ（マッチプレイ）**であり、その外側にチーム管理機能がある。

プレイヤーの3つの役割：

役割	内容
選手	試合中にアスリートを操作。チームスポーツでは操作対象が状況に応じて切り替わる
コーチ	先発メンバーの選定、攻守の戦略設定、選手交代を行う
ゼネラルマネージャー	選手の雇用・解雇・トレード、チームの長期的強化を担当

競技モード

スポーツゲームはあらゆる競技形態を提供する：

- ・ **シーズンモード** – 1シーズンを通じてチャンピオンシップを目指す
- ・ **エキシビションモード** – 1試合限りの対戦
- ・ **サドンデス** – 最初の得点で勝敗が決まる短時間モード
- ・ **ラウンドロビン** – 総当たり戦
- ・ **トーナメントモード** – 敗退制のトーナメント
- ・ **フランチャイズモード（ダイナスティモード）** – 数シーズンにわたりチームを育成。個人競技では**キャリアモード**と呼ばれる

また、スポーツゲームに特有の「コンピュータ同士の対戦」モードも重要である。プレイヤーが試合を観戦したり、プレイしたくない試合を自動シミュレートする際に使用される。

キーポイント：スポーツゲームの構造

- ・ 試合プレイが中核、管理機能が補完する二層構造
- ・ プレイヤーは選手・コーチ・GMの3役割を担う
- ・ シーズン、フランチャイズなど多様な競技モードが長期的プレイを支える
- ・ コンピュータ同士の対戦モードはシーズン制ゲームで不可欠

コアメカニクス

物理シミュレーション

スポーツゲームの物理エンジンは、試合中の物体と人体の挙動を決定する。ボールの物理挙動は比較的単純だが、人体の物理挙動は複雑である。現代のスポーツゲームは路面の摩擦係数（雨天時の滑り）なども考慮する。

重要な設計原則： 物理シミュレーションは「できるだけリアル」であるべきではない。その理由は2つある：

1. プレイヤーはフィールドにいるのではなく、コントローラーを通じて操作している
2. プレイヤーはプロ選手ではない（時速95マイルの投球に対するバットのスイングタイミングは約0.04秒）

物理を現実と完全に一致させるのではなく、プロスポーツの**合理的なシミュレーション**に見えることが重要である。

選手レーティング

選手の能力評価は、物理エンジンにデータを供給するために不可欠である。

共通レーティング： - スピード、俊敏性、体重、加速力、跳躍力、持久力、怪我耐性

ポジション別レーティング（例：アメフトのQB）： - パス距離、パス精度、手先の器用さ、状況認識力

選手AI

スポーツゲームのAIは、アクションゲームの敵AIとは根本的に異なる。各選手は**チームの集団目標**に基づいて知的に行動しなければならない。

AI設計の手順： 1. **状態空間を定義する** - 試合のルールと戦術に基づいてゲーム状態を洗い出す（例：サッカーのコーナーキックは「キック前」「ボール空中」「他の選手に触れた後」など複数の状態に分かれる） 2. **集団目標と個人目標を設定する** - 各状態における各選手の役割を決定 3. **具体的な行動を定義する** - 移動方向、アニメーションなどを指定



サッカーのコーナーキック状態遷移の例

何もしていない選手にも「フィジエット」（体重移動やストレッチなどの待機アニメーション）を入れ、プレイ中はアクションの方を向かせるべきである。

アーケードモード vs シミュレーションモード

- ・ **アーケードモード** – アクションを増やしリアリズムを犠牲にする（例：打率を.333から.500に引き上げ）
- ・ **シミュレーションモード** – 現実に忠実な再現を優先

設計上は、**まずシミュレーションモードを完成させ**、次にアーケード向けの調整を行うべきである。

キーポイント：スポーツゲームのコアメカニクス

- ・ 物理シミュレーションは「リアル」より「プレイアブル」を優先する
- ・ 選手レーティングは共通属性とポジション別属性の二層で構成
- ・ AIは状態空間の定義→集団/個人目標の設定→具体行動の定義の順で設計
- ・ アーケード/シミュレーションの切り替えはプレイヤー層の幅を広げる

プレゼンテーション層

インタラクションモデルとカメラ

スポーツゲームでは、プレイヤーの操作対象が試合状況に応じて**自動的に切り替わる**。例えばバスケットボールではボールを持つ選手に操作が移る。この切り替わりに伴い、UIのボタン機能も刻々と変化する。

カメラの指針： - 一対一の競技ではアスリートの動きが見える視点を選ぶ（一人称は避ける） - チーム競技ではやや高い位置からのサイドビューまたはエンドビューが有効 - 野球のように焦点が2つに分かれる競技では**ピクチャー・イン・ピクチャー**方式が有効



MVP Baseball 2005のピッチャー視点

音声解説（オーディオコメンタリー）

TVスポーツの実況を再現するため、多くのスポーツゲームは音声解説を実装する：

- ・ **実況アナウンサー** – プレイごとの状況を逐一解説
- ・ **解説者** – 戦術面の分析や選手の背景情報を提供
- ・ **スタジアムアナウンサー** – 場内放送

解説を設計するには、実際のTV中継を録画して書き起こし、どのイベントがどの種類のコメントを引き起こすかのパターンを分析する。プログラマーと連携して、ソフトウェアが検出可能なイベントに基づいてコメントを設計する。

天候とインスタントリプレイ

天候は試合に直接影響する。雨や雪はトラクションを低下させ、暑さは選手の疲労を早める。

インスタントリプレイはスポーツゲームに不可欠な機能であり、以下の操作が必要である： - 再生・停止・早送り・巻き戻し・コマ送り - カメラの3次元移動と角度調整 - 特定の選手やボールへのカメラロック

ライセンスと権利

スポーツゲーム開発には法的配慮が不可欠である：

- ・ **チーム・リーグの商標** – リーグの許諾なしにチーム名やロゴは使用不可
- ・ **選手の肖像権** – 個人のパブリシティ権の取得が必要
- ・ **写真の著作権** – 撮影者と被写体双方の許諾が必要

ライセンス取得は高コスト・長期間を要するため、それがなくても成立するゲーム設計を考慮すべきである。

キーポイント：スポーツゲームのプレゼンテーション

- ・ 操作対象の自動切り替えとUIの動的変更がスポーツゲーム特有の課題
- ・ 音声解説はTV中継のパターン分析に基づいて設計する
- ・ インスタントリプレイはプレイヤー体験と販促素材の両面で重要
- ・ ライセンス・権利問題は開発初期から計画に組み込む

まとめ

RPGとスポーツゲームの比較

観点	RPG	スポーツゲーム
プレイヤーの知識	ゲーム世界を新たに学ぶ	現実のスポーツの知識を持っている
コアメカニクス		物理シミュレーション・選手レーティング

観点	RPG	スポーツゲーム
	ダイスロール・属性・スキルツリー	
主要な課題	戦闘・探索・パズル・経済	身体的操作・戦略・チーム管理
物語の重要度	極めて高い	低い（シーズン展開が物語の代替）
キャラクター成長	ジャンルの中核	レーティングの微調整にとどまる
UI複雑度	非常に高い	状況依存で動的に変化

両ジャンルに共通するのは、**コアメカニクスの綿密な設計**がプレイヤー体験の質を決定するという点である。RPGではキャラクター属性と確率計算の設計が、スポーツゲームでは物理シミュレーションとAIの設計が、ゲームの成否を左右する。

本資料は Ernest Adams 『Fundamentals of Game Design』 第15章・第16章に基づく。

ewpage

第13回：乗り物シミュレーションと建設・経営シミュレーション

第1部：乗り物シミュレーション (Vehicle Simulations)

1.1 乗り物シミュレーションとは

乗り物シミュレーションとは、実在または架空の乗り物を操縦する感覚を再現するゲームジャンルである。空・地上・水上・宇宙といった多様な環境を舞台に、レース、探索、戦闘などの体験を提供する。

このジャンルの最大の特徴は**真実らしさ (verisimilitude)** の追求にある。プレイヤーは実際に運転・操縦しているかのような感覚を期待する。私たちの多くは自動車の運転経験があるため、時速200kmでの走行や武装車両の操作といった非日常の体験であっても、「本物らしさ」に対する基準を持っている。

一方、架空の乗り物を扱う場合は、重力やG、燃料容量などの現実的制約から自由に、移動の感覚そのものを自在にデザインできる。

1.2 ゲームの特徴

フライトシミュレーター

フライトシミュレーターは大きく **民間機**と**軍用機**の2種類に分かれる。

- ・ **民間フライトシム**：航空機の外観と性能の再現に重きを置き、戦闘要素はない。Microsoft Flight Simulatorが代表例で、夜間飛行、悪天候、計器飛行など多様な状況での操縦を体験できる。着陸が最も困難なチャレンジとなる。
- ・ **軍用フライトシム**：操縦に加え、敵機や地上施設への攻撃といったミッション遂行が求められる。現代の長距離ミサイル戦よりも、第一次・第二次世界大戦のドッグファイト（近接空中戦）が人気を保っている。



図1：Crimson Skiesのパイロット視点。シンプルな計器パネルが特徴的

ドライビングシミュレーター

ドライビングシミュレーターも2つのカテゴリに分かれる。

- ・ **実在レースの再現**：IndyCar、NASCAR、Formula 1などの公式レースを模擬する。公式名称やロゴの使用にはライセンスが必要。
- ・ **架空のレース**：都市や田園地帯、ファンタジー環境を舞台にした自由なレースゲーム。

プレイヤーの分類

プレイヤーは**ピュアリスト（純粋主義者）**と**カジュアルプレイヤー**に大別される。ピュアリストはフラップの戻し忘れによる損傷まで再現することを求めるが、カジュアルプレイヤーは細部よりも「速く飛び回り、撃ちまくる」体験を重視する。この両者のバランスをどう取るかが設計上の重要な判断となる。

競技モード

乗り物シミュレーションでは以下の競技モードが一般的である。

モード	内容
シングルプレイヤー	AI対戦相手との対戦
マルチプレイヤー	デスマッチ、レース
キャリアモード	パイロット/ドライバーの経歴を追う
キャンペーンモード	一連のミッションまたはレースシリーズ

キーポイント：乗り物シミュレーションの特徴 - 真実らしさ（verisimilitude）の追求がジャンルの核心 - 民間／軍用、公式レース／架空レースの分類が基本 - ピュアリスト向けのリアリズムとカジュアル向けの簡略化のバランスが設計の鍵 - キャリアモードやキャンペーンモードが継続的なプレイ動機を提供する

1.3 コアメカニクス：物理モデルと操作感

乗り物シミュレーションのコアメカニクスは**ほぼ全てが物理演算**に基づく。エンターテインメントの価値の多くは「本物の機械を操縦している感覚」から生まれるため、車両の徹底的なリサーチが不可欠である。

操作の妥協点

現実の乗り物を家庭用の入力デバイス（コントローラー、キーボード、マウス）で操作する以上、妥協は避けられない。特にB-52爆撃機のような大型複雑機体では顕著である。ターゲット層がピュアリストかカジュアルかによって、妥協の度合いが変わる。

飛行の簡略化

軍用フライトシムでは以下の簡略化が一般的である。

- ・ **コックピット計器の削減**：実機の膨大な計器を必要最小限に絞る
- ・ **ナビゲーションの簡略化**：天測航法の代わりにマップを提供
- ・ **自動協調飛行（Coordinated Flight）**：通常はエルロンとラダーを個別操作する必要があるが、ジョイスティックの左右で両方を同時制御し、自動的に協調旋回を実現する

速度感の演出

ドライビングシミュレーションでは速度感の表現が極めて重要である。主な手法は以下の通り。

- ・ **路面テクスチャの変化**：道路を均一な黒ではなく、連続的に変化する濃淡で表現する
- ・ **路側のオブジェクト**：木、標識、橋など垂直に立つ物体を配置し、通過感を演出
- ・ **サウンド**：エンジン音、路面ノイズ、風切り音、ドップラー効果
- ・ **スピードメーター/タコメーター**：論理的な速度認知を補助



図2：GT Legendsのトラックサイドビュー。路面テクスチャの変化が速度感を生む

AI対戦相手の設計

対戦相手の多様性を生み出す方法は2つある。

1. **車両性能の差異**：登坂力、旋回性能、最高速度など車両パラメータを変える
2. **ドライバー行動の変動**：「完璧なAI」をベースに、ギアチェンジのタイミングやライン取りにばらつきを持たせる

この2つの組み合わせ（車両特性 x ドライバースキル）が対戦相手のバリエーションを生む。

ダメージモデル

ダメージの扱いは3段階に分けて考えられる。

レベル	方式	適用例
簡易	ダメージなし（衝突時はバウンド）	カジュアル・子供向け
中間	単一HP値で管理	携帯端末向け等
詳細	部位別ダメージ＋性能への影響	本格シミュレーション

キーポイント：コアメカニクス - 物理演算がゲームプレイの基盤。リサーチと妥協のバランスが設計の本質 - 飛行の簡略化（計器削減、自動協調飛行等）でアクセシビリティを確保 - 速度感は視覚（路面・路側物）と聴覚（エンジン音・ドップラー効果）で演出 - AI対戦相手は「完璧からの偏差」で個性を表現する

1.4 その他の乗り物と知的財産権

多様な乗り物

飛行機と自動車以外にも、様々な乗り物シミュレーションが存在する。

- ・ **ボート・船舶**：パワーボートやジェットスキーは自動車と似た速度感を提供するが、流体中を動くため旋回特性が異なる。潜水艦シミュレーション（Silent Hunter IIIなど）は三次元移動と潜望鏡・魚雷発射のドラマ性で人気がある。
- ・ **戦車・メック**：戦車は回転砲塔が特徴的で、移動方向と射撃方向が独立するUIの設計が課題となる。メック（MechWarriorなど）はSFの架空兵器であり、現実の制約に縛られない自由な設計が可能。
- ・ **宇宙船**：実在の宇宙船は動きが遅すぎるため、ほとんどがSF設定。Wing Commanderのような戦闘機タイプとStar Trekのような大型艦タイプに分かれる。



図3：Jetboat Superchamps。ボートレースのサードパーソン視点

知的財産権（IP）

- ・ **軍用車両**：一般消費者向けに販売されていないため、製造元の許可なく描写・シミュレーションが可能
- ・ **自動車**：実在の車名やロゴを使用するにはメーカーからのライセンスが必要。さらに、車両が大破・炎上する描写を制限される場合もある。Interstate '76のように「似ているが別の車」を作る回避策もある

1.5 プレゼンテーション層

カメラモデル

乗り物シミュレーションでは複数のカメラ視点を提供するのが標準的である。

飛行・ドライビング共通の視点： - **パイロット/ドライバー視点**：コックピット内からの一人称視点（デフォルト） - **コックピット除去視点**：計器を省略し、前方を全画面表示 - **チェイスビュー**：車両後方からの追従視点 - **前後左右の外部視点**：四方からの固定視点 - **フリーカメラ**：リプレイ用の自由視点

ドライビング固有の視点： - **トラックサイドビュー**：コース脇の固定カメラ（TV中継風） - **観客席ビュー**：ゴールライン正面 - **飛行船ビュー**：上空からの俯瞰

UIデザインの課題

最大の課題は、実車両の複雑な操作系を家庭用デバイスにマッピングすることである。モーションセンサー付きコントローラー（傾きでステアリング）は有効だが、90度以上の回転は要求すべきではない。フォースフィードバック対応のハンドルやペダルはサポートすべきだが、**標準コントローラーだけでも楽しめる設計**が必須である。

キーポイント：プレゼンテーション層 - 複数のカメラ視点の提供が標準。プレイ用とリプレイ鑑賞用を分ける - 軍用車両はIP問題が少ないが、民間自動車はライセンスが必要 - 特殊コントローラー対応は歓迎されるが、標準入力での快適さが最優先

第2部：建設・経営シミュレーション（Construction and Management Simulations）

2.1 建設・経営シミュレーションとは

建設・経営シミュレーション（CMS）は、経済的制約の中で何かを「建てる」ゲームである。プレイヤーの目標は敵を倒すことではなく、進行中のプロセスの中でより良い建設を行うことにある。

定義： CMSとは、課題の大部分が経済的なもので、成長に関わるゲームである。建設活動が不可欠な要素であり、パターン認識や探索の課題も含まれることがある。物理的な操作やコンフリクトの課題は、他ジャンルとのハイブリッドでない限り回避される。

SimCityが最初の成功したCMSであり、高速アクションや暴力なしでもコンピュータゲームが成功できることを証明した。CMSは通常、**自由建設モード**と**シナリオモード**の両方を備える。

2.2 ゲームの特徴

プレイヤーの役割

CMSにおけるプレイヤーの役割は、現実の職業に直接対応しない場合が多い（市長が個人的に道路を引いたりゾーニングを行ったりはしない）。CMSの魅力は「自分だけのものを作る」という創造の欲求に応えることにある。ティベル川のほとりの小さな集落を壮大なローマに育てる——それはデザイナーのローマでも歴史のローマでもなく、プレイヤー自身のローマである。

進行と勝利条件

CMSの多くは他ジャンルと異なり、レベル間の進行やストーリーを含まない。明確な勝利条件が設定されないことも多く、敗北条件（典型的には破産）を回避する限りゲームは永続する。勝利条件を設ける場合は、自由建設モードよりもシナリオの中で「資産総額50億ドル到達」のような達成目標を設定するのが効果的である。

間接的コントロール

CMSの大半は**間接的コントロール**のゲームである。ウォーゲームでは兵士に直接「ここへ行け」と命令するが（直接的コントロール）、CMSではプロセスを限定的にしか変更できず、試行錯誤を通じて変更の影響を学んでいく。例えば、価格を下げる行為自体は直接的だが、それによる売上増加は間接的コントロールの結果である。



図4：オリジナルSimCity。低性能マシン向けに設計されたため、内部経済が理解しやすい

SimCityの経済モデル

SimCityの基本メカニズムを例に、CMSの構造を理解する。

1. **ゾーニング**：土地を住宅・商業・工業の3種に区分（コスト発生）
2. **人口流入**：ゾーンに住民（Sims）が移り住み、税収が発生（正のフィードバック）
3. **インフラ需要**：電力（発電所・送電線）、交通（道路・鉄道）が必要

4. **公共サービス**：消防署、警察署の建設・維持
5. **人口閾値による要求**：一定の人口に達するとスタジアムや空港を要求される
6. **バランス機構**：インフラやサービスへの支出が正のフィードバックを抑制する

キーポイント：CMSの基本特徴 - 「敵を倒す」のではなく「プロセスの中で建設する」ゲーム - 間接的コントロールが中心。プレイヤーは試行錯誤で影響を学ぶ - 勝利条件なしで永続するデザインも一般的 - 創造の欲求に応えることがCMSの魅力の源泉

2.3 コアメカニクス：リソースフローの設計

CMSは他のどのジャンルよりも多くのリソースを扱う。

主要リソース

リソース	ソース（供給源）	ドレイン（消費先）
資金	住民への課税、金鉱	購入・建設
人口	移住・出生	移出・死亡
食料	農場	住民の消費
労働力	人口	建設活動
土地	環境に存在（探索で取得）	消失しないが有限
建設資材	環境から採取（木材等）	建設で消費
建物	資金＋資材＋労働力で建設	解体・災害で消失

建設メカニズムの2類型

- ・ **購入即配置（Purchase-and-Place）**：資源を消費して即座にオブジェクトが出現する。SimCityはこの方式で、ゾーニングや施設建設が瞬時に行われる。建設が主活動のゲームに適する。

- ・ **計画建設 (Plan-and-Build)** : 建設エリアを指定すると、時間をかけて建設が進む。建設中の建物が半透明で表示されることもある。The Settlersシリーズでは木材や石材を少しずつ運搬する必要があり、リソースフロー管理の負担と引き換えに、より細かいコントロールを提供する。



図5 : Bridge Itでは、実際に荷重を支えられる橋を建設する必要がある

ドレインとメンテナンス

ドレインはリソースをゲームから恒久的に除去する仕組みである。

- ・ **劣化 (Decay)** : 建物や道路が摩耗し、修繕が必要になる (SimCityの道路舗装など)
- ・ **メンテナンスコスト** : 自動化された維持費。プレイヤーは支払うだけだが、建設利益の無限蓄積を防ぐ重要なバランス機構
- ・ **災害 (Disaster)** : 火災、竜巻、モンスター襲来など、プレイヤーに即時対応を迫るイベント



図6：Theme Park World。アドバイザーキャラクターが画面右下でプレイヤーを補助する

アドバイザーとマインドリーディング

CMSでは、プレイヤーが全体を監視しきれない問題を補うツールが重要である。

- ・ **アドバイザー**：問題発生時にポップアップで警告するキャラクター。緊急事態の通知やグローバルな状態の報告を行う。閾値の設定が重要で、低すぎると煩わしくなる。
- ・ **マインドリーディング**：個々のシミュレーション住民をクリックして、内部状態（目標、幸福度等）を確認できる機能

キーポイント：コアメカニクス - リソースの「ソース→コンバーター→ドレイン」の流れがCMSの骨格 - 購入即配置と計画建設の2方式はゲームの性質に応じて選択 - メンテナンスと災害がリソースの無限蓄積を防ぐバランス装置 - アドバイザーやマインドリーディングがプレイヤーの情報負荷を軽減

2.4 ゲームワールドとプレゼンテーション

ゲームワールドの設計

CMSはプロセスのゲームだが、プロセスを画面上に意味ある形で表示し、プレイヤーの想像力を刺激する必要がある。多くのCMSは屋外空間を上空から見下ろす構成を取る。

- ・ **2D/2.5D**の世界が多く、3Dエンジンを使用していても視点はアイソメトリック（斜め見下ろし）が一般的。水道管を敷設する際に地下鉄との交差を気にする必要がないなど、第三次元の省略がプレイヤーの計画に集中する助けとなる。
- ・ 3D環境ではズームイン/アウトと自由なカメラ移動が可能で、全体俯瞰とローカル問題への注目を切り替えられる。



図7：Capitalism II。SimCity風の都市ビューと経営要素を組み合わせた大規模ビジネスシミュレーション

プレゼンテーション層

CMSの視覚表現は**創造性と成長**のテーマにふさわしい明るさを持つべきである。霧や暗闘、影は不要で、ベルギーの漫画家エルジェの「リーニュ・クレール」スタイル（明確な線、フラットな影、均一な照明）が参考になる。

インタラクションモデル

プレイヤーは通常**マルチプレゼント**（世界全体を見渡せる状態）であり、アバターは持たない。ただしDungeon Keeperのように、一時的にクリーチャーの視点で歩き回れるウォークスルーモードを提供するゲームもある。これは娯楽性を高めるが、経営面では不要な機能である。

ハイブリッドゲーム

CMSとウォーゲームを融合したハイブリッドゲームも存在する。Civilization、Dungeon Keeper、The Settlersなどが代表例である。

ハイブリッド設計の原則：**経済シミュレーションを先に設計し、その後に他の要素を追加する**。経済設計の誤りは他の全ての要素に波及するためである。例えば、武器生産が速すぎると、プレイヤーは戦略より物量で敵を圧倒してしまう。

純粋ビジネスシミュレーション

建設を伴わず、経済プロセスのみを扱うジャンルもある。Hollywood Mogul（映画ビジネス）やMr. Bigshot（株式市場）などが例である。数値中心のゲームプレイを視覚的に魅力あるものにすることが設計上の最大の課題で、アニメーション、ナレーション、音楽などの演出が重要となる。



図8：Mr. Bigshot。建設要素のない純粋なビジネスシミュレーション



図9：Capitalism IIでは製品とその原材料の視覚化が経営判断を支援する

キーポイント：ゲームワールドとプレゼンテーション - アイソメトリック視点で2D/2.5Dの世界を見下ろすのが定番 - 明るく清潔な視覚スタイルが創造性のテーマに合致 - ハイブリッド設計では経済シミュレーションの設計を最優先にする - 純粋ビジネスシムでは数値のゲームプレイを視覚的に魅力的にする工夫が必要

まとめ：2つのシミュレーションジャンルの比較

比較項目	乗り物シミュレーション	建設・経営シミュレーション
核心的体験	操縦の真実らしさ	建設と成長の創造性
主要な課題	物理的操作・反射神経	経済的判断・リソース管理
コアメカニクス	物理演算モデル	リソースフロー（ソース/ドレイン）
コントロール	直接的（車両を直接操作）	間接的（プロセスに影響を与える）
視点	一人称／チェイスビュー	俯瞰／アイソメトリック

比較項目	乗り物シミュレーション	建設・経営シミュレーション
勝利条件	レース順位、ミッション達成	多くの場合なし（敗北条件のみ）
対象プレイヤー	ピュアリスト～カジュアル	創造的実験を好むプレイヤー

両ジャンルに共通するのは、**シミュレーション（模擬）としての説得力**が求められることである。乗り物シムでは物理的な真実らしさが、CMSでは経済プロセスの論理的な一貫性が、それぞれプレイヤーの没入感を支えている。

本教科書は Ernest Adams 『Fundamentals of Game Design』 第17章・第18章に基づく

ewpage

第14回：アドベンチャーゲームとパズル・人工生命ゲーム

1. アドベンチャーゲーム

1.1 アドベンチャーゲームとは何か

「アドベンチャーゲーム」という名称は、1970年代に開発されたテキストゲーム Adventure（別名 Colossal Cave）に由来する。現在のアドベンチャーゲームは、その直系の子孫にあたるが、当時にはなかった多くの機能を備えている。

定義：アドベンチャーゲーム プレイヤーが操作する主人公キャラクターについてのインタラクティブな物語である。ストーリーテリングと探索がゲームの本質的要素であり、パズル解きと概念的チャレンジがゲームプレイの大部分を占める。戦闘、経済管理、アクションチャレンジは最小限か存在しない。

ゲーム内の関係性は数値的ではなく象徴的であり、経済システムの最適化はゲーム体験に含まれない。この点がRPGとの重要な違いである。

1.2 アドベンチャーゲームの発展

- ・ **テキスト時代**：MITの学生が Adventure に触発されて Zork を開発し、Infocom 社を設立
- ・ **グラフィカル時代**：LucasArtsとSierra On-Lineがジャンルを牽引。Myst は長年 PC史上最も売れたゲームだった
- ・ **現在**：3Dグラフィックスで臨場感が向上。「死んだジャンル」ではなく、宣伝が少ないだけで健在

1.3 アクションアドベンチャー

3Dハードウェアの登場により、アクションとアドベンチャーのハイブリッドである**アクションアドベンチャー**が生まれた（例：Indiana Jones、Zelda シリーズ）。純粋なアドベンチャーよりテンポが速く、身体的・概念的チャレンジの両方を含む。現在は伝統的アドベンチャーより人気が高い。

リプレイ性の問題：パズルに唯一の解がある場合、再プレイの価値が低い。解法に選択肢を与えストーリーに影響させることで改善できる。ただし実際には多くのプレイヤーはゲームを完了しないため、大きな問題にはならない。

1.4 ゲームの特徴

舞台設定と感情表現

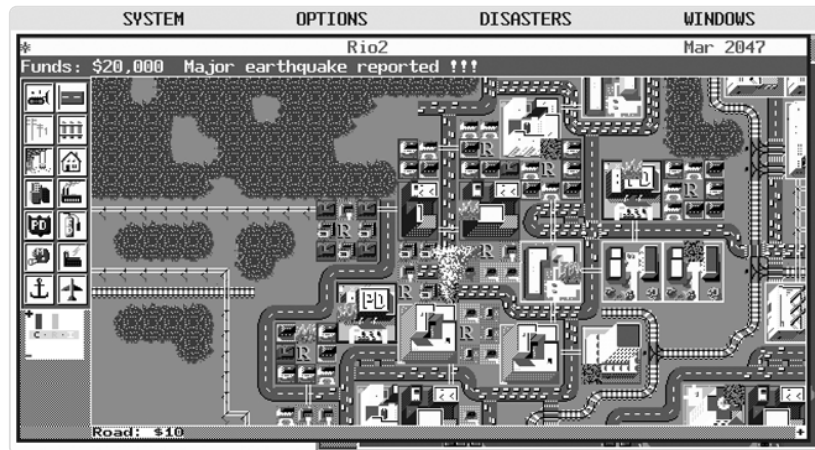
アドベンチャーゲームでは舞台設定が他のどのジャンルよりもエンターテインメント価値に貢献する。ペースが遅いことで、独特の感情的トーンを持つ世界を創造できる（例：Phantasmagoria のホラー、Myst のシュールさ、Shadow of the Colossus の壮かさ）。

インタラクションモデルとカメラ

アドベンチャーゲームは常にアバターベースのインタラクションモデルを使用する。カメラモデルには3つの主要な方式がある：

カメラモデル	特徴	代表例
コンテキスト感応型	アバターの現在位置に最適なカメラアングルを自動選択。映画的な演出が可能	Leisure Suit Larry、A Vampyre Story
一人称視点	世界への没入感が最も高いが、アバターが見えない。アクション寄りのプレイを誘発しやすい	Myst
三人称視点	アバターを常に表示。アクションアドベンチャー	Indiana Jones、Gabriel Knight 3

カメラモデル	特徴	代表例
	で一般的。カメラ操作の自由度が重要	



コンテキスト感応型カメラの例 - Leisure Suit Larry



A Vampyre Storyのカメラ演出

ストーリーと空間構造

アドベンチャーゲームはストーリーを空間にマッピングする。その構造は時代とともに進化してきた：

1. **初期型（探索重視）**：広大な空間を自由に歩き回る。ストーリーの進行感は薄い
2. **章立て型（ストーリー重視）**：ゲームをチャプターに分割し、次章に進むと後戻りできない。フォールドバック・ストーリー構造と同等

3. **アクションアドベンチャー型**：より直線的で、FPSに近い空間構造。探索よりも戦闘チャレンジを重視



ストーリー駆動型アドベンチャーの構造図

探索とインベントリ

プレイヤーは探索可能な領域でパズルを解き、新領域の開放やストーリーの進行を得る。**インベントリ管理**はほとんどの作品で必須であり、ポップアップ式ボックスの固定サイズで携帯量を自然に制限する方法が一般的である。

1.5 チャレンジ（パズル）の種類

アドベンチャーゲームのチャレンジの大部分は、水平思考（ラテラルシンキング）で解く概念的パズルである：

- ・ **鍵と扉のパズル**：進行を妨げる障害物（扉）と、それを除去するオブジェクト（鍵）
- ・ **謎の機械の操作**：つまみを操作して正しい表示を出す組み合わせ錠のようなパズル
- ・ **到達困難なオブジェクトの入手**：見えるが手が届かない物を、工夫して入手する
- ・ **人物の操作**：人物の弱点を会話で探り、説得・気をそらすなどして通過する
- ・ **迷路の攻略**：控えめに使い、必ず観察力で発見できる手がかりを含める
- ・ **暗号解読、記憶パズル、収集パズル、探偵的推理、社会的問題の理解**

設計原則：パズルには複数の論理的解法を許容すること。プレイヤーが論理的に試みた方法が理由なく機能しない場合、フラストレーションが生じる。

1.6 避けるべき設計上の問題

問題	説明
試行錯誤でしか解けないパズル	手がかりのない組み合わせ錠。プレイヤーは全パターンを総当たりするしかない
概念的な飛躍（ノンセクイター）	「ソンプレロをブルドーザーに被せる」式の非論理的解法
非論理的空間	北に進んでBに着くが、南に戻ってもAに戻らない。手がかりなしの試行錯誤になる
外部知識を要求するパズル	特定の映画やTV番組の知識がないと解けない
正しいピクセルをクリックするパズル	画面上の極小の領域をクリックさせる怠惰な設計
逆順パズルの多用	パズルより先に解法（鍵）を見つけてしまい、チャレンジにならない
FedExパズルの多用	物をA地点からB地点に運ぶだけの単調な繰り返し

1.7 プレゼンテーション層

アドベンチャーゲームは、プレイヤーがコンピュータを使っていることを隠すよう努める。UIは直感的でシンプルであるべきだ。

- ・ **移動方式：**ポイント＆クリック（間接制御、伝統的標準）とダイレクトコントロール（直接操作、アクションアドベンチャーの標準）

- ・ **オブジェクト識別**：動的ハイライト（カーソル通過で反応）が主流。ハント＆クリックは非推奨
- ・ **操作方式**：ワンボタンアクション（汎用の「使う」ボタン）またはメニュー駆動（「見る」「話す」「触る」のアイコン選択）



The Longest Journeyのポップアップメニュー（目・口・手のアイコン）

キーポイントまとめ（アドベンチャーゲーム） - アドベンチャーゲームはインタラクティブな物語であり、探索・パズル・ストーリーテリングが核心 - アクションアドベンチャーはアクション要素との混合型で、現在より主流 - 舞台設定が他のどのジャンルよりも重要な役割を果たす - パズル設計では、論理的な複数解法の許容と、避けるべきアンチパターンの理解が不可欠 - UIは没入感を損なわないシンプルさが求められる

2. 人工生命（A-Life）ゲーム

2.1 人工生命ゲームとは

人工生命（A-Life）は、生物学的プロセスをモデル化するコンピュータサイエンスの一分野であり、特に**創発特性**（emergent properties）——複雑なシステムの相互作用から予期せず生じる性質や行動——の研究を行う。商用A-Lifeゲームは、ユニークな生物の集団を管理・育成することに焦点を当てる。

コンウェイのライフゲーム

コンウェイの Life は2次元グリッド上のセル・オートマトンで、3つの単純なルール（死亡・生存・誕生）から「グライダー」等の創発的パターンが生まれ、A-Life研究の原点となった。



コンウェイのライフゲーム

2.2 人工ペット

人工ペットはA-Lifeゲームの一カテゴリーで、コンピュータやモバイルデバイス上に住むシミュレートされた動物である。

- ・ **例**：Nintendogs（リアルな犬の世話）、Tamagotchi（キーチェーン型の簡易電子ペット）
- ・ **設計上の特徴**：
 - ほぼ常にかわいらしく、訓練・世話・観察がゲームプレイの中心
 - 豊富なAI（多様な刺激と行動レパートリー）が必要
 - 感情やムードを持ち、プレイヤーが観察で状態を読み取り、働きかけで影響を与えられる
 - 学習能力が必須（学習が遅すぎても速すぎても問題）
 - 明確な勝利条件がないため、厳密にはゲームよりも**ソフトウェア玩具**に近い



Nintendogs

2.3 The Sims : 最も成功したA-Lifeゲーム

The Sims は郊外の家庭をシミュレートする「仮想ドールハウス」で、人工ペットとCMS（建設・管理シミュレーション）の特徴を併せ持つ。

ニーズシステム

各シムには6つのニーズ（**空腹**、**衛生**、**膀胱**、**体力**、**楽しさ**、**社交**）があり、マズローの欲求階層説に基づく。低次の生理的ニーズが優先され、充足後に高次ニーズへ移行する。

スキルと性格

- ・ **スキル**（料理、修理、カリスマ等6種）は就職・収入に影響し、時間管理がゲームの核心
- ・ **性格特性**は親密度計算に使用。「我慢できない」「類は友を呼ぶ」「正反対が惹かれ合う」の3ルールで関係性を決定

成功の理由

1. **創造性**：家の建築からスキン制作、写真付きストーリー作成まで幅広い創作活動
2. **人間関係の重視**：人間関係をテーマにし、幅広い層（特に女性プレイヤー）に支持された

2.4 ゴッドゲーム

ゴッドゲームは、プレイヤーが限定的な力を持つ神の役割を担うサブジャンルである。

- ・ 俯瞰視点で民衆を間接制御し、ライバルの神と対決する
- ・ 民衆の数に比例して「マナ」（神の力）が増加し、土地整備や自然災害（洪水、地震等）に消費する
- ・ 人口→マナ→人口の正のフィードバックを、繁殖遅延・コスト増大で制御
- ・ 代表作：Populous、Dungeon Keeper、Black & White

2.5 遺伝的A-Lifeゲーム

個体ではなく集団を世代にわたって管理するA-Lifeゲーム。代表作は Creatures シリーズ。

- ・ **ゲノム設計**：可変特性をアレル（対立遺伝子）として定義し、優性・劣性の仕組みで多様性を維持
- ・ **突然変異と自然淘汰**：未成熟期→成熟期の段階を設け、不適応個体を繁殖前に淘汰。致死的突然変異は避ける
- ・ **Spore**：人工ペット+RTS+ゴッドゲームのハイブリッドであり、遺伝的A-Lifeゲームではない



The Sims 3の画面

キーポイントまとめ（人工生命ゲーム） - A-Lifeゲームは生物学的プロセスのシミュレーションに基づき、創発特性が核心概念 - 人工ペットは豊富なAIと学習能力が重要で、勝利条件のないソフトウェアトイ的性質を持つ - The Sims の成功はマズ

ローの欲求階層に基づくニーズシステムと、人間関係シミュレーションの魅力にある

- ゴッドゲームはマナ経済と間接制御が特徴
- 遺伝的A-Lifeゲームではゲノム・突然変異・自然淘汰の設計が鍵

3. パズルゲーム

3.1 パズルゲームの定義

パズルゲームとは、パズル解きが主要な活動であるゲームである。パターン認識、論理的推論、プロセスの理解などが求められる。ただし、ランダムなパズルの寄せ集めではなく、関連するチャレンジの変奏として提示されるべきである。

- ・ Tetris のような厳しい時間制約下のパズルは、論理よりも身体的協応が求められるため、アクションパズルに分類される
- ・ 商業的成功のためには、**挑戦的だが難しすぎず、視覚的に魅力的で、十分なプレイ量が必要**

3.2 Scott Kimの8ステップ

パズルデザイナーのスcott・キムが提唱したパズルゲーム設計の8段階：

ステップ	内容
1. インスピレーション	他のゲーム、芸術（M.C.エッシャーなど）、物語、操作のダイナミクス（切り替え、スライド、接続など）からアイデアを得る
2. 単純化	本質的な「厄介な核心スキル」に集中し、無関係な要素を排除。ピースを標準化し、操作を最小限に
3. 構築セット作成	プロトタイプを作成してルールを検証。プレイヤーが自作パズルを作れる機能も検討

ステップ	内容
4. ルール定義	ボード（グリッド？ネットワーク？）、ピース（形・属性）、手（許可・禁止・副作用）、勝利条件を決定
5. パズル構築	問題から解へのパスに分岐と行き止まりを配置。プレイヤーに「わかった！」という洞察の瞬間を与える
6. テスト	難易度、楽しさ、代替解法、ルールの誤り、UIの使いやすさを検証
7. 配列設計	易→難の単調増加ではなく、 のこぎり歯型 （難しくなった後に易しい問題を挟む）が効果的。メタパズルでモチベーションを維持
8. プレゼンテーション	サウンド、グラフィック、アニメーション、UI。パズルが90%の戦いなので、まずパズルを完成させてから取り組む

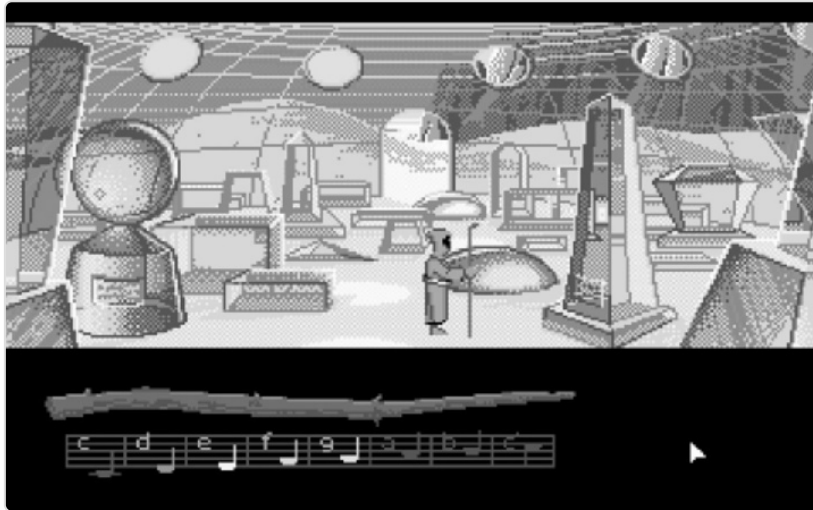
3.3 コンピュータがパズルにもたらすもの

コンピュータは物理的に不可能なパズルを実現し、以下の利点をもたらす：

- ・ **非物理的操作**（色の変更、同時制御）、**ルールの自動強制**、**アンドゥ機能**
- ・ **計算機能**（自動生成、ヒント提供）、**チュートリアル**、**体験の構造化**
- ・ **演出効果**（サウンド・アニメーション）、**オンラインプレイ**（対戦・コミュニティ）

3.4 勝利条件の確認における注意

プレイヤーが**勝利条件を満たしたかどうか**だけを確認し、**想定した方法で解いたか**は問わないことが重要である。Interstate '76 では、設計者が隠しスロープを想定したが、プレイヤーは地雷爆発で壁を越える方法を発見した。しかし判定がスロープ使用を確認していたため正当な脱出が認められなかった。このような設計は避けるべきである。



The Incredible Machine

キーポイントまとめ（パズルゲーム） - パズルゲームはパズル解きが主活動で、関連性のあるチャレンジの変奏として設計する - Scott Kimの8ステップは、インスピレーションから単純化、プロトタイプ、テスト、配列までの体系的設計手法 - のこぎり歯型の難易度曲線が、単調増加より効果的 - コンピュータはルール強制・アンドゥ・自動生成など、物理パズルにない利点をもたらす - 勝利条件の判定は「結果」を見るべきで、「方法」を限定すべきではない

本講のまとめ

アドベンチャーゲームはインタラクティブな物語・探索・パズルが核心で、舞台設定と没入感を重視したUIが鍵。**人工生命ゲーム**は創発特性を活用し、The Sims のニーズシステムと人間関係シミュレーションが成功例。**パズルゲーム**はScott Kimの8ステップによる体系的設計と、公平な勝利条件判定が重要である。

参考：Fundamentals of Game Design, Chapter 19 “Adventure Games” & Chapter 20 “Artificial Life and Puzzle Games”

第15回：オンラインゲームと特定層向けデザイン

第I部：オンラインゲーム（Chapter 21）

1. オンラインゲームとは何か

オンラインゲームとは、プレイヤーのマシンがネットワークで接続された**マルチプレイヤー分散型ゲーム**を指す。一つの画面を共有するローカルマルチプレイヤーゲームとは本質的に異なる。オンラインゲームはジャンルではなく**テクノロジー（技術基盤）**であり、プレイヤー同士を接続する仕組みである。したがって、どのジャンルのゲームにも適用可能だが、特有の設計上の課題が存在する。

注意： オンラインゲームとオンラインギャンブルは異なる産業であり、本章では扱わない。

2. オンラインゲームの利点

プレイヤーの社会的交流

オンラインゲームは社会的インタラクションの機会を提供する。特に女性はソーシャルな要素に惹かれやすく、シングルプレイヤー市場よりもオンラインゲーム市場での女性比率が高い。デザイナーはゲーム設計者であると同時に**社会的設計者（ソーシャルアーキテクト）**でもある。オンラインゲームはプレイヤーを導く体験ではなく、社会的状況を培養する「ペトリ皿」のようなものであり、何が起きるかを事前に予測するのは極めて困難である。

人間の知性がAIを代替する

シングルプレイヤーゲームでは高度なAIが必要だが、オンラインゲームではプレイヤー同士が対戦するため、AIの負担を大幅に軽減できる。Guild Warsのようにプレイヤーがチームで協力してAI敵と戦うゲームも存在するが、多くの場合、プレイヤー自身が知性を提供する。

ネットワークプレイの優位性

ローカルプレイでは画面共有による情報隠蔽の困難さ、UI要素の重複、参加人数の制限などの問題がある。オンラインプレイではこれらを解消し、各プレイヤーが固有の視点と情報を持てる。数万人規模の同時接続も可能であり、いつでも対戦相手が見つかる。

キーポイント： オンラインゲームの最大の利点は、社会的交流の場を提供すること、AIの代わりに人間の知性を活用できること、そしてローカルプレイの制約を解消できることである。

3. オンラインゲームの課題

技術的課題

通信モデル： 主に2つのモデルがある。 - **クライアント/サーバ型：** サーバがゲームエンジンを実行し、クライアントは情報を表示する - **ピアツーピア型 (P2P)：** プレイヤーのコンピュータが直接通信する。ホストが切断した場合の自動ホストマイグレーションが必要

レイテンシ (遅延)： インターネットは冗長性を重視した設計であり、データの到達時間を保証しない。高速回線のプレイヤーが有利になるため、ターンベース制にするか接続速度によるマッチメイキングが考えられる。

パケットロスと順序乱れ： 欠落や順番の入れ替わりに対処するため、各パケットにシリアル番号を付与し、再送要求の仕組みを備える必要がある。

没入感の維持が困難

オンラインゲームでは、ロールプレイに没入しないプレイヤーが現実世界的话题を持ち込むことが避けられない。World of Warcraftではロールプレイ推奨サーバを用意して対応している。

不正行為（ミスビヘイビア）

匿名の見知らぬ人とプレイすることで、単純な無礼から嫌がらせ、チート、詐欺まで様々な問題が生じる。特に子供が参加するゲームでは安全な環境の確保が法的義務ともなりうるため、カスタマーサービス担当者による監視が必要になる。

継続的なコンテンツ供給

オフラインゲームと異なり、オンラインゲームは広告収入・課金・サブスクリプションで収益を得る。プレイヤーの関心を維持するため、新コンテンツを継続的に提供しなければならない。

カスタマーサービスの負荷

オフラインゲームよりも遥かに多くのサポート人員が必要で、技術的問題だけでなくUI説明、コンテンツに関する質問、不正行為者の調査・処罰なども求められる。

キーポイント： レイテンシ、パケット問題、チート対策、コンテンツ継続供給、カスタマーサービスなど、オンラインゲームの運営はオフラインゲームに比べて格段に複雑かつ高コストである。

4. オンラインゲームの設計上の問題

途中参加と途中離脱

途中参加への対応策： - 頻繁に新しいマッチを開始し、ロビー（待機エリア）を設ける - 勝利条件を撤廃し、個人の達成に焦点を当てる（永続世界的方式） - 経験者と初心者 of 直接対決を抑制し、スキルマッチングを導入する - 直接対決は合意制にする

途中離脱への対応策： - 離脱者は没収（信頼性の高いネットワーク環境向け） - 没収より軽いペナルティ（EverQuestではアバターが1分間残留） - その時点でリードしているプ

レイヤーの勝利 - 引き分け扱い／切断記録として保存 - レフェリー制度（World Cyber Gamesの方式）

リアルタイムとターンベース

リアルタイムは自由度と没入感が高いが、戦略ゲームをアクションゲーム化する傾向がある。ターンベースでは、プレイヤー数制限（4～5人が上限）、ターン時間制限、タイムアウト時のデフォルト行動、待機中の別活動許可が重要である。

チャット機能

チャットはソーシャルニーズの中核だが、不正行為の温床にもなる。対策として以下がある： - **限定コンテンツ**： 定型文のみ許可（Mario Kartの方式） - **不適切語フィルタ**： 不完全で回避されやすいが一定の効果はある - **苦情・警告システム**： 通報ボタンでカスタマーサービスに自動転送 - **無視（イグノア）機能**： プレイヤー主導で特定ユーザーのメッセージをブロック - **モデレーターによる管理**： 最も効果的だが最もコストが高い

共謀（コルージョン）と技術的セキュリティ

共謀はルールに反して対戦相手同士が協力する不正行為であり、完全な防止は不可能である。デザイナーは秘密情報の共有、資産の裏での受け渡し、故意の敗北（八百長）の可能性を検討し、影響を最小化すべきである。

技術的セキュリティでは以下の原則が重要： - **暗号化された通信プロトコル**を使用する - **ハートビート機構**で接続状態を監視する - **機密データはプレイヤーのPCに保存しない**（サーバ側管理） - **プレイヤーが持つべきでないデータを送信しない**（戦場の霧など） - **機密処理はクライアントに任せない**（戦闘結果の判定はサーバで行う）

キーポイント： 途中参加・離脱、チャット管理、チート・共謀対策は、オンラインゲーム設計において不可避の課題であり、ゲームの性質に応じた適切な対処法を選択する必要がある。

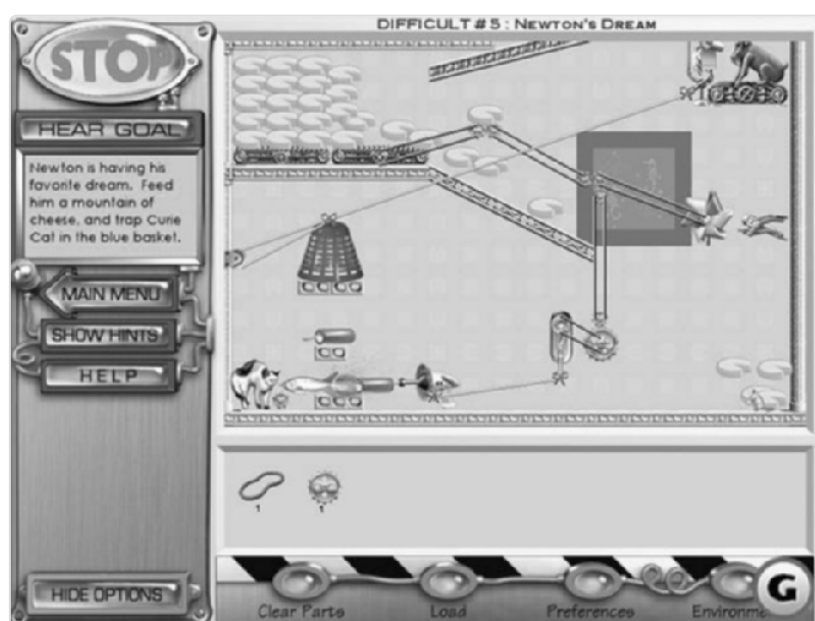
5. 永続世界 (Persistent World / MMOG)

永続世界とは、World of Warcraft、EverQuestなどに代表される、プレイヤーのアバターの状態がセッション間で保持される恒常的なオンライン環境である。1978年以来のテキストベースMUD (Multi-User Dungeon) に起源を持つ。

通常のゲームとの違い

要素	通常のゲーム	永続世界
ストーリー	明確な物語構造と結末	クエストはあるが最終的な結末はない
プレイヤーの役割	ストーリーに制約される	自由に選択・変更できる
ゲームプレイ	反応的 (課題への対応)	表現的・能動的 (自ら行動を選択)
勝利条件	明確に設定される	存在しない (個人の達成が目標)

永続世界では、秘密を保つことが不可能であり (先行プレイヤーが後続に情報を伝える)、エマージェントナラティブ (創発的物語) が核となる。優れた物語はプレイヤー間の相互作用から生まれるが、これはプレイヤーのロールプレイ能力に依存する。



Bartleの4タイプのプレイヤー分類

Richard Bartleの4タイプのプレイヤー

MUD研究者Richard Bartleは、永続世界のプレイヤーを以下の4タイプに分類した：

タイプ	志向	行動
Achiever（達成者）	世界に働きかける	目標達成、レベルアップに注力
Explorer（探検者）	世界と交流する	未知の場所や仕組みの発見を好む
Socializer（社交家）	他プレイヤーと交流する	コミュニケーション、関係構築を重視
Killer（殺戮者）	他プレイヤーに働きかける	他者への支配や競争を求める

健全なコミュニティにはこれら4タイプの適切なバランスが必要であり、一つのタイプを優遇・抑制すると他のタイプの人口にも連鎖的に影響する。

ワールドモデル（世界の設計パターン）

Raph Kosterが提唱した5つの基本モデル： 1. **スカベンジャーモデル**： 収集・宝探し中心 2. **ソーシャルモデル**： ロールプレイと社会的達成が中心 3. **D&Dモデル**： 環境との戦闘・クエスト中心（最も普及） 4. **PvPモデル**： プレイヤー間の対戦で成長（EVE Onlineなど） 5. **ビルダーモデル**： 世界そのものを構築する表現型

アバターの死とプレイヤーキラー問題

永続世界における死のペナルティには段階がある： - **永久死**： 最も厳しく、長期プレイには不向き - **属性低下による復活**： 能力値の一時的減少 - **所持品喪失による復活**： 装備やアイテムの一部を失う

プレイヤーキラー（PK）問題 は永続世界設計における最大の論争点の一つである。Ultima Onlineの初期では無制限PvPが無秩序な暴力を招き、最終的にPvP許可/禁止の別サーバに分割された（約80%が非PvPを選択）。対策として、犯罪フラグ制、レピュテーション制、PvPスイッチ制、ファクション制PvP、完全非PvPなどが開発されている。

時間と経済の設計

永続世界では時間の巻き戻しや早送りが不可能であり、全プレイヤーが同じ時間軸で進行する。経済設計も重要で、Ultima Onlineの閉鎖経済は、プレイヤーの買い溜め行動を予測できず深刻なインフレを引き起こした。

キーポイント： 永続世界の設計では、プレイヤー分類の理解、適切なワールドモデルの選択、PK問題への対策、時間と経済の管理が不可欠である。プレイヤーは仮想世界に現実以上の公平さを期待するため、その期待を適切にマネジメントすることが運営の鍵となる。

第II部：特定層向けデザイン（付録A）

ゲームのコンセプト段階ではターゲットオーディエンスの選定が不可欠である。この節では、ゲーム産業が長年十分にサービスを提供してこなかった3つのグループ——成人女性、子供一般、女兒——と、障害を持つプレイヤーへのアクセシビリティについて扱う。

6. 成人女性へのリーチ

ジェンダーインクルーシブネス

米国の調査では、成人女性ゲーマー（34%）は10代男子ゲーマー（18%）より多い。男女の嗜好差はポップ心理学が主張するほど大きくなく、異性愛女性の最大50%が子供時代に「おてんば」だったと自認する調査もある。女性プレイヤーを引きつけるために「女性向けゲーム」を作る必要はなく、**特定の性別を遠ざける要素を排除する**ことが重要である。

女性が敬遠しがちな要素： - 過度に性的なキャラクターデザイン - 単調で反復的なプレイ - 意味のある目標のないプレイ（ハイスコアだけでは不十分） - シングルプレイの孤立感

男女の傾向の違い（西洋の研究に基づく一般論）

- ・ **学習スタイル：** 女性は事前説明を好み、男性は試行錯誤に積極的
- ・ **リスクへの態度：** 女性は計算できないリスクを回避する傾向がある

- ・ **社会性**： 女性は人間関係やソーシャルな体験を重視する
- ・ **紛争解決**： 女性は暴力に文脈や正当性を求め、横方向の思考で解決策を探す
- ・ **知的チャレンジ**： 女性はパズルゲームや洗練された解決策を好む
- ・ **アバターへの愛着**： 女性はアバターを自己表現の手段として捉え、カスタマイズを重視する

設計原則：「**女性はマーケットであり、ジャンルではない**」 これらの傾向を「女性向け機能」として実装するのではなく、本質的に面白いゲームを設計しつつ、これらの知見をデザイン判断の参考にすべきである。

キーポイント： 女性向けゲームを独立ジャンルとして捉えるのではなく、排除的な要素を取り除き、社交性・物語性・カスタマイズ性を充実させることで、より幅広い層にアピールできる。

7. 子供向けデザイン

子供向けゲームは年齢層によって大きく異なり、以下の4つのカテゴリーが一般的である：
 - **幼児・就学前（3～6歳）** - **小学校低学年（5～8歳）** - **小学校高学年（7～12歳、トゥイン）** - **中高生（13歳以上、ティーン）**

設計上の考慮点

- ・ **手と目の協応**： 幼児は運動能力が未発達、10代はしばしば大人を上回る
- ・ **認知負荷**： 低年齢向けではパズルの要素数と推論の連鎖を少なくする
- ・ **頻繁な報酬**： 子供は進捗だけでは不十分で、成功を示すフィードバックを頻繁に必要とする
- ・ **視覚デザイン**： 不要な情報を排除したシンプルで焦点の明確なUI
- ・ **言語の複雑さ**： 年齢相応の語彙と構文を使う（サン＝テグジュペリ『星の王子さま』は好例）
- ・ **適切なコンテンツ**： 子供の関心に寄り添う（ハリーポッターの成功はこの原則の好例）

Carolyn Handler Millerの「7つの死のキス」

子供向けエンターテインメントで避けるべき7つの誤解：

#	死のキス	説明
1	甘いものが好き	幼児以上は世界が甘くないことを知っている
2	良いものを与えよ	娯楽に説教を混ぜてはいけない
3	とにかく楽しませればよい	子供も質の低いコンテンツには満足しない
4	常に安全に	危険や恐怖の要素を排除すると退屈になる
5	子供はみんな同じ	年齢による興味・能力の差は大きい
6	全部説明せよ	子供は試行錯誤を楽しむ、長い説明は不要
7	健全なキャラクターを	「健全」は「退屈」と同義、個性のあるキャラクターを



The Sims 3：社交性・表現性・カスタマイズ性を重視したデザインは、女性や幅広い層に支持されている

キーポイント： 子供向けデザインでは年齢層ごとの発達段階を理解し、大人の先入観（甘さ、教育的内容、安全第一）を押しつけず、子供自身の関心と能力に合わせた体験を提供することが重要である。

8. 女兒向けゲーム

ゲーム産業は長年男性主導であり、女兒向けゲームはほとんど作られてこなかった。1990年代半ばから市場開拓が進み、UbisoftのImagineシリーズや、Barbie、Bratz、Nancy Drewなどのライセンスタイトルが成功を収めている。

Mattelのアプローチ

Barbieの成功はターゲット層（4～8歳）の明確な絞り込みに基づいている。8歳以降は興味が変わるため、ワンサイズ・フィッツ・オール方式は取らない。

Jesyca Durchinの知見（元Mattelエグゼクティブプロデューサー）

- ・ 女兒の興味は男児より多様で、毎年のように変化する

- ・ **実証済みの遊びのパターン**（ファッション・グラマー・養育・冒険・アクション・収集・コミュニケーション）に基づいて設計する
- ・ 「もの（stuff）」の収集はスコアに代わるモチベーション
- ・ 美しく現実に根ざした環境が重要
- ・ 感覚的なインターフェース（色彩豊かで音のあるUI）を好む

Kaye Ellingの「5つのC」

C	概念	説明
Characterization	キャラクター性	自己を投影できるアバター
Context	文脈・環境	魅力的で敵意のない環境
Control	コントロール	ゲームに振り回されず、自分が主導権を持つ
Customization	カスタマイズ	アバターやアイテムの豊富なカスタマイズ
Creativity	創造性	自己表現の機会（The Simsの成功の鍵）

よくある誤解の訂正

- ・ 「女兒はコンピュータが苦手」 → 技術的詳細に興味がないだけで、ゲームプレイは楽しむ
- ・ 「女兒は暴力が嫌い」 → 意味のない暴力が退屈なだけで、文脈ある暴力は許容される
- ・ 「女兒は甘いものが好き」 → ミステリーやサスペンスも好む
- ・ 「女兒は怖がり」 → 「不気味（spooky）」は好きだが「怖い（scary）」は嫌い

キーポイント： 女兒向けゲーム設計では、年齢による急速な興味の変化を理解し、実証済みの遊びパターンに基づいて設計する。ステレオタイプを押し付けるのではなく、自己表現・カスタマイズ・収集要素を軸にした魅力的な体験を提供すべきである。

9. アクセシビリティ

障害を持つプレイヤーのためのアクセシビリティは、ゲーム産業が遅れている分野である。身体的障害は視覚、聴覚、運動の3カテゴリーに分類される。

視覚障害への対応

- ・ **低視力**： 高コントラストモード、テクスチャ変更可能なオプション、黒地に黒のUI配色を避ける
- ・ **拡大が必要なプレイヤー**： フォントサイズ変更、複数解像度対応、拡大鏡機能の提供
- ・ **色覚異常**： 男性の約6%に影響する。黄色と緑だけで区別する設計を避け、色以外の手がかり（位置・形状）も併用する。Vischeck（www.vischeck.com）でテスト可能

聴覚障害への対応

- ・ 聴覚イベントに視覚的手がかりを付与（火花、マズルフラッシュなど）
- ・ 音楽・効果音・台詞を個別にボリューム調整可能にする
- ・ コントローラの振動機能を活用する
- ・ 字幕・クローズドキャプション機能を実装する（Half-Life 2が好例）

運動障害への対応

- ・ タイムプレッシャーを軽減し、ゲーム速度の調整を可能にする
- ・ 操作を単純化する（シングルボタン操作対応など）
- ・ コントローラのキーマッピングを完全にカスタマイズ可能にする

キーポイント： アクセシビリティ対応は技術的に難しいものばかりではなく、字幕機能やフォントサイズ変更など低コストで実装できるものも多い。「ゲームは健常者のもの」という前提を疑い、より多くの人が楽しめる設計を目指すべきである。

まとめ：本講のポイント

1. **オンラインゲームはジャンルではなく技術基盤**であり、社会的交流、人間知性の活用、スケーラビリティが強みだが、レイテンシ・チート・コンテンツ供給・カスタマーサービスなど固有の課題を伴う
2. **永続世界（MMOG）** は明確な勝利条件を持たず、プレイヤーの自発的行動と社会的相互作用が体験の核となる。プレイヤータイプのバランス管理、PK問題への対応、経済設計が設計上の主要課題である
3. **ジェンダーインクルーシブデザイン** では、女性を排除する要素を取り除くことが最も効果的であり、「女性向けジャンル」を作ることではない
4. **子供向けデザイン** では年齢層ごとの発達段階を尊重し、大人の思い込みを排除する「7つの死のキス」を避ける
5. **アクセシビリティ** は視覚・聴覚・運動の3領域それぞれに具体的な対応策があり、多くは低コストで実装可能である

本教材は Ernest Adams『Fundamentals of Game Design』Chapter 21 および Appendix に基づく